

GUIDA COMPLETA ALL'INTELLIGENZA ARTIFICIALE

Dal Codice all'Esperto — Edizione 2026

Per sviluppatori junior che vogliono padroneggiare l'IA dalla teoria alla produzione

VOLUME 1 — FONDAMENTA

Come funziona l'IA: dai dati ai modelli

Introduzione al Volume → Come leggere questa guida → Setup dell'ambiente (Python, Jupyter, GPU/Colab) → Cosa ti serve già sapere e cosa imparerai

Cap. 1 — Cos'è davvero l'Intelligenza Artificiale → La differenza tra programmare regole e far imparare dai dati → Mappa dell'IA: ML, Deep Learning, LLM, Agenti — dove si collocano → Cosa riesce (e cosa non riesce) a fare un'IA oggi → Timeline storica: dai perceptron a GPT-4o → *Analogia guida: l'IA come un apprendista che impara dagli esempi*

Cap. 2 — I Dati: il carburante dell'IA → Tipi di dati: tabellari, immagini, testo, audio → Dataset, campioni, feature, etichette — il vocabolario base → Dati di training, validation e test: perché la distinzione è critica → Il problema del bias nei dati e le sue conseguenze reali → Come esplorare un dataset con Python (pandas, matplotlib) → *Esercizio pratico: analisi esplorativa su un dataset reale*

Cap. 3 — Matematica Intuitiva per l'IA → Vettori e matrici senza paura: cosa rappresentano davvero → Il gradiente spiegato con una pallina che rotola in una valle → Probabilità: dal lancio di dadi alla previsione del prossimo token → Entropia e cross-entropy: misurare l'incertezza → Cosa devi calcolare tu vs. cosa fa la libreria → *Ogni formula è preceduta da un'analogia, poi dal codice NumPy equivalente*

Cap. 4 — Come Impara una Macchina → Il ciclo: predici → misura l'errore → correggiti → ripeti → La funzione di perdita (loss): il termometro dell'errore → Gradient descent: scendere verso la soluzione → Learning rate: perché i passi troppo grandi fanno inciampare → Overfitting e underfitting: il nemico silenzioso → Tecniche di regolarizzazione: dropout, L1/L2, early stopping → *Esercizio pratico: implementa gradient descent da zero in 20 righe*

Cap. 5 — Il Neurone Artificiale e le Reti Neurali → Dal neurone biologico a quello artificiale (senza biologia superflua) → Pesi, bias e funzione di attivazione: cosa fa ognuno → Perché serve la non-linearità (con esempio visivo) → Funzioni di attivazione: ReLU, GELU, Sigmoid, Softmax — quando usare quale → MLP: organizzare i neuroni in strati → Forward pass e backward pass passo per passo → *Esercizio pratico: costruisci un MLP da zero con NumPy*

Cap. 6 — Machine Learning Classico → Supervised, Unsupervised, Reinforcement: i tre paradigmi con esempi concreti → Regressione lineare e logistica: la base di tutto → Alberi

decisionali e Random Forest: potenti e interpretabili → XGBoost e Gradient Boosting: perché dominano sui dati tabellari → SVM: l'intuizione del margine massimo → Clustering con K-Means: trovare strutture senza etichette → Metriche di valutazione: accuracy, precision, recall, F1, AUC-ROC → *Esercizio pratico: classifica email spam con scikit-learn, end-to-end*

Cap. 7 — Ottimizzatori e Training Loop → SGD, Momentum, Adam, AdamW: cosa cambia nella pratica → Learning rate scheduling: warmup, cosine annealing, plateau → Batch size: effetti su velocità e generalizzazione → Come leggere le curve di training per diagnosticare problemi → *Esercizio pratico: confronta 3 ottimizzatori sullo stesso problema*

Progetto di Fine Volume 1 → Pipeline ML completa su un dataset reale: dal CSV al modello deployato in locale → Classificazione + valutazione + interpretazione dei risultati con SHAP

APPENDICI (in tutti i volumi)

Appendice A — Glossario completo (ogni termine tecnico spiegato in italiano semplice)

Appendice B — Cheatsheet Python per l'IA (NumPy, pandas, PyTorch, HuggingFace) **Appendice**

C — Risorse aggiornate: corsi, paper fondamentali, community **Appendice D** — Roadmap di studio consigliata con milestone verificabili **Appendice E** — Soluzioni commentate degli esercizi pratici

VOLUME 1 — FONDAMENTA

Come funziona l'IA: dai dati ai modelli

Capitolo 1

Cos'è davvero l'Intelligenza Artificiale

"Un computer non pensa. Ma può imparare a riconoscere schemi così bene da sembrare che pensi."

Prima di cominciare: a chi serve questa guida

Sei uno sviluppatore. Sai scrivere codice, capisci cos'è una funzione, hai già lavorato con API, database o script Python. Ma quando senti parlare di "reti neurali", "modelli linguistici" o "vettori di embedding", hai la sensazione che ci sia un mondo intero che non hai ancora esplorato.

Questa guida è per te.

Non ti chiederà di avere un dottorato in matematica. Non darà nulla per scontato. Ogni concetto nuovo verrà prima spiegato con un'analogia del mondo reale, poi con una formula (quando necessaria, sempre motivata), infine con del codice Python che puoi eseguire subito.

Alla fine dei tre volumi saprai non solo *usare* l'IA, ma *costruirla, valutarla e metterla in sicurezza* in un contesto reale.

Iniziamo dall'inizio.

1.1 La differenza fondamentale: regole scritte vs. schemi appresi

Immagina di dover scrivere un programma che riconosca se una foto contiene un gatto.

Con la programmazione tradizionale, ragioneresti così:

```
# Approccio tradizionale (regole scritte a mano)
def è_un_gatto(immagine):
    if ha_orecchie_appuntite(immagine) and ha_baffi(immagine):
        if ha_coda(immagine) and ha_quattro_zampe(immagine):
            return True
    return False
```

Il problema è immediato: come definisci `ha_orecchie_appuntite()`? Come gestisci un gatto di profilo? O sfocato? O con un cappellino di Natale? Le eccezioni si moltiplicano all'infinito. I

ricercatori ci hanno provato per decenni con sistemi basati su regole, e non hanno mai funzionato abbastanza bene.

L'approccio del Machine Learning capovolge il ragionamento:

```
# Approccio ML (schemi appresi dai dati)
modello = addestra(10_000_foto_di_gatti, 10_000_foto_di_non_gatti)
risultato = modello.predici(nuova_foto) # funziona su foto mai viste prima
```

Invece di scrivere le regole, **mostri al programma migliaia di esempi** e lui trova da solo gli schemi che distinguono un gatto da tutto il resto. Non gli dici *come* riconoscere un gatto: gli mostri *cosa* è un gatto, tante volte, finché impara.

Questa è l'idea centrale di tutta l'intelligenza artificiale moderna: **sostituire le regole scritte a mano con schemi appresi automaticamente dai dati**.

1.2 La mappa dell'IA: orientarsi nel campo

Il termine "Intelligenza Artificiale" è un ombrello enorme sotto cui vivono concetti molto diversi tra loro. Vediamo come si relazionano.

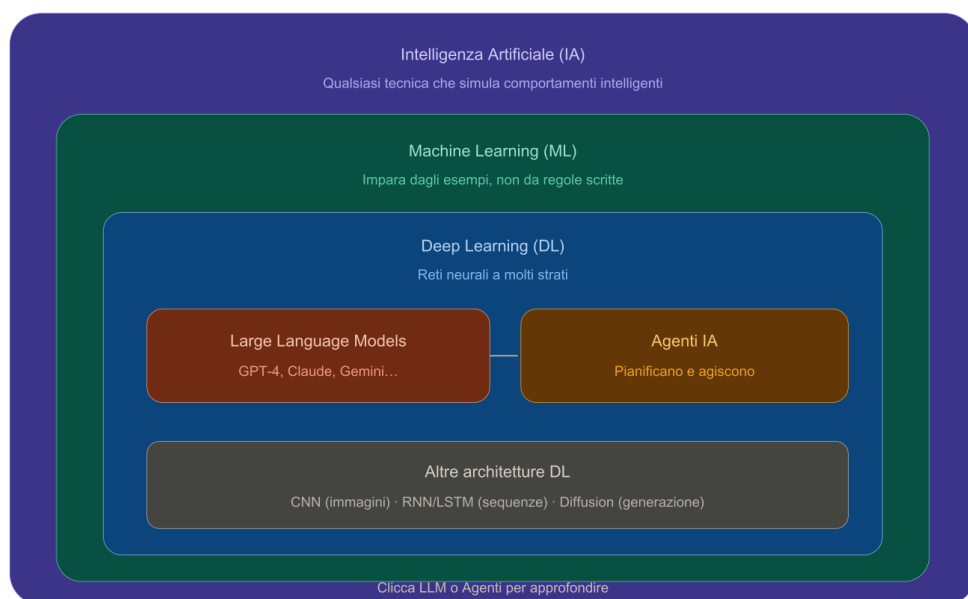


Figura 1.1 — La gerarchia dell'IA: ogni livello è un sottoinsieme del precedente. Gli LLM sono un tipo di Deep Learning specializzato nel linguaggio; gli Agenti usano gli LLM per agire nel mondo.

Intelligenza Artificiale (IA) è il campo più ampio: qualsiasi tecnica che permette a una macchina di simulare comportamenti intelligenti. Include sia i vecchi sistemi basati su regole (come i motori di scacchi degli anni '90) sia le moderne reti neurali.

Machine Learning (ML) è il sottoinsieme dell'IA in cui la macchina impara dai dati senza essere esplicitamente programmata per ogni caso. Tutti i sistemi ML sono IA, ma non tutta l'IA è ML.

Deep Learning (DL) è un sottoinsieme del ML che usa reti neurali con molti strati ("deep" = profondo). Il termine indica la profondità della rete, non la profondità del pensiero. È la tecnologia dietro il riconoscimento di immagini, la traduzione automatica, i modelli vocali.

Large Language Models (LLM) sono un tipo specifico di modelli di Deep Learning addestrati su enormi quantità di testo. GPT-4, Claude, Gemini sono tutti LLM. Sono ottimi con il linguaggio naturale, il codice, il ragionamento testuale.

Agenti IA sono sistemi in cui un LLM non risponde solo a domande, ma pianifica ed esegue sequenze di azioni: naviga il web, scrive file, chiama API. Il modello non è più passivo: agisce nel mondo.

***Analogia utile:** pensa all'IA come alla medicina. La medicina è il campo generale. La chirurgia è una specializzazione. La chirurgia laparoscopica è una tecnica specifica dentro la chirurgia. Allo stesso modo: IA → ML → Deep Learning → LLM → Agenti.*

1.3 Cosa riesce (e cosa non riesce) a fare un'IA oggi

Essere chiari su questo evita sia l'euforia ingiustificata sia la delusione.

Cosa sa fare molto bene

- **Riconoscimento di pattern in grandi quantità di dati:** immagini, audio, testo, dati tabellari.
- **Generazione di contenuti:** testo, codice, immagini, audio sintetico.
- **Classificazione e predizione:** spam o no spam, prezzo di una casa, diagnosi medica.
- **Traduzione e trasformazione linguistica:** tra lingue, tra stili, tra formati.
- **Ragionamento su contesti testuali:** rispondere a domande su documenti, scrivere codice da una descrizione.

Cosa ancora non sa fare bene

- **Ragionamento causale robusto:** confonde correlazione e causazione.
- **Contare e fare aritmetica complessa:** i LLM possono sbagliare 127×456 (ed è per questo che gli agenti usano calcolatrici esterne).
- **Ricordare tra sessioni diverse:** senza memoria esplicita, ogni conversazione ricomincia da zero.
- **Capire il mondo fisico:** non ha esperienza corporea, fatica con intuizioni spaziali e fisiche.
- **Essere sempre affidabile:** "allucinare" — generare informazioni false dette con sicurezza — è un problema reale e ancora aperto.

***Regola pratica per lo sviluppatore:** usa l'IA per i task dove una risposta "abbastanza buona l'80% delle volte" ha valore. Per task dove ogni errore è critico (operazioni chirurgiche, trading automatico, infrastrutture nucleari), serve supervisione umana obbligatoria.*

1.4 Come ci siamo arrivati: la storia in 15 tappe

La storia dell'IA non è una linea retta verso il successo. È una sequenza di entusiasmi, delusioni e rinascite. Conoscerla ti aiuta a capire perché certi approcci esistono e altri sono stati abbandonati.

1950	● Alan Turing — "Computing Machinery and Intelligence" Fondazione
1956	● Conferenza di Dartmouth Fondazione
1958	● Rosenblatt inventa il Perceptron ML classico
1974-80	● Primo AI Winter Crisi
1986	● Rumelhart e Hinton — Backpropagation Deep Learning
1987-93	● Secondo AI Winter Crisi
1997	● Deep Blue batte Kasparov ML classico
1998	● LeCun pubblica LeNet Deep Learning
2012	● AlexNet vince ImageNet Rivoluzione DL
2017	● "Attention Is All You Need" — il Transformer Era LLM
2018	● BERT e GPT-1 Era LLM
2020	● GPT-3 — 175 miliardi di parametri Era LLM
2022	● ChatGPT — 100M utenti in 2 mesi Mainstream
2024-26	● Agenti, MCP, modelli multimodali Oggi

Figura 1.2 — Timeline interattiva della storia dell'IA. Filtra per era e clicca su ogni evento per approfondire.

| Anno | Evento | Perché è importante | |-----|-----|-----| | **1950** | Alan Turing propone il "Turing Test" | Primo tentativo formale di definire cosa significa "pensare" per una macchina | | **1956** | Conferenza di Dartmouth: nasce il termine "AI" | Fondazione ufficiale del campo come disciplina accademica | | **1958** | Frank Rosenblatt inventa il Perceptron | Il primo neurone artificiale funzionante — precursore diretto di tutto ciò che esiste oggi | | **1974-1980** | Primo AI Winter | I finanziamenti si esauriscono: le promesse non si realizzano, i computer sono troppo lenti | | **1986** | Rumelhart e Hinton pubblicano la backpropagation | Finalmente si sa come addestrare reti neurali a più strati — il motore teorico del DL moderno | | **1997** | Deep Blue batte Kasparov a scacchi | Prima sconfitta del campione del mondo contro una macchina: shock culturale globale | | **1998** | LeCun pubblica LeNet | Prima CNN pratica: riconosce cifre scritte a mano con accuratezza quasi umana | | **2006** | Hinton introduce il Deep Belief Network | La "rinascita" del deep learning: si dimostra che reti molto profonde si possono addestrare | | **2012** | AlexNet vince ImageNet con un margine enorme | La GPU + deep learning + dati = rivoluzione. L'errore scende dal 26% al 15% in un anno | | **2017** | "Attention Is All You Need" — il Transformer | L'architettura che sta alla base di tutti gli LLM moderni. Uno dei paper più influenti della storia | | **2018** | BERT e GPT-1 | Il pre-training su larga scala funziona: un modello addestrato su testo generico batte i sistemi specializzati | | **2020** | GPT-3 con 175 miliardi di parametri | Il modello impara a fare cose nuove senza essere riaddestrato (few-shot learning) | | **2022** | ChatGPT — 100 milioni di utenti in 2 mesi | L'IA generativa diventa mainstream: il prodotto tech a crescita più rapida della storia | | **2023-2024** | GPT-4, Claude, LLaMA, agenti autonomi, MCP | Ecosistema esplosivo: modelli open source competitivi, agenti che agiscono nel mondo reale | | **2025-2026** | Modelli di ragionamento, AI Scientist, dibattito AGI | I modelli iniziano a risolvere problemi scientifici nuovi; la domanda su cosa significhi "generale" diventa urgente |

I due AI Winter: perché sono importanti

Tra il 1974 e il 1980, e poi di nuovo tra il 1987 e il 1993, i fondi per la ricerca sull'IA si prosciugarono quasi completamente. I ricercatori avevano promesso troppo, i computer non erano abbastanza potenti, i dati erano insufficienti.

Questo ci insegna tre cose ancora valide oggi:

1. **Il progresso nell'IA è ciclico**, non lineare. Ci saranno probabilmente altre frenate.
2. **Tre ingredienti sono necessari insieme**: algoritmi, dati, potenza di calcolo. Mancarne anche solo uno blocca tutto.
3. **L'hype è pericoloso**: esagerare le capacità attuali crea aspettative che, quando non si realizzano, distruggono fiducia e finanziamenti.

1.5 Le tre categorie di IA: ANI, AGI, ASI

Quando leggi di IA nei giornali, questi tre termini vengono spesso confusi. Eccoli chiariti una volta per tutte.

ANI — Artificial Narrow Intelligence (IA Ristretta) È *tutta* l'IA che esiste oggi, senza eccezioni. Un sistema ANI eccelle in un task specifico ma non sa fare nulla al di fuori di esso. AlphaGo batte i campioni mondiali a Go, ma non sa fare una tazza di tè. GPT-4 scrive codice brillante, ma non può guidare un'auto. Tutti i modelli che userai in questa guida sono ANI.

AGI — Artificial General Intelligence (IA Generale) Un sistema teorico capace di svolgere qualsiasi task cognitivo che un essere umano potrebbe fare, con flessibilità e trasferimento di conoscenza tra domini. Oggi non esiste. I modelli più avanzati mostrano *segni* di generalizzazione cross-domain che sorprendono anche i loro creatori, ma nessuno ha ancora raggiunto la vera AGI.

ASI — Artificial Super Intelligence (Super-Intelligenza) Un sistema teorico che supererebbe l'intelligenza umana in *tutti* i domini. È oggetto di speculazione filosofica e di dibattito etico molto serio. Il "quando" e il "se" sono aperti.

***Nota pratica:** il 99% del lavoro concreto che farai come sviluppatore riguarda la ANI. Concentrati su quella.*

1.6 Il vocabolario minimo indispensabile

Prima di andare avanti, assicuriamoci che queste parole abbiano un significato preciso per te. Le incontrerai decine di volte nei prossimi capitoli.

| Termine | Significato semplice | Esempio | |-----|-----|-----| | **Modello** | Il "programma" che ha imparato dai dati | Un file `.pt` o `.bin` che contiene miliardi di numeri | | **Training (addestramento)** | Il processo con cui il modello impara dai dati | Mostrare 1 milione di foto di gatti al modello | | **Inferenza** | Usare il modello già addestrato per fare predizioni | Chiedere al modello "c'è un gatto in questa foto?" | | **Dataset** | La raccolta di dati usata per addestrare o valutare | 50.000 email etichettate come spam/non-spam | | **Feature** | Una caratteristica numerica di un dato di input | La lunghezza di un'email, il numero di link che contiene | | **Label / Etichetta** | La risposta corretta per un esempio | "spam" oppure "non spam" | | **Parametri / Pesi** | I numeri dentro il modello che cambiano durante il training | GPT-4 ha circa 1.8 trilioni di parametri | | **Iperparametri** | I parametri che tu scegli *prima* del training | Learning rate, dimensione del batch, numero di layer | | **Loss / Perdita** | Una misura di quanto il modello si sbaglia | Se prevede 0.9 su un'email che non è spam, la loss è alta | | **Gradiente** | La direzione in cui i pesi devono muoversi per ridurre la loss | Un vettore che punta verso l'errore maggiore | | **Epoch** | Un passaggio

completo su tutto il dataset durante il training | Se il dataset ha 10.000 esempi, 1 epoch = 10.000 esempi visti | | **Batch** | Un sottoinsieme del dataset processato insieme | 32 immagini elaborate contemporaneamente prima di aggiornare i pesi | | **Overfitting** | Quando il modello memorizza il training set invece di generalizzare | Perfetto sul training, pessimo su dati nuovi | | **Benchmark** | Un test standard per confrontare modelli diversi | MMLU misura la conoscenza generale degli LLM |

Non è necessario memorizzare tutto ora. Queste definizioni sono qui come riferimento: ogni volta che incontrerai un termine nuovo, saprai dove cercarlo.

1.7 Setup dell'ambiente: pronti a scrivere codice

Prima di concludere questo capitolo introduttivo, prepariamo l'ambiente. Avrai bisogno di questo setup per tutti gli esercizi pratici dei volumi successivi.

Opzione A: Google Colab (consigliata per iniziare)

Google Colab è un ambiente Jupyter online con GPU gratuita. Non richiede installazioni.

4. Vai su colab.research.google.com
5. Crea un nuovo notebook
6. Nel menu in alto: Runtime → Change runtime type → T4 GPU

Opzione B: Installazione locale

```
# Crea un ambiente virtuale (buona pratica sempre)
python -m venv venv-ai
source venv-ai/bin/activate # su Windows: venv-ai\Scripts\activate

# Installa le librerie fondamentali
pip install numpy pandas matplotlib scikit-learn torch torchvision jupyter

# Avvia Jupyter
jupyter notebook
```

Verifica che tutto funzioni

```
# Esegui questo snippet per verificare l'installazione
import numpy as np
import torch

print(f"NumPy: {np.__version__}")
print(f"PyTorch: {torch.__version__}")
print(f"GPU disponibile: {torch.cuda.is_available()}")

# Un piccolo test
x = np.array([1, 2, 3, 4, 5])
print(f"Media: {x.mean()}, Deviazione standard: {x.std():.2f}")
```

Se vedi i numeri di versione senza errori, sei pronto.

Riepilogo del Capitolo 1

Hai imparato:

- **La differenza fondamentale** tra programmazione tradizionale (regole scritte) e ML (schemi appresi): nel ML mostri esempi invece di scrivere istruzioni.
- **La mappa dell'IA**: IA → ML → Deep Learning → LLM → Agenti. Ogni livello è un sottoinsieme del precedente.
- **I limiti reali dell'IA oggi**: ottima nel riconoscimento di pattern, debole nel ragionamento causale e nell'aritmetica complessa.
- **La storia in 15 tappe**: due AI Winter ci insegnano che servono algoritmi + dati + calcolo insieme.
- **ANI vs AGI vs ASI**: tutta l'IA attuale è ANI. L'AGI non esiste ancora.
- **Il vocabolario base**: modello, training, inferenza, dataset, feature, label, parametri, loss, gradiente.

Esercizio 1 — Riflessione guidata

Obiettivo: consolidare la distinzione tra approccio basato su regole e approccio ML.

Considera questi tre problemi:

7. Dato un array di numeri, trova il massimo.
8. Data una foto, determina se il soggetto sorride.
9. Data una sequenza di clic su un e-commerce, predici se l'utente comprerà entro 24 ore.

Domande:

- Per ognuno, quale approccio useresti: regole scritte o ML? Perché?
- Per i problemi che hai scelto per il ML: che tipo di dati ti servirebbero per addestrare il modello?
- Riesci a identificare un caso in cui il ML potrebbe essere la scelta sbagliata anche se tecnicamente applicabile?

(Non c'è una risposta unica. L'obiettivo è ragionare sul trade-off, non trovare la soluzione "corretta".)

Esercizio 2 — Primo codice

Obiettivo: prendere confidenza con l'ambiente e con la manipolazione di dati.

```
import numpy as np

# Supponiamo di avere 10 email con queste caratteristiche:
# [lunghezza_email, numero_link, numero_parole_maiuscole]
email_features = np.array([
    [120, 2, 3], # email 1
    [45, 0, 1], # email 2
    [800, 15, 42], # email 3 — sembra spam?
    [200, 1, 5], # email 4
    [90, 3, 8], # email 5
```

```

[600, 12, 38], # email 6 - sembra spam?
[150,  0,  2], # email 7
[55,   1,  0], # email 8
[750, 18, 50], # email 9 - sembra spam?
[100,  2,  4], # email 10
])

etichette = np.array([0, 0, 1, 0, 0, 1, 0, 0, 1, 0]) # 0=non spam, 1=spam

# Domanda 1: qual è la lunghezza media delle email spam?
email_spam = email_features[etichette == 1]
email_no_spam = email_features[etichette == 0]

print("Email spam - statistiche:")
print(f"  Lunghezza media: {email_spam[:, 0].mean():.1f}")
print(f"  Link medi: {email_spam[:, 1].mean():.1f}")
print(f"  Maiuscole medie: {email_spam[:, 2].mean():.1f}")

print("\nEmail non-spam - statistiche:")
print(f"  Lunghezza media: {email_no_spam[:, 0].mean():.1f}")
print(f"  Link medi: {email_no_spam[:, 1].mean():.1f}")
print(f"  Maiuscole medie: {email_no_spam[:, 2].mean():.1f}")

# Domanda 2 (da completare tu):
# Scrivi una funzione is_spam_rule(email) che usa soglie sui valori
# per classificare un'email come spam o no.
# Poi testala sulle 10 email e confronta con le etichette reali.
# Riesci a scrivere regole che classifichino correttamente tutte e 10?

```

Esegui il codice, osserva i risultati, e prova a completare la Domanda 2. Nel Capitolo 6 vedrai come un algoritmo di ML risolve lo stesso problema automaticamente — e come si confronta con le tue regole scritte a mano.

Prossimo capitolo: I Dati — il carburante dell'IA. Scopriremo che il 70% del lavoro di un progetto ML è sui dati, non sui modelli.

Capitolo 2

I Dati: il carburante dell'IA

"In God we trust. All others must bring data." — W. Edwards Deming

Perché i dati vengono prima di tutto

Prima di scrivere una sola riga di codice ML, prima di scegliere un'architettura, prima di toccare PyTorch, devi capire i tuoi dati. Non è un consiglio scontato: è la lezione che ogni data scientist impara nel modo più doloroso, ovvero lavorando per settimane su un modello per scoprire che il problema era nei dati, non nell'algoritmo.

Una regola empirica del settore dice che in un progetto ML reale:

- **70% del tempo** va su dati: raccolta, pulizia, esplorazione, trasformazione
- **20% del tempo** va sull'addestramento e ottimizzazione del modello
- **10% del tempo** va sul deploy e il monitoraggio

Questa distribuzione sorprende quasi sempre chi inizia. L'immagine romantica del data scientist che progetta reti neurali elaborate è vera solo per quell'ultimo 20%. Il resto è pazienza, rigore e familiarità profonda con i dati.

In questo capitolo costruiamo quella familiarità.

2.1 Tipi di dati: la materia prima dell'IA

Non tutti i dati sono uguali. Il tipo di dato che hai determina quale architettura userai, quali preprocessing sono necessari e quali problemi incontrerai.

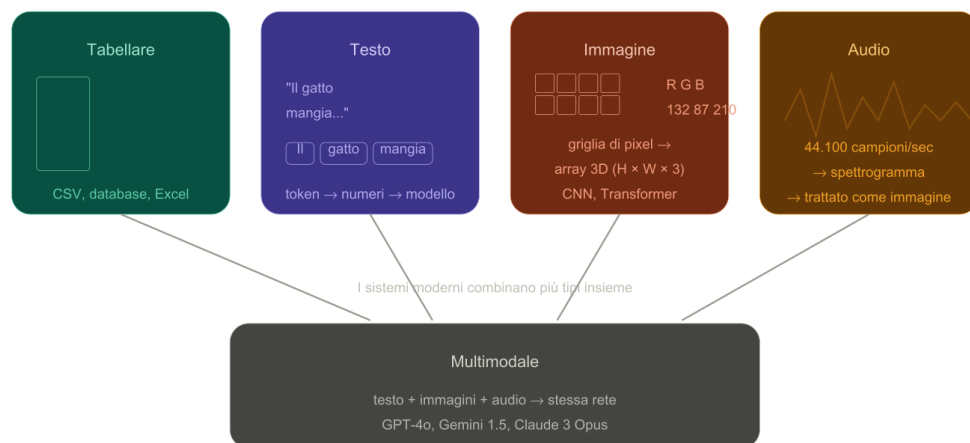


Figura 2.1 — panoramica visiva dei tipi di dati - I quattro tipi di dati principali in ML e come si rappresentano numericamente.

Dati tabellari (strutturati)

La forma più familiare per uno sviluppatore: righe e colonne, come un database o un CSV. Ogni riga è un campione, ogni colonna è una feature.

```
import pandas as pd

# Esempio: dataset clienti di un e-commerce
df = pd.DataFrame({
    'età': [25, 34, 28, 45, 31],
    'spesa_media': [120.5, 340.0, 89.0, 520.0, 210.0],
    'categoria': ['tech', 'moda', 'tech', 'sport', 'moda'],
    'ha_acquistato': [1, 0, 1, 1, 0] # ← questa è la label
})
```

Caratteristiche chiave:

- Ogni feature ha un significato preciso e indipendente
- Mix di valori numerici e categorici
- Spesso con valori mancanti (NaN)
- Dimensioni tipiche: da centinaia a milioni di righe, da decine a migliaia di colonne

Modelli migliori: XGBoost, LightGBM, Random Forest (Cap. 6). Le reti neurali spesso non battono questi su dati tabellari.

Dati testuali (non strutturati)

Testo grezzo in qualsiasi forma: email, articoli, codice sorgente, tweet, documenti PDF. Non ha una struttura predefinita: due email possono avere lunghezze completamente diverse.

```
# Esempio: recensioni di prodotti
testi = [
    "Prodotto ottimo, arrivato in tempi brevissimi. Lo consiglio!",
    "Deludente. La qualità non corrisponde alla descrizione.",
    "Nella media, niente di speciale ma funziona.",
]
etichette = [1, 0, 0] # 1=positiva, 0=negativa/neutra
```

La sfida principale: un computer non capisce le parole. Il testo deve essere convertito in numeri prima di poter fare qualsiasi cosa. Questa conversione si chiama **tokenizzazione** (Cap. 12) ed è uno dei passaggi più critici nel lavoro con LLM.

Cosa sono i token? Un token è l'unità minima in cui il testo viene spezzato prima di essere elaborato da un modello linguistico. Non corrisponde necessariamente a una parola intera: può essere una parola, una parte di parola, un segno di punteggiatura o anche un singolo carattere, a seconda del tokenizer usato. Per esempio, la parola "inaspettatamente" potrebbe essere divisa in più token ("inas", "pett", "ata", "mente"), mentre "gatto" probabilmente è un token solo. I numeri vengono spesso tokenizzati cifra per cifra: "127" diventa tre token separati ["1", "2", "7"], il che spiega parzialmente perché i LLM fanno fatica con l'aritmetica.

Immagini

Un'immagine digitale è una griglia di pixel. Ogni pixel ha uno o più valori numerici: in scala di grigi è un solo numero (da 0=nero a 255=bianco), in RGB sono tre numeri (rosso, verde, blu).

```
import numpy as np
```

```
# Un'immagine RGB 224×224 è un array 3D
immagine = np.random.randint(0, 256, size=(224, 224, 3), dtype=np.uint8)
print(f"Shape: {immagine.shape}")      # (224, 224, 3)
print(f"Pixel [0,0]: {immagine[0,0]}") # es. [132, 87, 210] → R, G, B

# Un batch di 32 immagini è un array 4D
batch = np.zeros((32, 224, 224, 3))
print(f"Shape batch: {batch.shape}")   # (32, 224, 224, 3)
```

La struttura spaziale è fondamentale: pixel vicini tendono ad avere valori simili (un cielo blu occupa una zona contigua, non pixel sparsi). Le CNN (Cap. 9) sono progettate proprio per sfruttare questa proprietà.

Audio

Un file audio è una sequenza di campioni di ampiezza nel tempo. Una registrazione a 44.1 kHz ha 44.100 misurazioni per ogni secondo di audio.

```
# Rappresentazione semplificata di 1 secondo di audio (mono, 16kHz)
audio = np.zeros(16000) # 16.000 campioni
# In pratica si lavora con librerie come librosa o torchaudio
```

Spesso l'audio viene convertito in uno spettrogramma (una rappresentazione visiva delle frequenze nel tempo) prima di essere processato, così si possono usare le stesse tecniche delle immagini.

Dati multimodali

I sistemi moderni sempre più spesso combinano più tipi di dati insieme. Un modello come GPT-4o riceve testo + immagini contemporaneamente. Un sistema di moderazione di contenuti può analizzare testo + audio + metadati (timestamp, account, geolocalizzazione) tutti insieme.

2.2 La struttura di un dataset ML

Un dataset ML ha sempre tre componenti fondamentali.

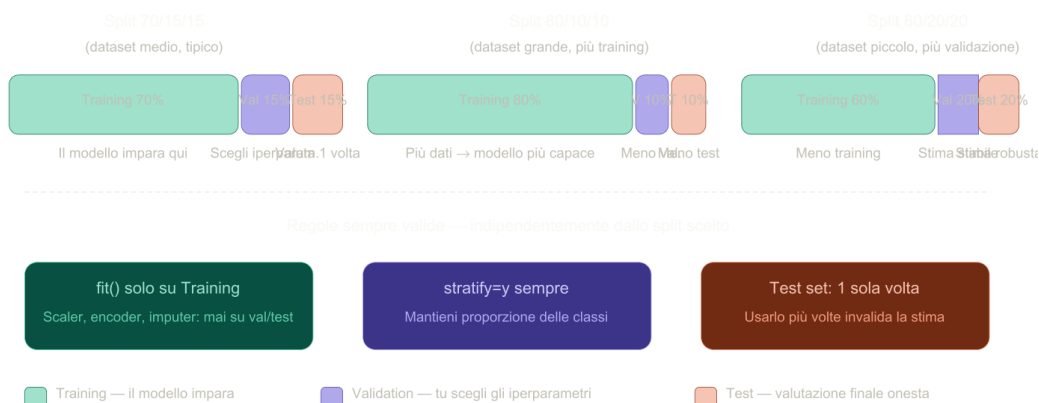


Figura 2.2 — Tre scenari di split a confronto

Figura 2.2 — La divisione train/validation/test. Muovi gli slider per esplorare proporzioni diverse e clicca su ciascun segmento per capirne il ruolo.

Campioni, feature e label

Un **campione** (o esempio, o istanza) è una singola osservazione: un'email, un'immagine, un paziente, una transazione.

Le **feature** (caratteristiche) sono le proprietà misurabili di ogni campione. Alcune sono fornite direttamente (l'età di un paziente), altre vengono estratte (il numero di parole maiuscole in un'email).

La **label** (etichetta) è il valore che il modello deve imparare a predire. In supervised learning è sempre presente nel training set, mai nei dati di produzione che il modello vedrà nel mondo reale.

```
# Dataset ben strutturato
X = df[['età', 'spesa_media', 'categoria']] # feature matrix: shape (n_campioni,
n_feature)
y = df['ha_acquistato'] # label vector: shape (n_campioni,)

print(f"Feature matrix: {X.shape}") # (5, 3)
print(f"Label vector: {y.shape}") # (5,)
```

Training set, validation set, test set

Questa è la distinzione più importante di tutto il ML. Sbagliare qui invalida completamente il tuo lavoro.

Training set (60–80%): i dati su cui il modello impara. Il modello vede questi dati molte volte durante l'addestramento. Non li useremo mai per valutare le prestazioni reali.

Validation set (10–20%): i dati su cui valuti il modello durante lo sviluppo. Usi questo set per scegliere iperparametri, confrontare architetture, decidere quando fermare il training. Il modello non si addestra su questi, ma la tua scelta delle iperparametri è influenzata da come il modello performa qui.

Test set (10–20%): i dati che tocchi una volta sola, alla fine, per riportare le prestazioni finali. Simulano il comportamento su dati mai visti. Se usi il test set per fare decisioni di design, stai "contaminando" la valutazione.

***Analogia:** immagina di prepararti per un esame. Il training set è il libro di testo (lo studi in profondità). Il validation set sono gli esercizi del libro (ti aiutano a capire dove sei debole e a correggere il metodo di studio). Il test set è l'esame vero (lo dai una volta sola, e il voto riflette quanto hai davvero imparato).*

```
from sklearn.model_selection import train_test_split

# Prima split: separa test set (20%)
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Seconda split: separa validation set dal resto (20% del totale)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42
    # 0.25 × 0.80 = 0.20 del dataset totale
)

print(f"Training: {len(X_train)} campioni")
print(f"Validation: {len(X_val)} campioni")
```

```
print(f"Test: {len(X_test)} campioni")
```

Il parametro `random_state=42` garantisce che la divisione sia riproducibile: eseguendo il codice più volte otterrai sempre gli stessi split. Scegli qualsiasi numero, ma usalo sempre uguale.

2.3 Feature: numeriche, categoriche e ordinali

Non tutte le feature sono uguali. La differenza di tipo determina come le tratti.

Feature numeriche continue

Valori reali che possono assumere qualsiasi numero: età, prezzo, temperatura, peso. I modelli ML lavorano naturalmente con questi.

Attenzione alla scala: una feature "età" va da 0 a 100, mentre "reddito annuo" va da 0 a 500.000. Se le usi così come sono, il modello darà molto più peso al reddito solo perché i suoi valori sono più grandi numericamente, non perché sia più informativa. La soluzione è la normalizzazione (vedi sezione 2.5).

Feature categoriche nominali

Categorie senza un ordine naturale: colore, paese, categoria merceologica. Non puoi codificarle come numeri interi così come sono, perché il modello interpreterebbe "Italia=3, Francia=1, Germania=2" come se Italia fosse più grande di Francia, il che non ha senso.

La soluzione classica è il **one-hot encoding**: ogni categoria diventa una colonna binaria.

```
import pandas as pd

df = pd.DataFrame({'categoria': ['tech', 'moda', 'tech', 'sport']})

# One-hot encoding
df_encoded = pd.get_dummies(df, columns=['categoria'])
print(df_encoded)
#   categoria_moda  categoria_sport  categoria_tech
# 0              0              0              1
# 1              1              0              0
# 2              0              0              1
# 3              0              1              0
```

Feature categoriche ordinali

Categorie con un ordine naturale: taglia (S < M < L < XL), giudizio (basso < medio < alto). Qui la codifica numerica ha senso (S=0, M=1, L=2, XL=3), perché l'ordine è significativo.

2.4 Il problema del bias nei dati

Questa sezione è tra le più importanti del volume. I modelli ML non creano bias: li amplificano. Se il tuo dataset è distorto, il tuo modello lo sarà ancora di più, e con un'apparenza di oggettività che lo rende più pericoloso.

Bias di selezione

I dati non rappresentano uniformemente la popolazione reale. Un sistema di riconoscimento facciale addestrato principalmente su volti di persone con pelle chiara performerà molto peggio su persone con pelle scura, non perché il modello sia "razzista", ma perché non ha visto abbastanza esempi diversi.

Caso reale documentato: nel 2018, una ricerca del MIT Media Lab ha dimostrato che sistemi commerciali di riconoscimento del genere avevano un tasso di errore dell'1% su uomini con pelle chiara e del 35% su donne con pelle scura. La causa principale era la distribuzione sbilanciata nei dataset di training.

Bias di misurazione

Le feature sono misurate in modo non uniforme tra gruppi diversi. Se usi i precedenti penali come feature per predire la recidiva, stai incorporando le distorsioni del sistema giudiziario (certi gruppi vengono arrestati più frequentemente a parità di comportamento) nel tuo modello.

Bias di conferma nei label

Chi ha etichettato i dati porta con sé i propri pregiudizi. Se i curriculum vengono etichettati come "adatti" o "non adatti" da valutatori con pregiudizi inconsci, il modello imparerà quegli stessi pregiudizi.

Implicazione pratica: prima di addestrare un modello, chiediti sempre:

10. Chi ha raccolto questi dati? Con quale obiettivo?
11. Chi è sovrarappresentato? Chi è sottorappresentato?
12. Le label riflettono la realtà o il pregiudizio di chi ha etichettato?

2.5 Preprocessing: preparare i dati per il modello

I modelli ML richiedono che i dati siano in un formato specifico. Il preprocessing trasforma i dati grezzi in questo formato.

Gestione dei valori mancanti (NaN)

Quasi tutti i dataset reali hanno valori mancanti. Puoi ignorarli (molti algoritmi non li gestiscono), eliminarli (rischi di perdere informazioni) o imputarli (sostituirli con una stima).

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'età': [25, np.nan, 28, 45, np.nan],
    'spesa_media': [120.5, 340.0, np.nan, 520.0, 210.0],
})

# Strategia 1: elimina righe con NaN (semplice, ma perde dati)
df_clean = df.dropna()

# Strategia 2: imputa con la mediana (robusta agli outlier)
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='median')
```



```
df_imputed = pd.DataFrame(
    imputer.fit_transform(df),
    columns=df.columns
)
print(df_imputed)
# Ogni NaN è sostituito con la mediana della sua colonna
```

Normalizzazione e standardizzazione

Due tecniche per portare le feature sulla stessa scala.

Standardizzazione (Z-score): trasforma ogni valore sottraendo la media e dividendo per la deviazione standard. Il risultato ha media 0 e deviazione standard 1.

```
x_std = (x - media) / deviazione_standard
```

Min-Max scaling: porta tutti i valori nell'intervallo [0, 1].

```
x_norm = (x - min) / (max - min)

from sklearn.preprocessing import StandardScaler, MinMaxScaler
import numpy as np

X = np.array([[25, 120.5], [34, 340.0], [28, 89.0], [45, 520.0]])

# Standardizzazione
scaler_std = StandardScaler()
X_std = scaler_std.fit_transform(X)
print("Dopo standardizzazione:")
print(X_std.round(2))
# Ogni colonna ha ora media ≈ 0 e std ≈ 1

# Min-Max
scaler_mm = MinMaxScaler()
X_mm = scaler_mm.fit_transform(X)
print("Dopo min-max:")
print(X_mm.round(2))
# Ogni colonna è ora nell'intervallo [0, 1]
```

Regola fondamentale: il `fit()` va fatto solo sul training set, mai sull'intero dataset. Se calcoli la media e la deviazione standard includendo il test set, stai "contaminando" la valutazione: il modello avrebbe indirettamente visto informazioni sui dati di test. Si chiama **data leakage** ed è uno degli errori più comuni e più insidiosi nel ML.

```
# SBAGLIATO - data leakage
scaler.fit(X_intero)          # include info del test set!
X_train_scaled = scaler.transform(X_train)
X_test_scaled  = scaler.transform(X_test)

# CORRETTO
scaler.fit(X_train)          # impara parametri solo dal training
X_train_scaled = scaler.transform(X_train)
X_test_scaled  = scaler.transform(X_test)  # applica gli stessi parametri
```

2.6 Analisi esplorativa: conoscere i dati prima di modellarli

Prima di qualsiasi modello, esplora i tuoi dati. L'analisi esplorativa (EDA — Exploratory Data Analysis) è il momento in cui capisci la struttura, individui anomalie e generi ipotesi.

Statistiche descrittive

```
import pandas as pd
import numpy as np

# Dataset esempio: stipendi e anni di esperienza
np.random.seed(42)
df = pd.DataFrame({
    'anni_esperienza': np.random.randint(0, 20, 200),
    'stipendio': np.random.normal(45000, 15000, 200).clip(20000, 120000),
    'ruolo': np.random.choice(['junior', 'mid', 'senior'], 200),
    'ha_laurea': np.random.choice([True, False], 200),
})

# Le statistiche di base ti dicono moltissimo
print(df.describe())
#           anni_esperienza      stipendio
# count          200.000         200.000
# mean             9.515        45231.123
# std              5.766        14821.456
# min              0.000        20000.000
# 25%              5.000        33210.000
# 50%             10.000        45012.000
# 75%             14.000        56890.000
# max             19.000       119234.000

# Controlla valori mancanti
print(df.isnull().sum())

# Controlla la distribuzione delle categorie
print(df['ruolo'].value_counts())
```

Visualizzazione

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# 1. Distribuzione degli stipendi (istogramma)
axes[0].hist(df['stipendio'], bins=30, edgecolor='white')
axes[0].set_title('Distribuzione stipendi')
axes[0].set_xlabel('Stipendio (€)')
axes[0].set_ylabel('Frequenza')

# 2. Relazione tra esperienza e stipendio (scatter)
axes[1].scatter(df['anni_esperienza'], df['stipendio'], alpha=0.5)
axes[1].set_title('Esperienza vs Stipendio')
axes[1].set_xlabel('Anni di esperienza')
axes[1].set_ylabel('Stipendio (€)')

# 3. Stipendio medio per ruolo (bar chart)
stipendio_per_ruolo = df.groupby('ruolo')['stipendio'].mean()
```

```
axes[2].bar(stipendio_per_ruolo.index, stipendio_per_ruolo.values)
axes[2].set_title('Stipendio medio per ruolo')
axes[2].set_ylabel('Stipendio medio (€)')

plt.tight_layout()
plt.savefig('eda_stipendi.png', dpi=150, bbox_inches='tight')
plt.show()
```

Cosa cercare nell'EDA

Outlier: valori estremi che potrebbero essere errori di misurazione o casi reali ma rari. Un dipendente con 200 anni di esperienza è un errore. Un CEO con stipendio 10× la media potrebbe essere un valore reale ma distorto.

Correlazioni: feature che variano insieme. Se "anni_esperienza" e "stipendio" sono fortemente correlate, hai trovato una feature informativa. Se due feature sono quasi identiche, ne puoi eliminare una.

Distribuzione sbilanciata delle label: se hai 1000 email non-spam e solo 10 spam, il tuo modello potrebbe imparare a dire sempre "non spam" e ottenere 99% di accuratezza pur essendo completamente inutile. Questo problema si chiama **class imbalance** e ha soluzioni specifiche (oversampling, undersampling, class weights).

```
# Controlla sempre la distribuzione delle label
print(df['ruolo'].value_counts(normalize=True))
# junior    0.35
# mid       0.34
# senior    0.31
# Bilanciato → nessun problema

# Esempio sbilanciato (problematico)
# spam      0.01
# non_spam  0.99
```

2.7 Qualità dei dati: la checklist

Prima di passare al modello, passa questa lista:

Controllo	Come verificarlo	Cosa fare se fallisce	----- ----- -----
Valori mancanti	<code>df.isnull().sum()</code>	Imputa o elimina righe/colonne	Duplicati
<code>df.duplicated().sum()</code>	Rimuovi con <code>df.drop_duplicates()</code>	Tipi corretti	<code>df.dtypes</code>
Converti con <code>astype()</code>	Outlier	Boxplot, IQR	Cappa, trasforma o rimuovi
Label bilanciate	<code>y.value_counts()</code>	Oversampling, undersampling, class weights	Data leakage
Audit del processo di split	Rifai lo split correttamente	Bias di selezione	Analisi demografica
Bilancia il dataset o documenta il limite	Range sensato	<code>df.describe()</code>	Normalizza o standardizza

Riepilogo del Capitolo 2

Hai imparato:

- I **quattro tipi principali di dati** in ML: tabellari, testuali, immagini, audio. Ogni tipo richiede approcci diversi.
- **Cosa sono i token**: l'unità minima in cui il testo viene spezzato prima di essere elaborato da un LLM — non coincide necessariamente con una parola.
- La **divisione training/validation/test** e perché è critica: confondere i tre set invalida tutta la valutazione.
- Le **feature** e come gestire tipi diversi: numeriche continue, categoriche nominali (one-hot encoding), categoriche ordinali.
- Il **bias nei dati**: i modelli non creano bias, li amplificano. Bias di selezione, di misurazione, di label.
- Il **preprocessing essenziale**: gestione dei NaN, normalizzazione, standardizzazione.
- Il **data leakage**: fit solo sul training set, mai sull'intero dataset.
- L'**analisi esplorativa (EDA)**: statistiche descrittive, visualizzazioni, cosa cercare.

Esercizio 1 — Esplorazione guidata

Obiettivo: praticare l'EDA su un dataset reale.

```
# Scarica il dataset Titanic (classico per iniziare)
import pandas as pd

url = 'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv'
df = pd.read_csv(url)

# Domande a cui rispondere con il codice:
# 1. Quante righe e colonne ha il dataset?
# 2. Quali colonne hanno valori mancanti? Quanti?
# 3. Qual è il tasso di sopravvivenza complessivo?
# 4. Il tasso di sopravvivenza differisce per sesso? E per classe?
# 5. Qual è la distribuzione delle età? Ci sono outlier?
# 6. Quali feature hanno correlazione più alta con 'Survived'?

# Inizia con queste righe:
print(df.shape)
print(df.head())
print(df.isnull().sum())
print(df['Survived'].value_counts(normalize=True))

# Poi esplora liberamente con .groupby(), .hist(), .corr()
```

Esercizio 2 — Preprocessing completo

Obiettivo: costruire una pipeline di preprocessing riutilizzabile.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
```

```

# Dataset sintetico con problemi reali
np.random.seed(42)
n = 300
df = pd.DataFrame({
    'età': np.where(np.random.rand(n) < 0.1, np.nan,
                    np.random.randint(18, 65, n).astype(float)),
    'reddito': np.random.exponential(30000, n) + 15000,
    'città': np.random.choice(['Milano', 'Roma', 'Napoli', 'Torino'], n),
    'anni_cliente': np.random.randint(0, 10, n),
    'ha_comprato': np.random.choice([0, 1], n, p=[0.7, 0.3]),
})

# Il tuo obiettivo:
# 1. Separa X e y
# 2. Fai il train/test split (80/20) con random_state=42
# 3. Applica one-hot encoding alla colonna 'città'
# 4. Imputa i NaN in 'età' con la mediana del training set
# 5. Standardizza le colonne numeriche (solo su training set!)
# 6. Verifica che non ci siano NaN nel risultato finale

# Suggerimento: usa sklearn.pipeline.Pipeline per concatenare
# le trasformazioni in modo che non ci sia data leakage

```

Prossimo capitolo: Matematica Intuitiva per l'IA — vettori, gradienti e probabilità spiegati senza dimostrazioni, con il codice che fa il lavoro pesante.

Capitolo 3

Matematica Intuitiva per l'IA

"La matematica non è difficile. Difficile è capire perché la stai imparando. Quando lo capisci, diventa ovvia."

Perché questo capitolo esiste

Molte guide all'IA presentano formule come se fossero sentenze da memorizzare. Questo capitolo fa il contrario: prima ti dà l'intuizione di cosa sta succedendo, poi mostra la formula come notazione compatta di quella intuizione, poi il codice che la calcola per te.

Non avrai bisogno di dimostrare nulla a mano. Avrai bisogno di capire cosa fa ogni operazione e perché è utile nel contesto dell'IA.

Quattro blocchi fondamentali: algebra lineare, calcolo differenziale, probabilità, entropia.

3.1 Algebra lineare: dati come frecce nello spazio

Scalari, vettori, matrici, tensori

Questi quattro termini descrivono la stessa cosa a dimensioni crescenti: una collezione di numeri.

Un **scalare** è un singolo numero. La loss di un modello in un dato momento è uno scalare. La temperatura oggi è uno scalare.

```
loss = 0.342    # uno scalare
```

Un **vettore** è una lista ordinata di numeri. Può rappresentare un punto nello spazio, oppure le feature di un campione. Se hai un utente descritto da età=25, reddito=35000, anni_cliente=3, quelle tre informazioni insieme formano un vettore.

```
import numpy as np

# Un utente come vettore di feature
utente = np.array([25, 35000, 3])    # shape: (3,)

# Un token come vettore embedding (ne parleremo al Cap. 12)
# Ogni parola diventa un vettore di 768 numeri in BERT
parola = np.random.randn(768)      # shape: (768,)
```

Intuizione geometrica: un vettore è una freccia. Ha una direzione e una lunghezza. Due vettori che puntano nella stessa direzione rappresentano cose simili. Due vettori perpendicolari rappresentano cose completamente diverse. Questa geometria è la base del funzionamento degli embedding (Cap. 14).

Una **matrice** è una griglia di numeri, righe per colonne. Un dataset di N campioni con D feature è una matrice N×D. I pesi di un layer di rete neurale sono una matrice.

```
# Dataset: 5 utenti, 3 feature ciascuno
X = np.array([
    [25, 35000, 3],
    [34, 52000, 7],
    [28, 41000, 2],
    [45, 78000, 12],
    [31, 47000, 5],
])
print(X.shape)  # (5, 3) → 5 righe, 3 colonne

# Matrice dei pesi di un layer: trasforma 3 input in 4 output
W = np.random.randn(4, 3)
print(W.shape)  # (4, 3)
```

Un **tensore** è la generalizzazione a N dimensioni. Un batch di 32 immagini RGB 224×224 è un tensore 4D: (32, 3, 224, 224) — 32 immagini, 3 canali colore, 224 righe, 224 colonne.

```
batch_immagini = np.zeros((32, 3, 224, 224))
print(batch_immagini.shape)  # (32, 3, 224, 224)
print(f"Elementi totali: {batch_immagini.size:,}")  # 9.633.792
```

In PyTorch e in tutta la letteratura ML, il termine "tensore" viene usato per indicare qualsiasi array multidimensionale, inclusi scalari (0D), vettori (1D) e matrici (2D).

Il prodotto scalare: misurare la somiglianza

Il prodotto scalare (o dot product) tra due vettori è l'operazione più usata in tutto il ML. Moltiplica i valori corrispondenti e li somma.

```
a · b = a1b1 + a2b2 + ... + anbn

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Tre modi equivalenti di calcolarlo
print(np.dot(a, b))      # 32
print(a @ b)             # 32 ← notazione moderna preferita
print(sum(a * b))        # 32
```

Perché è così importante? Geometricamente, il prodotto scalare misura quanto due vettori "si allineano". Se puntano nella stessa direzione, è grande e positivo. Se sono perpendicolari, è zero. Se puntano in direzioni opposte, è negativo.

Questo è esattamente il meccanismo dell'attention nei Transformer (Cap. 11): per capire quanto due token sono "correlati", si calcola il prodotto scalare dei loro vettori. Un valore alto significa che il token A deve "prestare attenzione" al token B.

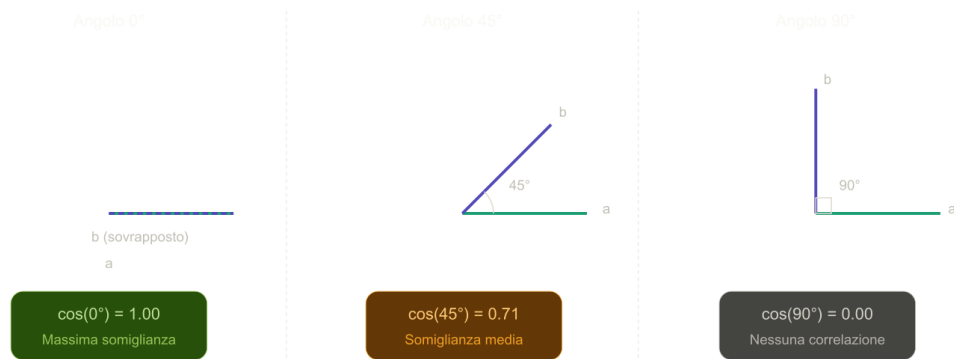


Figura 3.1 — Il prodotto scalare come misura di somiglianza geometrica tra vettori

Figura 3.1 — Il prodotto scalare come somiglianza geometrica tra vettori.

La moltiplicazione matriciale: trasformare lo spazio

La moltiplicazione tra una matrice W e un vettore x produce un nuovo vettore y . Questa operazione è il cuore di ogni layer di rete neurale: $y = W \cdot x + b$.

```
# Layer lineare: trasforma 3 feature in 4 output
W = np.array([
    [0.1, 0.2, 0.3], # pesi del neurone 1
    [0.4, 0.5, 0.6], # pesi del neurone 2
    [0.7, 0.8, 0.9], # pesi del neurone 3
    [1.0, 1.1, 1.2], # pesi del neurone 4
]) # shape: (4, 3)

x = np.array([1.0, 2.0, 3.0]) # input: shape (3,)
b = np.array([0.1, 0.1, 0.1, 0.1]) # bias: shape (4,)

y = W @ x + b # output: shape (4,)
print(y) # [1.5, 3.3, 5.1, 6.9] (circa)
```

Intuizione: moltiplicare per una matrice è come proiettare i dati in uno spazio diverso. Ogni layer di una rete neurale trasforma lo spazio delle feature, rendendolo progressivamente più facile da separare per il task in questione. Un classificatore di immagini trasforma pixel grezzi in spazio di feature visive, poi in spazio semantico, poi in spazio di decisione.

Broadcasting: calcolo su array di dimensioni diverse

NumPy (e PyTorch) permettono operazioni tra array di dimensioni diverse seguendo regole precise. Questo è fondamentale per capire come vengono elaborati i batch.

```
# Sottrarre la media da ogni colonna di un dataset
X = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # shape (3, 3)
media = X.mean(axis=0) # shape (3,): [4, 5, 6]

X_centrato = X - media # broadcasting: media viene "espansa" su ogni riga
# [[-3, -3, -3], [0, 0, 0], [3, 3, 3]]
```


3.2 Calcolo differenziale: come si impara sbagliando

La derivata come misura del cambiamento

La derivata di una funzione $f(x)$ in un punto x ci dice: "se aumento x di una quantità infinitesimale, di quanto cambia $f(x)$?"

Geometricamente è la pendenza della retta tangente al grafico in quel punto.

```
# Esempio:  $f(x) = x^2$ 
# La derivata è  $f'(x) = 2x$ 

import numpy as np

def f(x):
    return x**2

def f_derivata(x):
    return 2 * x

x = 3.0
print(f"f(3) = {f(x)}")           # 9.0
print(f"f'(3) = {f_derivata(x)}") # 6.0
# Significa: se aumenti x di 0.001, f aumenta di circa  $6 \times 0.001 = 0.006$ 

# Verifica numerica
epsilon = 0.0001
derivata_numerica = (f(x + epsilon) - f(x)) / epsilon
print(f"Derivata numerica: {derivata_numerica:.4f}") # ≈ 6.0
```

Perché ci interessa nel ML? La loss del modello è una funzione dei suoi parametri. La derivata della loss rispetto a un peso ci dice: "se aumento questo peso di un po', la loss aumenta o diminuisce? Di quanto?" Conoscere questo per ogni peso ci permette di spostarci nella direzione che riduce la loss. Questo è l'addestramento.

Il gradiente: derivata in N dimensioni

Quando la funzione dipende da milioni di parametri (come i pesi di una rete neurale), la derivata diventa un **gradiente**: il vettore di tutte le derivate parziali.

Ogni componente del gradiente risponde alla domanda: "se cambio solo questo peso, tenendo fissi tutti gli altri, come cambia la loss?"

$$\nabla L(w_1, w_2, \dots, w_n) = [\partial L / \partial w_1, \partial L / \partial w_2, \dots, \partial L / \partial w_n]$$

Analogia della montagna: immagina di essere in montagna con la nebbia. Non vedi la valle, ma puoi sentire la pendenza del terreno sotto i piedi. Il gradiente ti dice in quale direzione il terreno sale più ripidamente. Per scendere a valle (minimizzare la loss), devi muoverti nella direzione opposta al gradiente.

```
import torch # PyTorch calcola gradienti automaticamente

# Definisci i parametri con requires_grad=True
# per dire a PyTorch "voglio il gradiente rispetto a questi"
```

```
w = torch.tensor(3.0, requires_grad=True)

# Calcola una funzione (qui simuliamo una loss semplice)
loss = (w - 5) ** 2 # minimo in w=5, loss=0

# Calcola il gradiente
loss.backward()

print(f"w = {w.item():.1f}")
print(f"loss = {loss.item():.1f}")
print(f"gradiente = {w.grad.item():.1f}")
# gradiente = 2*(w-5) = 2*(3-5) = -4
# Negativo → la loss diminuisce se aumentiamo w → giusto, dobbiamo andare verso 5
```

La chain rule: il cuore della backpropagation

Una rete neurale è una funzione composta: l'output di ogni layer è l'input del successivo. Per calcolare il gradiente dei pesi del primo layer rispetto alla loss (che si calcola all'ultimo layer), usiamo la **regola della catena**.

Se $z = f(g(x))$, allora $dz/dx = (df/dg) \cdot (dg/dx)$.

In una rete con 50 layer, la backpropagation applica questa regola 50 volte di fila, moltiplicando i gradienti locali di ogni layer. Il risultato arriva fino ai pesi del primo layer.

```
# PyTorch lo fa automaticamente — non lo scrivi mai a mano
import torch
import torch.nn as nn

modello = nn.Sequential(
    nn.Linear(3, 10), # layer 1: 3 → 10
    nn.ReLU(),
    nn.Linear(10, 1), # layer 2: 10 → 1
)

x = torch.randn(1, 3) # input
y_vero = torch.tensor([[1.0]])

y_pred = modello(x) # forward pass
loss = (y_pred - y_vero)**2 # calcola loss

loss.backward() # ← backpropagation: calcola gradienti per TUTTI i pesi

# Ora ogni parametro ha il suo gradiente
for nome, param in modello.named_parameters():
    print(f"{nome}: gradiente shape {param.grad.shape}")
```

Nota pratica: non implementerai mai la backpropagation a mano in produzione. PyTorch, TensorFlow e JAX la calcolano automaticamente grazie all'**autograd** — un sistema che, mentre esegui il forward pass, costruisce un grafo computazionale e lo usa per calcolare i gradienti in ordine inverso.

3.3 Probabilità: ragionare nell'incertezza

Probabilità come grado di credenza

I modelli ML non producono certezze: producono distribuzioni di probabilità. Quando un classificatore di immagini dice "gatto: 87%, cane: 11%, altro: 2%", sta esprimendo incertezza quantificata.

I concetti fondamentali:

- $P(A)$ — probabilità che l'evento A accada. Sempre tra 0 e 1.
- $P(A|B)$ — probabilità condizionata: probabilità di A **dato che** B è già accaduto.
- $P(A, B)$ — probabilità congiunta: probabilità che A **e** B accadano insieme.

```
import numpy as np

# Simuliamo 10.000 lanci di un dado truccato
np.random.seed(42)
lanci = np.random.choice([1,2,3,4,5,6], size=10000, p=[0.1,0.1,0.1,0.1,0.1,0.5])

# P(6) — probabilità di ottenere 6
p_sei = np.mean(lanci == 6)
print(f"P(6) stimata: {p_sei:.3f}") # ≈ 0.500

# P(pari) — probabilità di ottenere un numero pari
p_pari = np.mean(lanci % 2 == 0)
print(f"P(pari) stimata: {p_pari:.3f}") # ≈ 0.600

# P(6 | pari) — probabilità di 6 sapendo che è uscito un numero pari
lanci_pari = lanci[lanci % 2 == 0]
p_sei_dato_pari = np.mean(lanci_pari == 6)
print(f"P(6|pari) stimata: {p_sei_dato_pari:.3f}") # ≈ 0.833
```

Il Teorema di Bayes

$$P(A|B) = P(B|A) \cdot P(A) / P(B)$$

In parole: la probabilità di A dopo aver osservato B è proporzionale a quanto B è probabile se A fosse vero, moltiplicato per quanto credevo in A prima dell'osservazione.

Questo teorema è alla base di decine di algoritmi ML e del RLHF (come il modello aggiorna le sue "credenze" sui comportamenti utili — Cap. 12).

La distribuzione softmax: da numeri grezzi a probabilità

Alla fine di un classificatore, il modello produce dei numeri grezzi chiamati **logit**. Il softmax li converte in probabilità che sommano a 1.

```
def softmax(logits):
    exp = np.exp(logits - np.max(logits)) # sottrai il max per stabilità numerica
    return exp / exp.sum()

# Output grezzo di un classificatore a 5 classi
logits = np.array([2.1, 0.5, -1.2, 3.4, 0.8])

probabilita = softmax(logits)
print(probabilita.round(3))
```

```
# [0.085, 0.017, 0.003, 0.860, 0.024] → somma = 1.0

# La classe con probabilità più alta è la predizione del modello
print(f"Classe predetta: {np.argmax(probabilita)}") # 3
```

Intuizione: il softmax esagera le differenze. Se un logit è leggermente più grande degli altri, il softmax gli assegna una probabilità molto più alta. Questo rende il modello "deciso" anche quando la differenza è piccola. La temperatura (che vedrai al Cap. 12) controlla quanto è "deciso": alta temperatura = distribuisce più uniformemente, bassa temperatura = ancora più deciso.

3.4 Entropia: misurare l'incertezza

Entropia di Shannon

L'entropia misura quanto è incerta una distribuzione di probabilità. Se sei certo al 100% del risultato ($P=1$ per un evento, 0 per tutti gli altri), l'entropia è 0. Se tutti i risultati sono ugualmente probabili, l'entropia è massima.

$$H(P) = -\sum P(x) \cdot \log_2 P(x)$$

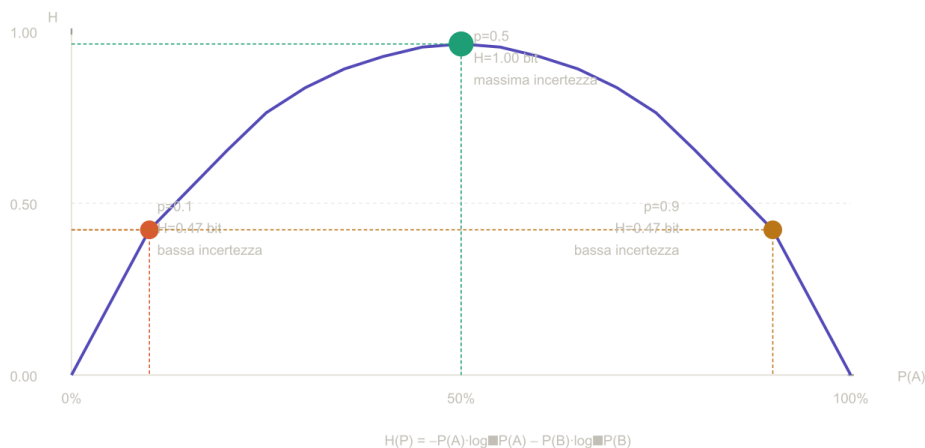


Figura 3.2 — Entropia di Shannon per distribuzione binaria

Figura 3.2 — Entropia di Shannon: da certezza assoluta ($H=0$) a massima incertezza ($H=\log_2 K$).

```
def entropia(probs):
    """Calcola l'entropia di Shannon di una distribuzione."""
    probs = np.array(probs)
    probs = probs[probs > 0] # evita log(0)
    return -np.sum(probs * np.log2(probs))

# Caso 1: certezza assoluta
print(f"Certezza totale: {entropia([1.0, 0.0, 0.0]):.3f} bit") # 0.000

# Caso 2: due opzioni equiprobabili (lancio di moneta)
print(f"Moneta equa: {entropia([0.5, 0.5]):.3f} bit") # 1.000

# Caso 3: quattro opzioni equiprobabili
print(f"Dado a 4 facce: {entropia([0.25]*4):.3f} bit") # 2.000
```

```
# Caso 4: moneta truccata
print(f"Moneta truccata: {entropia([0.9, 0.1]):.3f} bit")           # 0.469
```

L'unità "bit" ha un significato concreto: quante domande sì/no devi fare in media per determinare il risultato. Con entropia 3 bit, ti servono in media 3 domande.

Cross-entropy: la loss function del ML

La **cross-entropy** misura quanto la distribuzione predetta dal modello (Q) si discosta dalla distribuzione reale (P). È la loss function più usata nella classificazione.

$$H(P, Q) = -\sum P(x) \cdot \log Q(x)$$

Caso speciale con un'unica classe corretta (one-hot label):

```
loss = -log Q(classe_corretta)

import numpy as np

def cross_entropy(y_vero, y_pred):
    """
    y_vero: indice della classe corretta
    y_pred: probabilità predette (output del softmax)
    """
    return -np.log(y_pred[y_vero] + 1e-10) # 1e-10 evita log(0)

# Scenario 1: modello molto sicuro e corretto
probs_ottimo = np.array([0.01, 0.01, 0.97, 0.01]) # classe 2 è corretta
loss_1 = cross_entropy(2, probs_ottimo)
print(f"Modello sicuro e corretto → loss = {loss_1:.4f}") # ≈ 0.030

# Scenario 2: modello incerto
probs_incerto = np.array([0.25, 0.25, 0.25, 0.25])
loss_2 = cross_entropy(2, probs_incerto)
print(f"Modello incerto → loss = {loss_2:.4f}") # ≈ 1.386

# Scenario 3: modello sicuro ma sbagliato
probs_sbagliato = np.array([0.97, 0.01, 0.01, 0.01]) # classe 2 corretta, ma predice classe 0
loss_3 = cross_entropy(2, probs_sbagliato)
print(f"Modello sicuro ma sbagliato → loss = {loss_3:.4f}") # ≈ 4.605 - altissima!
```

Intuizione: la cross-entropy punisce il modello in proporzione a quanto è sicuro della risposta sbagliata. Un modello che dice "sono al 97% sicuro che sia sbagliato" riceve una penalità enorme. Un modello incerto riceve una penalità moderata. Questo incentiva il modello a essere umile quando non sa la risposta.

KL Divergence: quanto sono diverse due distribuzioni?

La **KL Divergence** (Kullback-Leibler) misura quanto una distribuzione Q si discosta da una distribuzione di riferimento P.

$$D_{KL}(P||Q) = \sum P(x) \cdot \log(P(x) / Q(x))$$

La userai in contesti avanzati come il RLHF (dove si minimizza la divergenza tra il modello originale e quello fine-tuned per evitare che diventi troppo diverso), i VAE (modelli generativi) e la distillazione.

```
def kl_divergence(p, q):
    p, q = np.array(p), np.array(q)
    mask = (p > 0) & (q > 0)
    return np.sum(p[mask] * np.log(p[mask] / q[mask]))

P = [0.4, 0.4, 0.2] # distribuzione reale
Q1 = [0.4, 0.4, 0.2] # identica a P
Q2 = [0.1, 0.8, 0.1] # diversa da P

print(f"D_KL(P||P) = {kl_divergence(P, Q1):.4f}") # 0.0000 - identiche
print(f"D_KL(P||Q2) = {kl_divergence(P, Q2):.4f}") # > 0 - diverse
```

Nota: la KL Divergence non è simmetrica — $D_{KL}(P||Q) \neq D_{KL}(Q||P)$. Cambia la risposta alla domanda che stai facendo: "quanto P si discosta da Q" è diverso da "quanto Q si discosta da P".

3.5 La mappa matematica del ML

Vediamo ora dove queste operazioni compaiono in concreto durante il ciclo di vita di un modello:

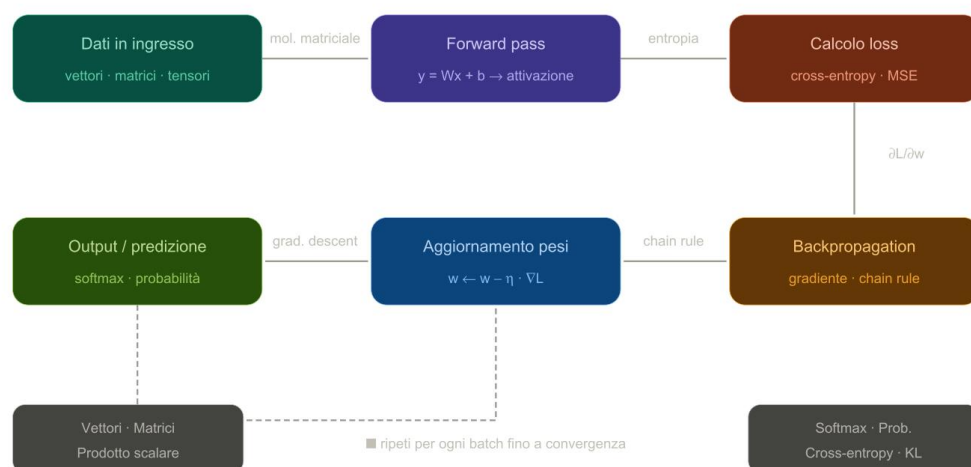


Figura 3.3 — Il ciclo di addestramento e le operazioni matematiche associate a ciascuna fase.

| Operazione | Dove appare | Perché | |-----|-----|-----| | Prodotto scalare | Attention, layer lineare, similarità embedding | Misurare allineamento tra vettori | | Moltiplicazione matriciale | Ogni layer della rete | Trasformare lo spazio delle feature | | Gradiente | Backpropagation | Sapere in quale direzione modificare i pesi | | Chain rule | Backpropagation su reti profonde | Propagare il segnale di errore | | Softmax | Output classificatore, attention | Convertire logit in probabilità | | Cross-entropy | Loss function | Penalizzare predizioni sbagliate/incerte | | KL Divergence | RLHF, VAE, distillazione | Misurare divergenza tra distribuzioni |

Riepilogo del Capitolo 3

Hai imparato:

- **Scalari, vettori, matrici, tensori**: gerarchie di array di numeri. Ogni dato in ML è uno di questi.
- **Prodotto scalare**: misura di somiglianza tra vettori. Alla base dell'attention.
- **Moltiplicazione matriciale**: trasformazione dello spazio. Ogni layer è $y = Wx + b$.
- **Gradiente**: vettore delle derivate parziali. Indica la direzione di salita della loss.
- **Chain rule**: permette di calcolare i gradienti attraverso reti profonde.
- **Probabilità condizionata e Bayes**: ragionare aggiornando credenze con nuove evidenze.
- **Softmax**: da logit a distribuzione di probabilità.
- **Entropia**: misura di incertezza. Zero con certezza assoluta, massima con distribuzione uniforme.
- **Cross-entropy**: loss function della classificazione. Penalizza sicurezza nelle risposte sbagliate.
- **KL Divergence**: distanza tra distribuzioni. Usata in RLHF e modelli generativi.

Esercizio 1 — Geometria dei vettori

Obiettivo: capire il prodotto scalare come strumento di similarità.

```
import numpy as np

# Tre "utenti" rappresentati come vettori di preferenze
# [azione, commedia, horror, fantascienza, documentari]
alice = np.array([5, 1, 1, 4, 3])
bob   = np.array([4, 1, 2, 5, 2])
carlo = np.array([1, 5, 1, 1, 4])

def similarita_coseno(a, b):
    """La similarità coseno normalizza per la lunghezza dei vettori."""
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# Domande:
# 1. Calcola la similarità coseno tra Alice e Bob, Alice e Carlo.
#    Chi le è più simile?
# 2. Perché usiamo la similarità coseno invece del prodotto scalare puro?
#    (Suggerimento: cosa succederebbe se Carlo votasse tutto doppio?)
# 3. Aggiungi un quarto utente "diana" con gusti simili ad Alice
#    e verifica che la similarità con Alice sia più alta di quella con Carlo.

# Questo è esattamente il meccanismo dei sistemi di raccomandazione
# e degli embedding semantici negli LLM.
```

Esercizio 2 — Gradient descent a mano

Obiettivo: implementare il ciclo base di apprendimento senza librerie.

```
import numpy as np
import matplotlib.pyplot as plt
```

```

# Problema: trova il valore di w che minimizza  $f(w) = (w - 7)^2$ 
# (Il minimo è ovviamente  $w=7$ , ma vogliamo vederlo trovare da solo)

def f(w):
    return (w - 7) ** 2

def gradiente_f(w):
    return 2 * (w - 7)

# Gradient descent manuale
w = 0.0          # punto di partenza
lr = 0.1         # learning rate
storia = [w]     # teniamo traccia del percorso

for step in range(30):
    grad = gradiente_f(w)
    w = w - lr * grad  # ← aggiornamento: muoviti nella direzione opposta al
    gradiente
    storia.append(w)

print(f"Valore finale di w: {w:.4f}")  # dovrebbe essere ≈ 7.0
print(f"Loss finale: {f(w):.6f}")     # dovrebbe essere ≈ 0.0

# Visualizza il percorso
plt.figure(figsize=(10, 4))
w_range = np.linspace(-2, 16, 200)
plt.plot(w_range, f(w_range), 'b-', label='f(w) = (w-7)2')
plt.scatter(storia, [f(w) for w in storia], c='red', s=20, zorder=5, label='passi
GD')
plt.xlabel('w'); plt.ylabel('f(w)')
plt.title('Gradient descent su f(w) = (w-7)2')
plt.legend(); plt.grid(True, alpha=0.3)
plt.savefig('gradient_descent.png', dpi=150, bbox_inches='tight')
plt.show()

# Domande:
# 1. Prova lr=0.5 e lr=1.1. Cosa succede?
# 2. Prova lr=0.01. Quanti step servono per arrivare vicino a w=7?
# 3. Modifica  $f(w) = (w-7)^2 + (w-3)^2$ . Dove si trova il minimo?
# (Risposta analitica: w=5. Verifica con GD.)

```

Esercizio 3 — Cross-entropy a mano

Obiettivo: capire intuitivamente cosa succede alla loss durante il training.

```

import numpy as np

# Simula le predizioni di un classificatore a 3 classi
# durante 5 "epoch" fittizie di training
etichetta_vera = 1  # la classe corretta è la 1

predizioni_per_epoch = [
    [0.33, 0.34, 0.33],  # epoch 1: modello casuale, incerto
    [0.20, 0.50, 0.30],  # epoch 2: inizia a capire

```



```

    [0.10, 0.70, 0.20], # epoch 3: migliora
    [0.05, 0.88, 0.07], # epoch 4: abbastanza sicuro
    [0.02, 0.96, 0.02], # epoch 5: molto sicuro
]

print(f"{'Epoch':>6} | {'P(classe corretta)':>20} | {'Cross-entropy loss':>18}")
print("-" * 52)
for i, probs in enumerate(predizioni_per_epoch, 1):
    p_corretta = probs[etichetta_vera]
    ce = -np.log(p_corretta)
    print(f"{i:>6} | {p_corretta:>20.3f} | {ce:>18.4f}")

# Osserva: man mano che P(classe corretta) aumenta, la loss diminuisce.
# La relazione è logaritmica: piccoli miglioramenti vicino a P=1
# producono grandi riduzioni di loss vicino a P=0.

# Domanda: a quale valore di P la cross-entropy è esattamente 1.0?
# (Suggerimento: risolvi  $-\log(P) = 1 \rightarrow P = ?$ )

```

Prossimo capitolo: Come impara una macchina — il ciclo completo di addestramento, dalla loss al gradient descent, con overfitting e tecniche di regolarizzazione.

Capitolo 4

Come impara una macchina

"Un modello non capisce nulla. Ma impara a sbagliare sempre meno."

4.1 Il ciclo dell'apprendimento

Nel capitolo precedente hai visto le singole operazioni matematiche. Ora vediamo come si combinano in un unico processo che trasforma dati grezzi in un modello funzionante.

Il ciclo di apprendimento di qualsiasi rete neurale — dal più semplice al più grande LLM — si riduce sempre a quattro passi che si ripetono migliaia di volte:

1. Predici — il modello prende un input e produce un output **2. Misura** — calcola quanto quell'output è sbagliato (la loss) **3. Attribuisce** — calcola quanto ogni peso ha contribuito all'errore (i gradienti) **4. Correggi** — sposta ogni peso nella direzione che riduce l'errore

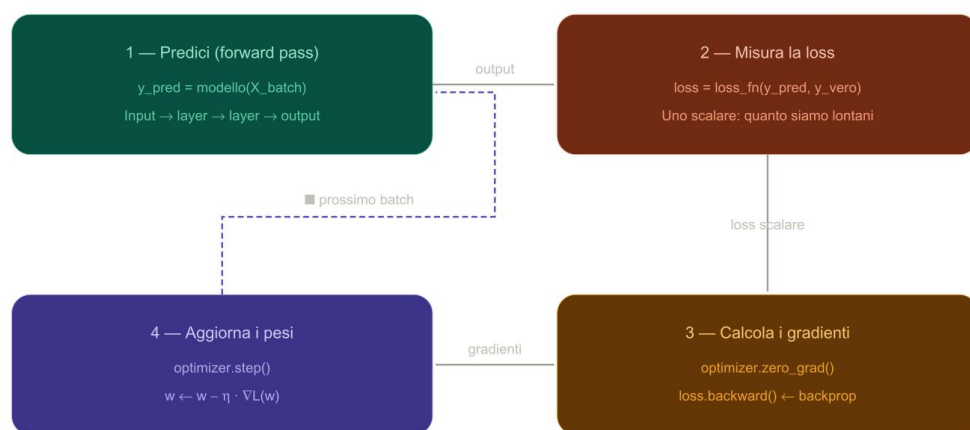


Figura 4.1 — Il ciclo di apprendimento: 4 passi che si ripetono milioni di volte

Figura 4.1 — Il ciclo di apprendimento: 4 passi che si ripetono milioni di volte.

Questo ciclo si chiama **training loop** ed è letteralmente il cuore di tutto il deep learning. Quando senti "il modello ha imparato", significa che questo ciclo è stato eseguito milioni di volte.

```
import torch
import torch.nn as nn

# Un training loop completo in PyTorch — solo 12 righe essenziali
modello = nn.Linear(3, 1)          # y = Wx + b, 3 input → 1 output
ottimizzatore = torch.optim.SGD(modello.parameters(), lr=0.01)
funzione_loss = nn.MSELoss()

for epoch in range(100):
    # 1. Predici
    y_pred = modello(X_train)
```

```

# 2. Misura l'errore
loss = funzione_loss(y_pred, y_train)

# 3. Azzeri i gradienti precedenti, poi calcola i nuovi
ottimizzatore.zero_grad()
loss.backward()          # backpropagation

# 4. Aggiorna i pesi
ottimizzatore.step()

if epoch % 10 == 0:
    print(f"Epoch {epoch}: loss = {loss.item():.4f}")

```

Ogni passata completa su tutto il dataset si chiama **epoch**. Il numero di epoch è un iperparametro che scegli tu: troppo poche e il modello non ha imparato abbastanza, troppe e rischi l'overfitting.

4.2 La loss function: il termometro dell'errore

La loss quantifica lo scarto tra ciò che il modello predice e ciò che dovrebbe predire. Minore è la loss, migliore è il modello su quel dataset.

MSE — per la regressione

L'**errore quadratico medio** (Mean Squared Error) è la loss standard quando devi predire valori continui (prezzi, temperature, redditi).

$$\text{MSE} = (1/N) \cdot \sum (\hat{y}_i - y_i)^2$$

Dove $\hat{y}_i$ è la predizione del modello e y_i è il valore reale.

```

import numpy as np

y_veri      = np.array([100, 200, 150, 300, 250])
y_predetti  = np.array([110, 190, 160, 280, 270])

# MSE a mano
errori = y_predetti - y_veri          # [10, -10, 10, -20, 20]
errori_quadrati = errori ** 2         # [100, 100, 100, 400, 400]
mse = errori_quadrati.mean()         # 220.0
print(f"MSE: {mse:.1f}")

# RMSE — stessa unità di misura dei dati originali
rmse = np.sqrt(mse)
print(f"RMSE: {rmse:.2f}")  # ≈ 14.83 → "in media sbaglio di ±14.83"

```

Perché quadratico? Il quadrato fa due cose: elimina il segno (un errore di -10 è grave quanto +10) e penalizza gli errori grandi più dei piccoli. Un errore di 20 non vale il doppio di uno da 10, ma il quadruplo (400 vs 100). Questo rende il modello particolarmente sensibile agli outlier.

Cross-entropy — per la classificazione

Già incontrata nel Cap. 3: penalizza la sicurezza nelle predizioni sbagliate. È la loss standard per qualsiasi task di classificazione, inclusi gli LLM (che "classificano" il prossimo token tra 50.000 candidati).

```

import torch
import torch.nn.functional as F

# Classificatore a 4 classi, batch di 3 esempi
logits = torch.tensor([
    [2.1, 0.5, -1.2, 0.3], # esempio 1: tende verso classe 0
    [0.1, 3.2, 0.8, 0.2], # esempio 2: tende verso classe 1
    [-0.5, 0.2, 2.8, 0.1], # esempio 3: tende verso classe 2
])
etichette_vere = torch.tensor([0, 1, 2]) # classi corrette

loss = F.cross_entropy(logits, etichette_vere)
print(f"Cross-entropy loss: {loss.item():.4f}")
# Bassa perché il modello tende già verso le classi corrette

```

MAE e Huber — varianti robuste

L'**errore assoluto medio** (MAE) usa `|errore|` invece di `errore2`. È meno sensibile agli outlier: un errore enorme non domina la loss.

La **Huber loss** combina le due: si comporta come MSE per errori piccoli (differenziabile, utile per l'ottimizzazione) e come MAE per errori grandi (robusta). È la scelta di default in molti problemi di regressione reali.

```

# Confronto delle tre loss su un outlier
errori = np.array([1, 2, 1, 50]) # 50 è un outlier

mse = np.mean(errori**2)
mae = np.mean(np.abs(errori))
print(f"MSE: {mse:.1f}") # 633.5 - dominata dall'outlier
print(f"MAE: {mae:.1f}") # 13.5 - molto più robusta

```

4.3 Gradient descent: scendere verso la soluzione

La loss è una funzione dei pesi del modello. Il gradient descent trova i pesi che minimizzano questa funzione muovendosi iterativamente nella direzione opposta al gradiente.

$$w_{\text{nuovo}} = w_{\text{vecchio}} - \eta \cdot \nabla L(w)$$

Il parametro η (eta) si chiama **learning rate**: controlla la dimensione di ogni passo.

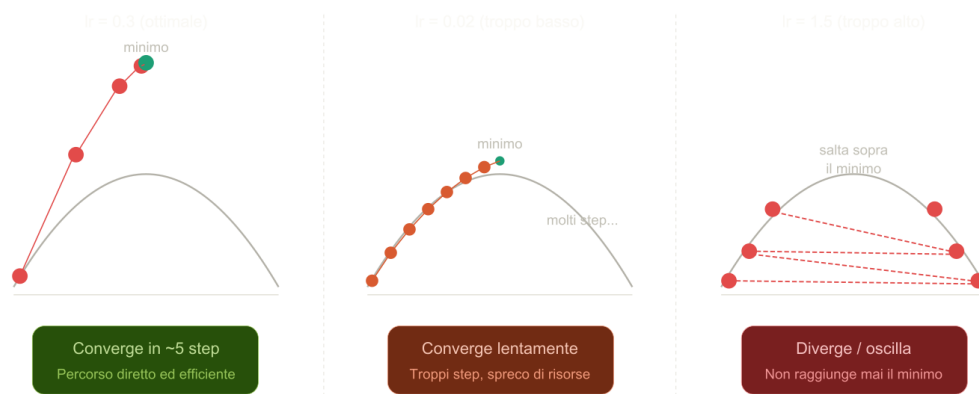


Figura 4.2 — Effetto del learning rate: ottimale, troppo basso, troppo alto

Figura 4.2 — Effetto del learning rate: ottimale, troppo basso, troppo alto. Gradient descent interattivo sulla funzione $f(w) = 0.4(w-2)^2 + 0.5$. Il punto verde è il minimo ($w=2$).

Le tre varianti principali

Batch Gradient Descent: calcola il gradiente su tutto il dataset prima di aggiornare i pesi. Stabile ma lentissimo su dataset grandi: con 10 milioni di esempi, fai un solo aggiornamento dopo aver processato tutti e 10 milioni.

Stochastic Gradient Descent (SGD): aggiorna i pesi dopo ogni singolo esempio. Velocissimo ma molto rumoroso: il gradiente di un singolo esempio può puntare in direzioni strane.

Mini-batch SGD: il compromesso ottimale e quello usato in pratica. Si processano gruppi di 32–256 esempi (un **batch**), si calcola il gradiente medio sul batch, si aggiornano i pesi. Abbastanza stabile da convergere, abbastanza veloce da essere pratico su GPU.

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

# Mini-batch training in PyTorch
dataset = TensorDataset(X_tensor, y_tensor)
loader = DataLoader(dataset, batch_size=32, shuffle=True)
# shuffle=True mescola i dati ad ogni epoch — importante per evitare
# che il modello impari l'ordine degli esempi invece dei pattern

for epoch in range(50):
    loss_totale = 0
    for X_batch, y_batch in loader:  # ← itera sui mini-batch
        y_pred = modello(X_batch)
        loss = funzione_loss(y_pred, y_batch)

        ottimizzatore.zero_grad()
        loss.backward()
        ottimizzatore.step()

    loss_totale += loss.item()

print(f"Epoch {epoch}: loss media = {loss_totale/len(loader):.4f}")
```

Il learning rate: il parametro più importante

Il learning rate è probabilmente l'iperparametro più critico dell'intero training. Impostarlo male compromette tutto:

- **Troppo alto:** i passi sono così grandi che il modello salta sopra il minimo e la loss oscilla o esplode invece di scendere.
- **Troppo basso:** i passi sono microscopici, il training converge ma richiede 100× più tempo.
- **Giusto:** la loss scende in modo costante e raggiungi un buon minimo in tempi ragionevoli.

```
# Visualizzare l'effetto del learning rate
learning_rates = [0.001, 0.01, 0.1, 1.0]

for lr in learning_rates:
    w = torch.tensor(10.0, requires_grad=True) # parte da w=10
    storia_loss = []

    for _ in range(50):
        loss = (w - 3.0) ** 2 # minimo in w=3
        loss.backward()
        with torch.no_grad():
            w -= lr * w.grad
        w.grad.zero_()
        storia_loss.append(loss.item())

    print(f"lr={lr:.3f} → w finale={w.item():.3f}, "
          f"loss finale={storia_loss[-1]:.4f}")

# lr=0.001 → lento, non converge in 50 step
# lr=0.01 → converge bene a w≈3
# lr=0.1 → converge veloce ma può oscillare
# lr=1.0 → diverge (overshooting)
```

4.4 Overfitting e underfitting: il dilemma della generalizzazione

Il vero obiettivo del ML non è minimizzare la loss sul training set — è generalizzare bene su dati mai visti. Questa distinzione è tutto.

Underfitting: il modello troppo semplice

Il modello non ha abbastanza capacità (parametri, profondità) per catturare i pattern nei dati. Sbaglia sia sul training set che sul validation set. Il sintomo è una loss alta su entrambi.

Cause comuni: modello troppo piccolo, training troppo breve, learning rate troppo basso, dati troppo pochi rispetto alla complessità del task.

Overfitting: il modello troppo complesso

Il modello memorizza il training set invece di generalizzare. Funziona perfettamente sui dati di training ma fallisce su quelli nuovi. Il sintomo classico: loss di training bassa, loss di validation alta e che cresce.

***Analogia:** uno studente che memorizza le risposte esatte degli esercizi del libro invece di capire i concetti. Sui vecchi esercizi ottiene 10/10. All'esame con domande nuove prende 4/10.*

Cause comuni: modello troppo grande rispetto ai dati disponibili, training troppo lungo, troppo pochi dati di training.

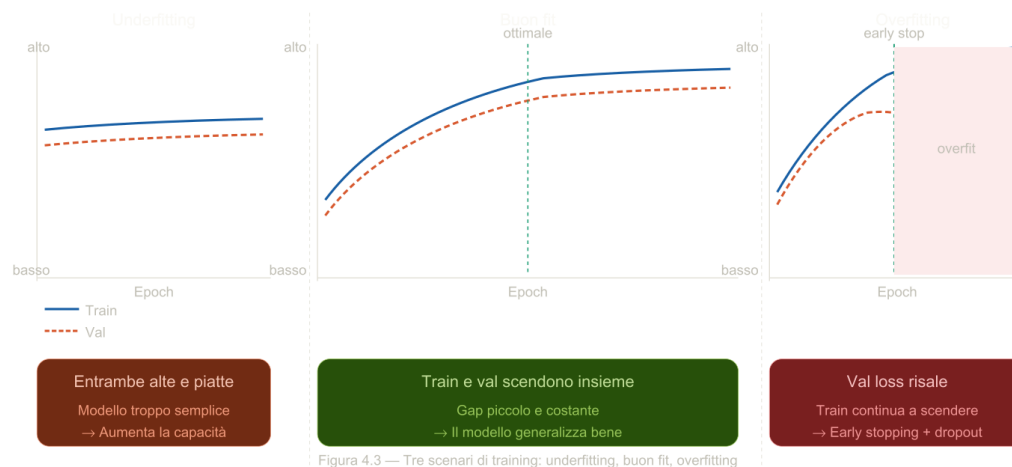


Figura 4.3 — Tre scenari di training: underfitting, buon fit, overfitting

Figura 4.3 — curve di training che mostrano underfitting, good fit, overfitting

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline

# Genera dati con una relazione quadratica + rumore
np.random.seed(42)
X = np.linspace(-3, 3, 50).reshape(-1, 1)
y = 0.5 * X.ravel()**2 - X.ravel() + np.random.randn(50) * 0.8

# Tre modelli di complessità crescente
gradi = [1, 2, 15] # grado 1=underfitting, 2=giusto, 15=overfitting
nomi = ['Underfitting (grado 1)', 'Buon fit (grado 2)', 'Overfitting (grado 15)']

fig, assi = plt.subplots(1, 3, figsize=(15, 4))
X_plot = np.linspace(-3, 3, 200).reshape(-1, 1)

for ax, grado, nome in zip(assi, gradi, nomi):
    pipeline = Pipeline([
        ('poly', PolynomialFeatures(degree=grado)),
        ('lr', LinearRegression())
    ])
    pipeline.fit(X, y)

    ax.scatter(X, y, alpha=0.6, s=20, color='steelblue')
    ax.plot(X_plot, pipeline.predict(X_plot), 'r-', linewidth=2)
    ax.set_title(nome, fontsize=11)
    ax.set_ylim(-4, 8)

plt.tight_layout()
plt.savefig('overfitting.png', dpi=150, bbox_inches='tight')
plt.show()
```

4.5 Tecniche di regolarizzazione

La regolarizzazione è l'insieme delle tecniche per ridurre l'overfitting senza ridurre la capacità del modello.

L2 Regularization (Weight Decay)

Aggiunge alla loss un termine proporzionale alla somma dei quadrati dei pesi: $\text{loss_totale} = \text{loss} + \lambda \cdot \sum w_i^2$

Questo penalizza i pesi grandi, spingendo il modello verso soluzioni più "semplici" dove nessun peso è dominante. In PyTorch si chiama `weight_decay` nell'ottimizzatore.

```
# L2 regularization in PyTorch - basta un parametro in AdamW
ottimizzatore = torch.optim.AdamW(
    modello.parameters(),
    lr=1e-3,
    weight_decay=1e-4  # λ: tipicamente tra 1e-5 e 1e-2
)
```

Dropout

Durante il training, a ogni forward pass si "spengono" casualmente una frazione dei neuroni (impostando la loro attivazione a zero). Il modello non può fare affidamento su nessun neurone specifico, quindi deve imparare rappresentazioni ridondanti e robuste.

A inference time, tutti i neuroni sono attivi (scalati per $1 - p$ per compensare).

```
import torch.nn as nn

modello_con_dropout = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Dropout(p=0.5),      # ← spegne il 50% dei neuroni durante training
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Dropout(p=0.3),      # ← spegne il 30% dei neuroni
    nn.Linear(128, 10),
)

# IMPORTANTE: devi dire al modello se sei in training o inference
modello_con_dropout.train()  # dropout attivo
modello_con_dropout.eval()   # dropout disattivato (inference)
```

Intuizione: immagina di addestrare una squadra di calcio in cui, ad ogni allenamento, 5 giocatori casuali vengono bendati. Gli altri 6 imparano a coprire tutti i ruoli e a non dipendere dai compagni bendati. Quando giocano la partita vera (tutti senza benda), sono molto più robusti.

Batch Normalization

Dopo ogni layer, normalizza le attivazioni del mini-batch (media≈0, varianza≈1). Stabilizza il training, permette learning rate più alti e agisce come una forma di regolarizzazione implicita.

```
modello_con_bn = nn.Sequential(
    nn.Linear(784, 256),
    nn.BatchNorm1d(256),  # ← normalizza il batch dopo il layer lineare
    nn.ReLU(),
    nn.Linear(256, 10),
)
```



```
)
```

Early Stopping

Monitora la validation loss durante il training. Se non migliora per N epoch consecutive (la **patience**), interrompi il training e riprendi i pesi dal punto migliore.

```
miglior_loss_val = float('inf')
patience         = 10
epoche_senza_miglioramento = 0
miglior_pesi     = None

for epoch in range(1000):
    # ... training loop ...
    loss_val_corrente = valuta_su_validation()

    if loss_val_corrente < miglior_loss_val:
        miglior_loss_val = loss_val_corrente
        epoche_senza_miglioramento = 0
        # Salva una copia dei pesi migliori
        miglior_pesi = {k: v.clone() for k, v in modello.state_dict().items()}
    else:
        epoche_senza_miglioramento += 1
        if epoche_senza_miglioramento >= patience:
            print(f"Early stopping all'epoch {epoch}")
            modello.load_state_dict(miglior_pesi) # ripristina i pesi migliori
            break
```

Data Augmentation

Invece di ridurre la capacità del modello, aumenta artificialmente la dimensione del training set creando varianti degli esempi esistenti.

Per le immagini: flip orizzontale, rotazione, ritaglio, variazione di colore. Per il testo: traduzione e ritraduzione (backtranslation), sostituzione con sinonimi. Per i dati tabellari: aggiunta di rumore gaussiano, mixup (interpolazione tra due esempi).

```
from torchvision import transforms

# Pipeline di augmentation per immagini
augmentation = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.RandomCrop(224, padding=16),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]),
])

# Attenzione: l'augmentation va applicata SOLO al training set, mai al
validation/test
```

4.6 Come leggere le curve di training

Le curve di loss durante il training sono una fonte di diagnosi potentissima. Impara a leggerle.

Training loss scende, validation loss scende in parallelo → tutto bene, il modello sta imparando e generalizzando.

Entrambe le loss alte e piatte → underfitting. Il modello non riesce ad imparare. Controlla: learning rate troppo basso? Modello troppo piccolo? Bug nel codice?

Training loss scende, validation loss scende poi risale → overfitting classico. Il punto in cui la validation loss inizia a risalire è il punto ottimale di early stopping.

Loss che oscilla selvaggiamente → learning rate troppo alto. Riducilo.

Loss che scende molto lentamente → learning rate troppo basso. Aumentalo, oppure usa uno scheduler.

Loss che scende a un certo punto e si blocca → possibile minimo locale, o learning rate da ridurre. Prova a riavviare con learning rate più basso da quel punto.

```
import matplotlib.pyplot as plt

# Visualizzare le curve di training - da fare sempre
storico = {'train_loss': [], 'val_loss': []}

for epoch in range(num_epochs):
    train_loss = esegui_training_epoch()
    val_loss = valuta_su_validation()

    storico['train_loss'].append(train_loss)
    storico['val_loss'].append(val_loss)

plt.figure(figsize=(10, 4))
plt.plot(storico['train_loss'], label='Training loss', color='steelblue')
plt.plot(storico['val_loss'], label='Validation loss', color='coral')
plt.axvline(x=np.argmin(storico['val_loss']),
            color='green', linestyle='--', label='Best epoch')
plt.xlabel('Epoch'); plt.ylabel('Loss')
plt.legend(); plt.grid(True, alpha=0.3)
plt.title('Curve di training')
plt.savefig('training_curves.png', dpi=150)
plt.show()
```

4.7 Iperparametri: cosa scegli tu, non il modello

I **parametri** del modello (pesi e bias) vengono appresi automaticamente. Gli **iperparametri** sono le scelte che fai tu prima di iniziare il training.

Iperparametro	Valori tipici	Come influenza
<code>learning_rate</code>	1e-4 — 1e-2	Velocità di convergenza, stabilità
<code>batch_size</code>	16 — 512	Stabilità del gradiente, uso memoria
<code>num_epochs</code>	10 — 1000	Quanto a lungo si allena
<code>weight_decay</code>	1e-5 — 1e-2	Forza della regolarizzazione L2
<code>dropout_rate</code>	0.1 — 0.5	Forza della regolarizzazione dropout
<code>num_layers</code>	2 — 100+	Capacità del modello
<code>hidden_size</code>	64 — 4096	Capacità di ogni layer

Come si scelgono? Principalmente con esperienza e con il validation set. Le strategie sistematiche includono il **grid search** (prova tutte le combinazioni), il **random search** (prova combinazioni

casuali, spesso più efficace) e il **Bayesian optimization** (usa i risultati precedenti per scegliere il prossimo punto da esplorare).

```
# Esempio di random search su learning rate e weight decay
import numpy as np

risultati = []
for _ in range(20):
    lr = 10 ** np.random.uniform(-4, -1)          # campiona in scala logaritmica
    wd = 10 ** np.random.uniform(-5, -2)

    # addestra il modello con questi iperparametri
    val_loss = addestra_e_valuta(lr=lr, weight_decay=wd)
    risultati.append({'lr': lr, 'wd': wd, 'val_loss': val_loss})

# Ordina per val_loss e prendi i migliori
migliori = sorted(risultati, key=lambda x: x['val_loss'])[:3]
print("Migliori configurazioni trovate:")
for r in migliori:
    print(f"  lr={r['lr']:.2e}, wd={r['wd']:.2e} → val_loss={r['val_loss']:.4f}")
```

Riepilogo del Capitolo 4

Hai imparato:

- Il **training loop**: predici → misura → attribuisce → correggi. Quattro passi ripetuti milioni di volte.
- Le **loss functions** principali: MSE per la regressione, cross-entropy per la classificazione.
- Il **gradient descent** e le sue varianti: batch, SGD, mini-batch (quello usato in pratica).
- Il **learning rate**: l'iperparametro più critico. Troppo alto = instabilità, troppo basso = lentezza.
- **Overfitting** (memorizza il training) vs **underfitting** (troppo semplice per imparare).
- Le **tecniche di regolarizzazione**: L2, dropout, batch normalization, early stopping, data augmentation.
- Come **leggere le curve di training** per diagnosticare problemi.
- La differenza tra **parametri** (appresi) e **iperparametri** (scelti da te).

Esercizio 1 — Training loop da zero

Obiettivo: costruire un training loop completo senza usare `nn.Module`.

```
import numpy as np
import matplotlib.pyplot as plt

# Dataset: regressione lineare  $y = 2x + 1 + \text{rumore}$ 
np.random.seed(42)
X = np.random.randn(100, 1)
y = 2 * X + 1 + np.random.randn(100, 1) * 0.5

# Inizializza i pesi manualmente
w = np.random.randn(1, 1) * 0.1  # peso
```

```

b = np.zeros((1, 1))          # bias
lr = 0.01

storia_loss = []

for epoch in range(200):
    # 1. Forward pass: predici
    y_pred = X @ w + b

    # 2. Calcola la MSE loss
    loss = np.mean((y_pred - y) ** 2)
    storia_loss.append(loss)

    # 3. Calcola i gradienti a mano (derivate della MSE)
    #  $dL/dw = (2/N) \cdot X^T \cdot (\hat{y} - y)$ 
    #  $dL/db = (2/N) \cdot \sum(\hat{y} - y)$ 
    N = len(X)
    dw = (2/N) * X.T @ (y_pred - y)
    db = (2/N) * np.sum(y_pred - y)

    # 4. Aggiorna i pesi
    w -= lr * dw
    b -= lr * db

print(f"Peso imparato: {w[0,0]:.3f} (vero: 2.0)")
print(f"Bias imparato: {b[0,0]:.3f} (vero: 1.0)")

# Visualizza la curva di loss
plt.plot(storia_loss)
plt.xlabel('Epoch'); plt.ylabel('MSE Loss')
plt.title('Convergenza del training loop')
plt.grid(True, alpha=0.3)
plt.show()

# Domande:
# 1. Prova lr=0.1 e lr=0.001. Cosa osservi nelle curve?
# 2. Aumenta il rumore a 2.0. Il modello converge comunque?
# 3. Aggiungi una seconda feature:  $y = 2x_1 + 3x_2 + 1 + \text{rumore}$ 

```

Esercizio 2 — Osservare l'overfitting

Obiettivo: riprodurre il fenomeno dell'overfitting e applicare l'early stopping.

```

import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt

# Dataset piccolo intenzionalmente (per forzare l'overfitting)
np.random.seed(42)
n_train = 50
X_np = np.random.randn(n_train, 10)          # 10 feature
y_np = X_np[:, 0] * 2 + np.random.randn(n_train) * 3 # solo la prima feature conta

X_train = torch.FloatTensor(X_np[:40])

```

```

y_train = torch.FloatTensor(y_np[:40]).unsqueeze(1)
X_val = torch.FloatTensor(X_np[40:])
y_val = torch.FloatTensor(y_np[40:]).unsqueeze(1)

# Modello volutamente grande per forzare l'overfitting
modello = nn.Sequential(
    nn.Linear(10, 512), nn.ReLU(),
    nn.Linear(512, 512), nn.ReLU(),
    nn.Linear(512, 1)
)

ottimizzatore = torch.optim.Adam(modello.parameters(), lr=1e-3)
loss_fn = nn.MSELoss()

train_losses, val_losses = [], []

for epoch in range(300):
    modello.train()
    pred_train = modello(X_train)
    loss_train = loss_fn(pred_train, y_train)
    ottimizzatore.zero_grad()
    loss_train.backward()
    ottimizzatore.step()

    modello.eval()
    with torch.no_grad():
        loss_val = loss_fn(modello(X_val), y_val)

    train_losses.append(loss_train.item())
    val_losses.append(loss_val.item())

plt.figure(figsize=(10, 4))
plt.plot(train_losses, label='Training loss')
plt.plot(val_losses, label='Validation loss')
plt.xlabel('Epoch'); plt.ylabel('MSE Loss')
plt.legend(); plt.grid(True, alpha=0.3)
plt.title('Overfitting: la val loss risale mentre la train loss scende')
plt.show()

# Domande:
# 1. In quale epoch inizia l'overfitting?
# 2. Aggiungi Dropout(0.5) dopo ogni ReLU. L'overfitting migliora?
# 3. Aggiungi weight_decay=1e-3 all'ottimizzatore. Effetto?
# 4. Implementa l'early stopping con patience=20.

```

Prossimo capitolo: Il Neurone Artificiale e le Reti Neurali — dalla singola unità computazionale all'architettura MLP, con la backpropagation spiegata passo per passo.

Capitolo 4 — Approfondimento pratico

Dal dataset al modello: un progetto completo

Prima di andare avanti verso le reti neurali, fermiamoci a mettere insieme tutto ciò che abbiamo visto finora in un unico flusso di lavoro reale. Questo capitolo è il tuo primo progetto ML end-to-end.

Il problema: predire il churn dei clienti

Un'azienda di telecomunicazioni vuole sapere quali clienti stanno per abbandonare il servizio (churn = abbandono). Se riesce a identificarli in anticipo, può fare un'offerta mirata e trattenerli.

Questo è un problema di **classificazione binaria**: per ogni cliente, il modello deve rispondere "abbandonerà" (1) oppure "rimarrà" (0).

È un problema perfetto per iniziare perché:

- I dati sono tabellari — il formato più comune nel mondo reale
- Ci sono sia feature numeriche che categoriche
- Le classi sono sbilanciate (pochi clienti abbandonano — problema frequente)
- Le metriche di valutazione hanno un significato di business immediato

Fase 1 — Caricare e ispezionare i dati

Il primo gesto in qualsiasi progetto ML è guardare i dati. Non i parametri del modello, non l'architettura — i dati.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Dataset Telco Customer Churn — disponibile su Kaggle
# Per questo esempio lo ricreiamo sinteticamente con le stesse caratteristiche
np.random.seed(42)
n = 1000

df = pd.DataFrame({
    # Feature demografiche
    'età': np.random.randint(18, 75, n),
    'anzianità_mesi': np.random.randint(1, 72, n), # quanto tempo è cliente

    # Feature sul contratto
    'tipo_contratto': np.random.choice(['mensile', 'annuale', 'biennale'], n,
                                         p=[0.55, 0.25, 0.20]),
    'fatturazione_online': np.random.choice(['Sì', 'No'], n),

    # Feature sull'utilizzo
```

```

'costo_mensile':    np.round(np.random.uniform(20, 120, n), 2),
'gb_usati_mese':    np.round(np.random.exponential(15, n), 1),
'chiamate_supporto': np.random.randint(0, 8, n),

# Feature sul servizio
'fibra_optica':     np.random.choice(['Si', 'No'], n, p=[0.6, 0.4]),
})

# Label: i clienti mensili con molte chiamate al supporto tendono ad abbandonare di più
prob_churn = (
    0.05 +
    0.35 * (df['tipo_contratto'] == 'mensile') +
    0.04 * df['chiamate_supporto'] +
    0.002 * df['costo_mensile'] -
    0.003 * df['anzianità_mesi']
).clip(0, 1)

df['churn'] = (np.random.rand(n) < prob_churn).astype(int)

print("=== PRIMA ISPEZIONE ===")
print(f"Shape del dataset: {df.shape}")          # (1000, 9)
print(f"\nTipi di colonna:\n{df.dtypes}")
print(f"\nPrime 3 righe:\n{df.head(3)}")

```

Osserva l'output con attenzione. Già qui puoi capire molto:

- Quante righe e colonne hai (`shape`)
- Quali colonne sono numeriche (`int64`, `float64`) e quali testuali (`object`)
- Se i dati hanno senso "a occhio"

```

print("\n=== VALORI MANCANTI ===")
print(df.isnull().sum())
# Qui non ce ne sono, ma in dati reali ci sarebbero quasi sempre

print("\n=== DISTRIBUZIONE DELLA LABEL ===")
conteggio = df['churn'].value_counts()
print(conteggio)
print(f"\nTasso di churn: {df['churn'].mean()*100:.1f}%")
# Output atteso: ~27% di churn – dataset sbilanciato

```

Questo ultimo controllo è critico. Se il 27% abbandona e il 73% rimane, un modello stupido che dice sempre "rimarrà" ottiene il 73% di accuratezza senza aver imparato nulla. Per questo le metriche contano più dell'accuratezza grezza.

```

print("\n=== STATISTICHE DESCRITTIVE ===")
print(df.describe().round(2))
# Guarda: media, min, max, deviazione standard di ogni colonna numerica
# Segnali di allarme: valori impossibili (età negativa?), range anomali

```

Fase 2 — Analisi esplorativa: cercare segnali

Dopo l'ispezione di base, cerchi pattern che anticipano la label. Ogni grafico risponde a una domanda specifica.

```

fig, assi = plt.subplots(2, 3, figsize=(15, 8))
fig.suptitle('Analisi esplorativa - Churn vs. feature principali', fontsize=13)

# Domanda 1: il tipo di contratto influenza il churn?
churn_per_contratto = df.groupby('tipo_contratto')['churn'].mean() * 100
assi[0,0].bar(churn_per_contratto.index, churn_per_contratto.values,
              color=['#E24B4A', '#378ADD', '#1D9E75'])
assi[0,0].set_title('Tasso churn per tipo contratto')
assi[0,0].set_ylabel('% churn')

# Domanda 2: il costo mensile è diverso tra chi abbandona e chi resta?
for label, colore in [(0, '#378ADD'), (1, '#E24B4A')]:
    dati = df[df['churn'] == label]['costo_mensile']
    assi[0,1].hist(dati, bins=25, alpha=0.6, color=colore,
                  label='Rimane' if label==0 else 'Abbandona')
assi[0,1].set_title('Distribuzione costo mensile')
assi[0,1].legend()

# Domanda 3: le chiamate al supporto predicono il churn?
churn_per_chiamate = df.groupby('chiamate_supporto')['churn'].mean() * 100
assi[0,2].plot(churn_per_chiamate.index, churn_per_chiamate.values,
               'o-', color='#D85A30')
assi[0,2].set_title('Tasso churn per chiamate supporto')
assi[0,2].set_ylabel('% churn')
assi[0,2].set_xlabel('N. chiamate')

# Domanda 4: l'anzianità protegge dal churn?
df['anzianità_fascia'] = pd.cut(df['anzianità_mesi'],
                               bins=[0,12,24,48,72],
                               labels=['0-12m', '13-24m', '25-48m', '49-72m'])
churn_x_anziania = df.groupby('anzianità_fascia', observed=True)['churn'].mean()*100
assi[1,0].bar(churn_x_anziania.index, churn_x_anziania.values, color='#534AB7')
assi[1,0].set_title('Tasso churn per anzianità')
assi[1,0].set_ylabel('% churn')

# Domanda 5: correlazione tra feature numeriche e churn
numeriche = ['età', 'anzianità_mesi', 'costo_mensile', 'gb_usati_mese',
             'chiamate_supporto', 'churn']
corr = df[numeriche].corr()['churn'].drop('churn').sort_values()
colori = ['#E24B4A' if v > 0 else '#378ADD' for v in corr.values]
assi[1,1].barh(corr.index, corr.values, color=colori)
assi[1,1].axvline(0, color='gray', linewidth=0.8)
assi[1,1].set_title('Correlazione con churn')

# Domanda 6: fibra ottica e churn
churn_fibra = df.groupby('fibra_ottica')['churn'].mean() * 100
assi[1,2].bar(churn_fibra.index, churn_fibra.values, color=['#EF9F27', '#1D9E75'])
assi[1,2].set_title('Tasso churn per tipo connessione')
assi[1,2].set_ylabel('% churn')

plt.tight_layout()
plt.savefig('eda_churn.png', dpi=150, bbox_inches='tight')
plt.show()

```

Cosa stai cercando in questi grafici? Differenze sistematiche tra chi abbandona e chi rimane. Se una feature non mostra nessuna differenza tra i due gruppi, probabilmente non aiuterà il modello a discriminare.

Fase 3 — Preprocessing: preparare i dati per il modello

I modelli ML lavorano solo con numeri. Questa fase trasforma il dataset grezzo nel formato corretto, senza perdere informazioni e senza introdurre data leakage.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

# --- 3a. Separa feature e label ---
# Rimuoviamo anche 'anzianità_fascia' che abbiamo creato solo per l'EDA
X = df.drop(columns=['churn', 'anzianità_fascia'])
y = df['churn']

print(f"Feature matrix shape: {X.shape}") # (1000, 8)
print(f"Label vector shape: {y.shape}") # (1000,)
print(f"\nColonne di X:\n{X.dtypes}")

# --- 3b. Split train / validation / test ---
# Prima separiamo il test set (20%) — non lo toccheremo più fino alla fine
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y # ← IMPORTANTE: mantiene la stessa % di churn in ogni split
)

# Poi dal rimanente creiamo il validation set (20% del totale = 25% del rimanente)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp,
    test_size=0.25,
    random_state=42,
    stratify=y_temp
)

print(f"\nTraining: {len(X_train)} campioni ({len(X_train)/len(X)*100:.0f}%)")
print(f"Validation: {len(X_val)} campioni ({len(X_val)/len(X)*100:.0f}%)")
print(f"Test: {len(X_test)} campioni ({len(X_test)/len(X)*100:.0f}%)")

# Verifica che stratify abbia funzionato
print(f"\nChurn nel training: {y_train.mean()*100:.1f}%")
print(f"Churn nel validation: {y_val.mean()*100:.1f}%")
print(f"Churn nel test: {y_test.mean()*100:.1f}%")
# Dovrebbero essere tutti circa uguali (~27%)
```

Perché `stratify=y`? Senza di esso, per pura sfortuna potresti ritrovarti con il 20% di churn nel training e il 40% nel test — il modello verrebbe valutato su una distribuzione molto diversa da quella su cui ha imparato. `stratify` garantisce che ogni split mantenga la stessa proporzione di classi.

```
# --- 3c. Pipeline di preprocessing ---
# Definiamo separatamente le colonne numeriche e categoriche
```

```

colonne_numeriche = ['età', 'anzianità_mesi', 'costo_mensile',
                     'gb_usati_mese', 'chiamate_supporto']

colonne_categoriche = ['tipo_contratto', 'fatturazione_online', 'fibra_ottica']

# ColumnTransformer applica trasformazioni diverse a colonne diverse
preprocessor = ColumnTransformer(transformers=[
    # Alle colonne numeriche: standardizzazione (media=0, std=1)
    ('num', StandardScaler(), colonne_numeriche),

    # Alle colonne categoriche: one-hot encoding
    # drop='first' elimina una categoria per evitare multicollinearità
    ('cat', OneHotEncoder(drop='first', sparse_output=False), colonne_categoriche),
])

# fit solo sul training set – MAI sul validation o test
preprocessor.fit(X_train)

# transform su tutti e tre gli split usando i parametri del training
X_train_proc = preprocessor.transform(X_train)
X_val_proc   = preprocessor.transform(X_val)
X_test_proc  = preprocessor.transform(X_test)

print(f"\nShape dopo preprocessing:")
print(f" X_train: {X_train_proc.shape}") # (600, N_feature_dopo_encoding)
print(f" X_val:   {X_val_proc.shape}")
print(f" X_test:  {X_test_proc.shape}")

```

Fase 4 — Scelta e addestramento del modello

Per un problema tabellare come questo, un buon punto di partenza è la **regressione logistica** — semplice, veloce, interpretabile. La useremo come baseline prima di provare modelli più complessi.

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, f1_score)

# --- 4a. Definiamo il modello ---
# max_iter=1000: numero massimo di iterazioni del solver (equivalente alle epoch)
# C=1.0: inverso della forza di regolarizzazione L2 (C piccolo = più
regolarizzazione)
# class_weight='balanced': corregge automaticamente per lo sbilanciamento delle
classi
modello = LogisticRegression(
    max_iter=1000,
    C=1.0,
    class_weight='balanced', # ← gestisce il dataset sbilanciato
    random_state=42
)

# --- 4b. Addestriamo sul training set ---
modello.fit(X_train_proc, y_train)
# In scikit-learn .fit() esegue internamente il training loop con gradient descent

```

```
print("Modello addestrato.")
print(f"Numero di feature usate: {modello.coef_.shape[1]}")
print(f"Iterazioni fino a convergenza: {modello.n_iter_[0]}")
```

Il modello è addestrato. Ma "addestrato" non significa ancora "buono". Dobbiamo misurarlo.

Fase 5 — Valutazione sul validation set

Regola d'oro: usi il validation set per capire come sta andando il modello durante lo sviluppo. Il test set lo tocchi solo alla fine, una volta sola.

```
# --- 5a. Predizioni sul validation set ---
# predict() restituisce la classe (0 o 1)
y_val_pred = modello.predict(X_val_proc)

# predict_proba() restituisce la probabilità di ogni classe
#[:,1] prende la probabilità della classe 1 (churn)
y_val_prob = modello.predict_proba(X_val_proc)[: , 1]

# --- 5b. Metriche di classificazione ---
print("=== RISULTATI SUL VALIDATION SET ===\n")
print(classification_report(y_val, y_val_pred,
                           target_names=['Rimane', 'Abbandona']))
```

L'output di `classification_report` contiene quattro metriche chiave. Vediamole una per una:

Precision (per la classe "Abbandona"): di tutti i clienti che il modello ha marcato come "abbandona", quanti lo hanno davvero fatto? Una precision bassa significa molti falsi allarmi — contatteremo clienti che in realtà non avrebbero abbandonato.

Recall (per la classe "Abbandona"): di tutti i clienti che hanno davvero abbandonato, quanti ne ha catturati il modello? Un recall basso significa che molti churner ci sfuggono senza che facciamo niente.

F1 Score: media armonica di precision e recall. Utile quando le due metriche sono in tensione tra loro (spesso lo sono).

Support: quanti esempi ci sono per ogni classe nel validation set.

```
# --- 5c. Confusion matrix: vedere gli errori in dettaglio ---
cm = confusion_matrix(y_val, y_val_pred)
print(f"\nConfusion Matrix:\n{cm}")
print(f"""
    Predetto: Rimane  Predetto: Abbandona
Vero: Rimane      {cm[0,0]:>6}          {cm[0,1]:>6}      ← Falsi Positivi
Vero: Abbandona  {cm[1,0]:>6}          {cm[1,1]:>6}      ← Falsi Negativi
""")
# Veri Negativi (rimane e predice rimane): buono
# Veri Positivi (abbandona e predice abbandona): buono
# Falsi Positivi (rimane ma predice abbandona): spreco di offerte
# Falsi Negativi (abbandona ma predice rimane): cliente perso

# --- 5d. AUC-ROC: misura la qualità del ranking ---
# Indipendente dalla soglia di decisione (default 0.5)
# 0.5 = casuale, 1.0 = perfetto
```

```
auc = roc_auc_score(y_val, y_val_prob)
print(f"AUC-ROC sul validation set: {auc:.4f}")
```

Quale metrica ottimizzare? Dipende dal contesto di business. Se ogni cliente perso costa molto, massimizza il recall. Se il costo delle offerte di retention è alto e non vuoi sprecarle su chi non abbandonerebbe comunque, massimizza la precision. In mancanza di indicazioni specifiche, l'AUC-ROC è la metrica più equilibrata.

Fase 6 — Iterazione: migliorare il modello

La prima versione raramente è la migliore. Ora confrontiamo con un modello più potente per capire se c'è margine di miglioramento.

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

# Proviamo tre modelli e confrontiamo sul validation set
modelli_da_testare = {
    'Regressione Logistica': LogisticRegression(
        max_iter=1000, C=1.0, class_weight='balanced', random_state=42
    ),
    'Random Forest': RandomForestClassifier(
        n_estimators=100,      # 100 alberi nel forest
        max_depth=8,          # profondità massima di ogni albero
        class_weight='balanced',
        random_state=42,
        n_jobs=-1              # usa tutti i core disponibili
    ),
    'Gradient Boosting': GradientBoostingClassifier(
        n_estimators=100,
        learning_rate=0.1,    # quanto pesa ogni nuovo albero
        max_depth=4,
        random_state=42
    ),
}

print(f"{'Modello':<28} {'AUC-ROC':>8} {'F1':>8} {'Recall':>8} {'Precision':>10}")
print("-" * 66)

risultati = {}
for nome, mod in modelli_da_testare.items():
    mod.fit(X_train_proc, y_train)
    proba = mod.predict_proba(X_val_proc)[:, 1]
    pred = mod.predict(X_val_proc)

    auc = roc_auc_score(y_val, proba)
    f1 = f1_score(y_val, pred)
    from sklearn.metrics import recall_score, precision_score
    rec = recall_score(y_val, pred)
    prec = precision_score(y_val, pred)

    risultati[nome] = {'auc': auc, 'f1': f1, 'modello': mod}
    print(f"{'nome':<28} {'auc':>8.4f} {'f1':>8.4f} {'rec':>8.4f} {'prec':>10.4f}")

# Il modello migliore sul validation set
migliore_nome = max(risultati, key=lambda k: risultati[k]['auc'])
```

```
modello_finale = risultati[migliore_nome]['modello']
print(f"\nModello selezionato: {migliore_nome}")
```

Fase 7 — Valutazione finale sul test set

Solo ora, dopo aver scelto il modello migliore, tocchiamo il test set. Una volta sola.

```
print("\n" + "="*50)
print("VALUTAZIONE FINALE SUL TEST SET")
print("="*50)
print("(Questo numero è quello che riporti. Non tornarci mai.)\n")

y_test_pred = modello_finale.predict(X_test_proc)
y_test_prob = modello_finale.predict_proba(X_test_proc)[: , 1]

auc_test = roc_auc_score(y_test, y_test_prob)
print(f"AUC-ROC finale: {auc_test:.4f}")
print(f"\n{classification_report(y_test, y_test_pred,
target_names=['Rimane', 'Abbandona'])}")

# Confronto validation vs test - non dovrebbero essere troppo diversi
auc_val = risultati[migliore_nome]['auc']
print(f"AUC Validation: {auc_val:.4f}")
print(f"AUC Test:          {auc_test:.4f}")
print(f"Differenza:        {abs(auc_val - auc_test):.4f}")

if abs(auc_val - auc_test) > 0.05:
    print("ATTENZIONE: differenza > 5% tra validation e test.")
    print("Possibile overfitting sul validation set (hai fatto troppe prove?).")
else:
    print("Differenza accettabile - il modello generalizza bene.")
```

Fase 8 — Interpretare il modello

Un modello che funziona ma che non sai spiegare è difficile da fidarsi e da migliorare. Guarda sempre cosa ha imparato.

```
# Feature importance per Random Forest / Gradient Boosting
if hasattr(modello_finale, 'feature_importances_'):

    # Recupera i nomi delle feature dopo l'encoding
    nomi_num = colonne_numeriche
    nomi_cat = preprocessor.named_transformers_['cat'].get_feature_names_out(
        colonne_categoriche
    ).tolist()
    tutti_i_nomi = nomi_num + nomi_cat

    importanze = pd.Series(
        modello_finale.feature_importances_,
        index=tutti_i_nomi
    ).sort_values(ascending=False)
```

```

print("\nFeature importance (top 8):")
print(importanze.head(8).round(4).to_string())

plt.figure(figsize=(10, 5))
importanze.head(10).plot(kind='barh', color='#378ADD')
plt.title('Feature più importanti per il modello')
plt.xlabel('Importanza relativa')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.savefig('feature_importance.png', dpi=150, bbox_inches='tight')
plt.show()

```

Fase 9 — Usare il modello su nuovi dati

Il modello addestrato ora può fare predizioni su clienti nuovi.

```

# Simuliamo 3 nuovi clienti da valutare
nuovi_clienti = pd.DataFrame({
    'età': [45, 22, 60],
    'anzianità_mesi': [48, 3, 24],
    'tipo_contratto': ['biennale', 'mensile', 'mensile'],
    'fatturazione_online': ['Sì', 'No', 'Sì'],
    'costo_mensile': [55.0, 89.0, 102.0],
    'gb_usati_mese': [12.5, 8.0, 25.0],
    'chiamate_supporto': [1, 5, 6],
    'fibra_optica': ['No', 'Sì', 'Sì'],
})

# Stessa pipeline di preprocessing del training
nuovi_proc = preprocessor.transform(nuovi_clienti)

# Predizioni
predizioni = modello_finale.predict(nuovi_proc)
probabilita = modello_finale.predict_proba(nuovi_proc)[:, 1]

print("Predizioni sui nuovi clienti:")
for i, (pred, prob) in enumerate(zip(predizioni, probabilita)):
    etichetta = "CHURN PROBABILE" if pred == 1 else "cliente stabile"
    print(f" Cliente {i+1}: {etichetta} (probabilità churn: {prob*100:.1f}%)")

```

Riepilogo del flusso completo

Questo è il processo che userai in quasi ogni progetto ML tabellare:

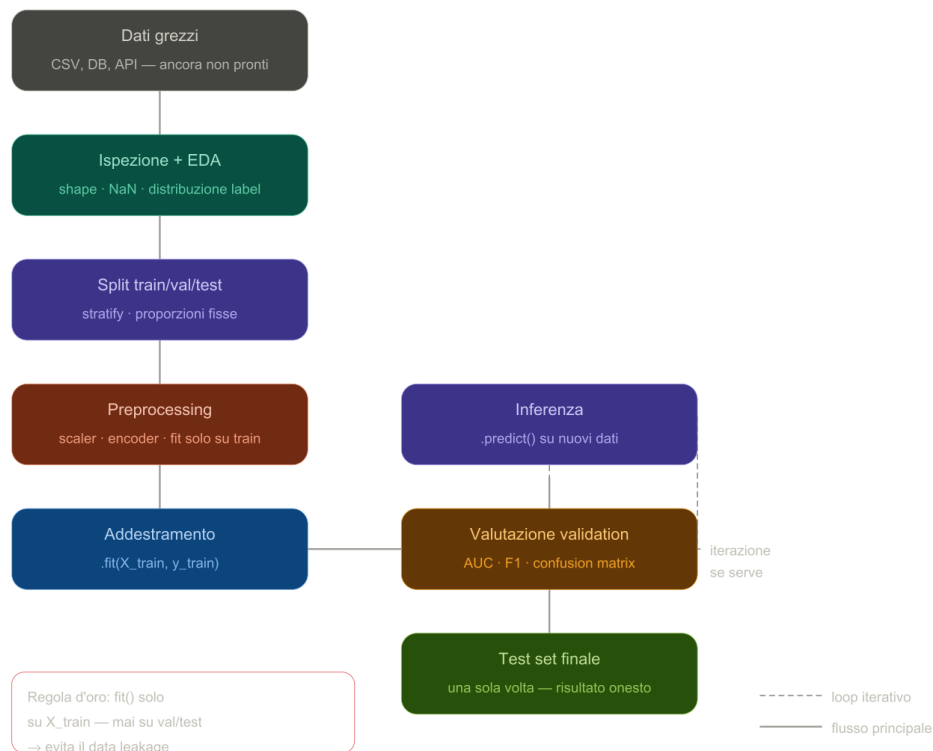


Figura 4.4 — Pipeline ML end-to-end. Le fasi a sinistra sono sequenziali; la colonna destra mostra il ciclo iterativo di valutazione e il test finale. Clicca su ogni blocco per approfondire.

Fase	Cosa fai	Strumento	Dettagli
Carica e ispeziona	<code>.shape</code> , <code>.dtypes</code> , <code>.head()</code> , <code>.isnull()</code>	<code>pandas</code>	Analisi esplorativa
Split	<code>train_test_split</code> con <code>stratify</code>	<code>sklearn</code>	Preprocessing
Preprocessing	<code>ColumnTransformer</code> + <code>StandardScaler</code> + <code>OneHotEncoder</code>	<code>sklearn</code>	Addestramento
Addestramento	<code>.fit(X_train, y_train)</code>	<code>sklearn</code>	Validazione
Validazione	<code>classification_report</code> , <code>roc_auc_score</code>	<code>sklearn</code>	Iterazione
Iterazione	Confronta più modelli sul validation set	<code>sklearn</code>	Test finale
Test finale	Una volta sola, con il modello scelto	<code>sklearn</code>	Interpretazione
Interpretazione	<code>feature_importances_</code>	<code>sklearn</code>	Inferenza
Inferenza	<code>.predict()</code> su nuovi dati	<code>sklearn</code>	

Esercizio — Estendi il progetto

Ora che hai il codice funzionante, ecco tre sfide progressive:

Livello base: aggiungi una nuova feature derivata al dataset, per esempio `costo_per_gb = costo_mensile / (gb_usati_mese + 1)`. Questa feature migliora l'AUC? Di quanto?

Livello medio: il Random Forest ha un iperparametro `max_depth`. Prova i valori `[3, 5, 8, 12, None]` e traccia l'AUC sul validation set per ciascuno. Qual è il valore ottimale? Cosa succede con `max_depth=None` (nessun limite)?

Livello avanzato: il dataset è sbilanciato (~27% churn). Prova a bilanciarlo con **oversampling** usando `imbalanced-learn`:

```
# pip install imbalanced-learn
from imblearn.over_sampling import SMOTE
```

```
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train_proc, y_train)

print(f"Prima di SMOTE: {y_train.value_counts().to_dict()}")
print(f"Dopo SMOTE:      {pd.Series(y_train_res).value_counts().to_dict()}")
# Ora le classi sono bilanciate – riaddestra il modello e confronta le metriche
```

Il prossimo capitolo entra nel dettaglio del componente fondamentale del deep learning: il neurone artificiale, il Multi-Layer Perceptron e la backpropagation passo per passo.

Capitolo 5

Il Neurone Artificiale e le Reti Neurali

"Una rete neurale non capisce il mondo come lo capisce un essere umano. Ma può imparare a imitarne l'output con una precisione sorprendente, e questo, nella pratica, è spesso tutto ciò che serve."

5.1 Dal neurone biologico a quello artificiale

Il cervello umano contiene circa 86 miliardi di neuroni, ognuno connesso ad altri attraverso sinapsi. Un neurone riceve segnali elettrici dai neuroni vicini, li integra, e se il segnale supera una certa soglia, "si attiva" e trasmette a sua volta un impulso.

Il neurone artificiale è un'astrazione matematica di questo meccanismo, drasticamente semplificata. Non simula la biochimica, non replica la complessità del cervello — cattura solo l'idea essenziale: aggrega segnali ponderati e decide se "attivarsi".

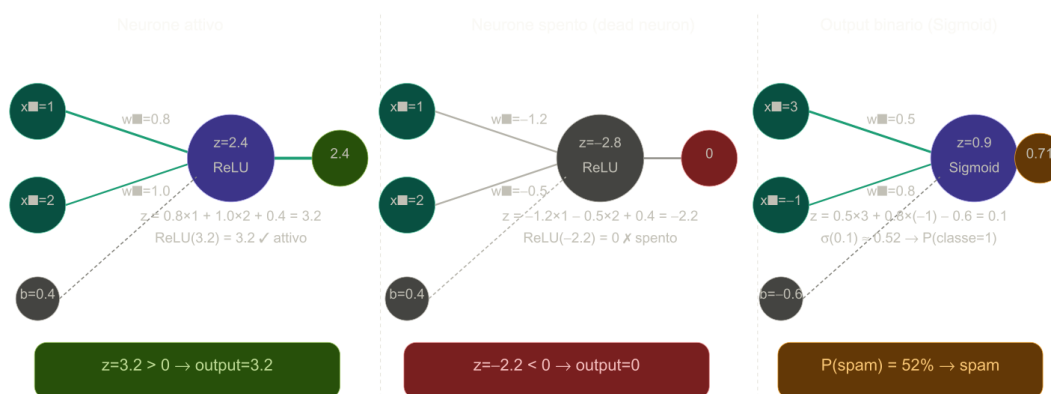


Figura 5.1 — Tre esempi di neurone artificiale: attivo (ReLU), spento (dead neuron), classificazione binaria (Sigmoid)

Figura 5.1 — Tre esempi di neurone artificiale: attivo (ReLU), spento (dead neuron), classificazione binaria (Sigmoid)

La formula è questa:

$$\begin{aligned} \text{output} &= f(w_1x_1 + w_2x_2 + \dots + w_nx_n + b) \\ &= f(w \cdot x + b) \end{aligned}$$

Dove:

- $x_1 \dots x_n$ sono gli **input** (le feature del campione)
- $w_1 \dots w_n$ sono i **pesi** (quanto ogni input conta per questo neurone)
- b è il **bias** (una costante che sposta la soglia di attivazione)
- f è la **funzione di attivazione** (introduce la non-linearità)

```
import numpy as np
```

```
def neurone(x, w, b, funzione_attivazione):
    """Un singolo neurone artificiale."""
    z = np.dot(w, x) + b    # somma pesata + bias
    return funzione_attivazione(z)

# Esempio: neurone che valuta se un'email è spam
# Input: [lunghezza_email, n_link, n_maiuscole]
x = np.array([500.0, 8.0, 15.0])
w = np.array([0.001, 0.3, 0.05]) # il n. di link conta di più
b = -2.0

relu = lambda z: max(0, z)
sigmoid = lambda z: 1 / (1 + np.exp(-z))

z = np.dot(w, x) + b
print(f"Somma pesata z = {z:.3f}")
print(f"Output sigmoid = {sigmoid(z):.3f}") # probabilità di spam
```

Il ruolo di ogni componente

I **pesi** sono ciò che il modello impara. All'inizio sono numeri casuali piccoli. Dopo migliaia di iterazioni di gradient descent, si sono aggiustati per dare più importanza alle feature informative e meno a quelle irrilevanti.

Il **bias** permette al neurone di attivarsi anche quando tutti gli input sono zero. Senza bias, un neurone con input nulli produrrebbe sempre output nullo, limitando fortemente la capacità espressiva del modello.

La **funzione di attivazione** è la parte non-lineare. Senza di essa, qualsiasi rete di N layer sarebbe equivalente a un singolo layer lineare — moltiplicare tante matrici insieme produce ancora una matrice. La non-linearità permette di approssimare funzioni arbitrariamente complesse.

5.2 Le funzioni di attivazione

Abbiamo già visto queste funzioni nel Cap. 3. Ora approfondiamo quando usarle e perché alcune sono preferite ad altre in certi contesti.

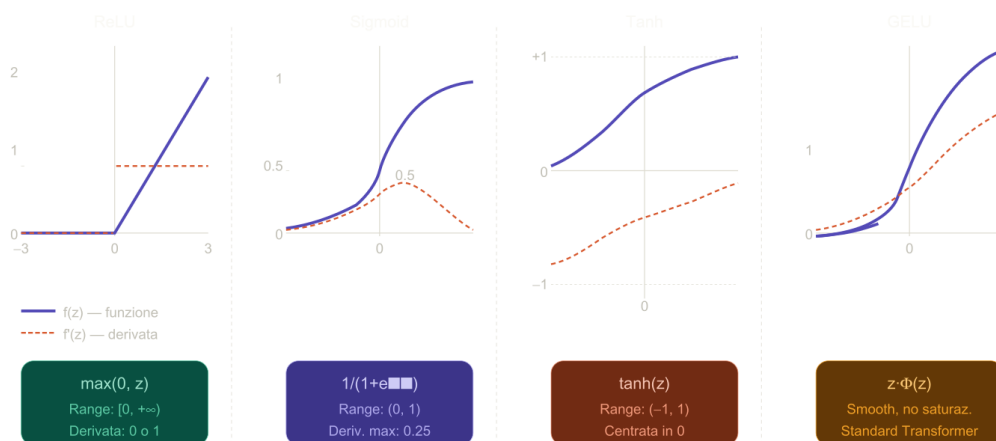


Figura 5.2 — Funzioni di attivazione (continua) e loro derivate (tratteggiata): ReLU, Sigmoid, Tanh, GELU

Figura 5.2 — Funzioni di attivazione (continua) e loro derivate (tratteggiata): ReLU, Sigmoid, Tanh, GELU. confronto funzioni di attivazione interattivo. La linea continua è $f(z)$, quella tratteggiata è la derivata $f'(z)$ — che

determina quanto "segnale di errore" passa durante la backpropagation. Osserva come Sigmoid abbia una derivata massima di soli 0.25, mentre ReLU arriva a 1.0.

ReLU — la scelta di default per i layer nascosti

```
ReLU(z) = max(0, z)
```

Semplice, efficiente, e funziona bene nella stragrande maggioranza dei casi per i layer nascosti. Il problema del **dying ReLU**: se i pesi si aggiustano in modo che un neurone produce sempre $z < 0$, il suo gradiente è sempre zero e smette di imparare. Soluzione: Leaky ReLU o GELU.

```
import numpy as np

def relu(z):      return np.maximum(0, z)
def leaky_relu(z, alpha=0.01): return np.where(z > 0, z, alpha * z)
def gelu(z):      return z * 0.5 * (1 + np.tanh(np.sqrt(2/np.pi) * (z + 0.044715 * z**3)))
def sigmoid(z):   return 1 / (1 + np.exp(-z))
def tanh(z):      return np.tanh(z)

z = np.array([-3, -1, 0, 1, 3])
print(f"z:      {z}")
print(f"ReLU:    {relu(z)}")
print(f"Leaky ReLU: {leaky_relu(z).round(2)}")
print(f"GELU:     {gelu(z).round(3)}")
print(f"Sigmoid:  {sigmoid(z).round(3)}")
```

Quando usare quale

| Funzione | Usala per... | Perché | |-----|-----|-----| | ReLU | Layer nascosti di CNN e MLP | Semplice, veloce, nessuna saturazione per $z > 0$ | | GELU | Layer nascosti nei Transformer | Smooth, performante, differenziabile ovunque | | SiLU/Swish | Layer nascosti in LLaMA, modelli moderni | Smooth, self-gated, leggermente migliore di GELU | | Sigmoid | Output di classificazione binaria | Produce un valore in (0,1) interpretabile come probabilità | | Softmax | Output di classificazione multiclasse | Produce una distribuzione di probabilità che somma a 1 | | Tanh | Layer nascosti di RNN | Centrata in 0, utile per sequenze |

Regola pratica: per i layer nascosti di qualsiasi rete, inizia con ReLU. Se usi un'architettura Transformer, usa GELU. Cambia funzione di attivazione solo se hai una ragione specifica.

5.3 Il Perceptron: il neurone che impara

Il perceptron è il modello più semplice che dimostra il concetto di apprendimento automatico. Un singolo neurone con soglia di attivazione a gradino: se la somma pesata supera 0, output = 1, altrimenti output = 0.

```
import numpy as np

class Perceptron:
    """Il modello più semplice di neurone che impara."""

    def __init__(self, n_feature, lr=0.1):
        # Inizializzazione dei pesi a zero (o piccoli valori casuali)
```

```

self.w = np.zeros(n_feature)
self.b = 0.0
self.lr = lr    # learning rate

def predict(self, x):
    z = np.dot(self.w, x) + self.b
    return 1 if z >= 0 else 0    # funzione a gradino

def fit(self, X, y, epochs=10):
    """Regola di aggiornamento del perceptron:
    se predizione corretta → nessuna modifica
    se sbagliato           → aggiusta pesi nella direzione giusta
    """
    for epoch in range(epochs):
        errori = 0
        for xi, yi in zip(X, y):
            y_pred = self.predict(xi)
            errore = yi - y_pred    # 0, +1, o -1

            # Aggiorna solo se sbagliato
            self.w += self.lr * errore * xi
            self.b += self.lr * errore
            errori += int(errore != 0)

        print(f"Epoch {epoch+1}: {errori} errori")
        if errori == 0:
            print("Convergenza raggiunta!")
            break

# Test: problema OR (linearmente separabile)
X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([0, 1, 1, 1])    # OR: 1 se almeno un input è 1

p = Perceptron(n_feature=2, lr=0.1)
p.fit(X, y, epochs=20)

print("\nPrevisioni finali:")
for xi, yi in zip(X, y):
    print(f"    {xi} → predetto: {p.predict(xi)}, vero: {yi}")

```

Il teorema di convergenza del perceptron garantisce che se i dati sono **linearmente separabili** (cioè separabili da una linea retta, o in N dimensioni da un iperpiano), il perceptron troverà la soluzione. Il famoso limite del perceptron, dimostrato da Minsky e Papert nel 1969: non può risolvere XOR, perché XOR non è linearmente separabile. Soluzione: più strati.

5.4 Il Multi-Layer Perceptron (MLP)

Un MLP, detto anche rete neurale "fully connected" o "densa", organizza i neuroni in strati sequenziali:

- **Layer di input:** riceve le feature del campione. Non ha pesi: è solo un passthrough.
- **Layer nascosti:** trasformano progressivamente le feature in rappresentazioni sempre più astratte.
- **Layer di output:** produce la predizione finale.

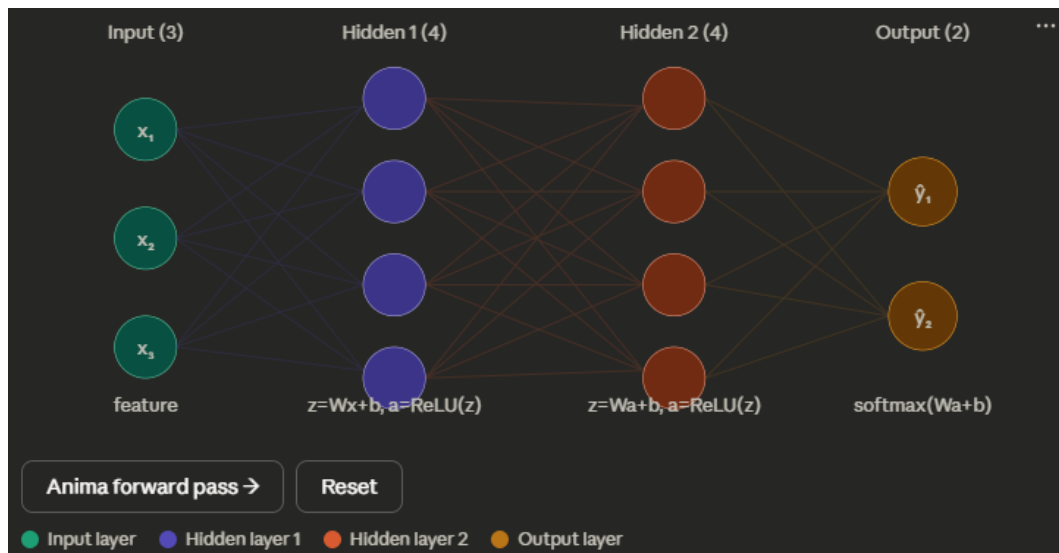


Figura 5.3 — architettura MLP animata 3→4→4→2

Forward pass passo per passo

```
import numpy as np

# MLP da zero - senza librerie - per capire cosa succede dentro
np.random.seed(42)

# Architettura: 3 input → 4 neuroni nascosti → 2 output
# (es: 3 feature → classificatore a 2 classi)

# Inizializzazione dei pesi con Xavier/Glorot
# Scala i pesi in base alla dimensione dei layer per evitare
# che le attivazioni esplodano o svaniscano nei primi step
def inizializza_pesi(n_in, n_out):
    limite = np.sqrt(6 / (n_in + n_out))
    W = np.random.uniform(-limite, limite, (n_out, n_in))
    b = np.zeros(n_out)
    return W, b

W1, b1 = inizializza_pesi(3, 4)    # layer 1: 3→4
W2, b2 = inizializza_pesi(4, 2)    # layer 2: 4→2

def relu(z):    return np.maximum(0, z)
def softmax(z):
    e = np.exp(z - z.max())    # stabilità numerica
    return e / e.sum()

# Un campione di input
x = np.array([0.5, -1.2, 0.8])

# --- Forward pass ---
# Layer 1
z1 = W1 @ x + b1                # somma pesata: shape (4,)
a1 = relu(z1)                   # attivazione ReLU: shape (4,)

# Layer 2
z2 = W2 @ a1 + b2                # somma pesata: shape (2,)
a2 = softmax(z2)                 # attivazione softmax: shape (2,)
```

```

print(f"Input x:          {x}")
print(f"z1 (pre-ReLU):    {z1.round(3)}")
print(f"a1 (post-ReLU):    {a1.round(3)}")
print(f"z2 (pre-softmax): {z2.round(3)}")
print(f"a2 (probabilità): {a2.round(3)} → somma: {a2.sum():.1f}")
print(f"Classe predetta: {a2.argmax()}")

```

Forward pass in PyTorch

```

import torch
import torch.nn as nn

# Lo stesso MLP con PyTorch – molto più compatto
class MLP(nn.Module):
    def __init__(self, n_input, n_hidden, n_output):
        super().__init__()
        # nn.Sequential concatena i layer in ordine
        self.rete = nn.Sequential(
            nn.Linear(n_input, n_hidden), # W1, b1
            nn.ReLU(),
            nn.Linear(n_hidden, n_output), # W2, b2
            # NB: NON mettiamo Softmax qui –
            # nn.CrossEntropyLoss lo applica internamente
        )

    def forward(self, x):
        return self.rete(x)

modello = MLP(n_input=3, n_hidden=4, n_output=2)

# Vediamo l'architettura
print(modello)
print(f"\nParametri totali: {sum(p.numel() for p in modello.parameters())}")
# Layer 1: 3×4 + 4 = 16 parametri
# Layer 2: 4×2 + 2 = 10 parametri
# Totale: 26 parametri

x = torch.tensor([[0.5, -1.2, 0.8]]) # batch di 1 campione
output = modello(x)
print(f"\nOutput (logit): {output.detach().numpy().round(3)}")

```

5.5 La Backpropagation — passo per passo

La backpropagation è l'algoritmo che permette alla rete di imparare. Sappiamo già (Cap. 3) che usa la chain rule per calcolare i gradienti. Ora vediamo cosa succede concretamente in un MLP semplice.

Il problema che risolve

Dopo il forward pass abbiamo una loss L . Vogliamo sapere: per ogni peso nella rete, se lo aumentassi di un po', la loss aumenterebbe o diminuirebbe? Di quanto?

La risposta è il gradiente: $\partial L / \partial W$ per ogni matrice di pesi W .

Backprop a mano su una rete a 2 layer

```
import numpy as np

# Rete minimalista: 2 input → 2 neuroni nascosti → 1 output
# Loss: MSE (per semplicità, anche se è regressione)

np.random.seed(0)

# --- Inizializzazione ---
W1 = np.array([[0.1, 0.2], [0.3, 0.4]]) # shape (2, 2)
b1 = np.array([0.0, 0.0])
W2 = np.array([[0.5, 0.6]]) # shape (1, 2)
b2 = np.array([0.0])
lr = 0.1

def sigmoid(z): return 1 / (1 + np.exp(-z))
def sigmoid_deriv(a): return a * (1 - a) # derivata di sigmoid

# Campione singolo
x = np.array([1.0, 2.0])
y_vero = np.array([1.0])

print("=== FORWARD PASS ===")
# Layer 1
z1 = W1 @ x + b1 # shape (2,)
a1 = sigmoid(z1) # shape (2,)
print(f"z1 = {z1.round(4)}")
print(f"a1 = {a1.round(4)}")

# Layer 2
z2 = W2 @ a1 + b2 # shape (1,)
a2 = sigmoid(z2) # shape (1,) - output finale
print(f"z2 = {z2.round(4)}")
print(f"a2 = {a2.round(4)} ← predizione")

# Loss MSE
loss = 0.5 * (a2 - y_vero)**2
print(f"\nLoss = {loss[0]:.6f}")

print("\n=== BACKWARD PASS (chain rule) ===")
# Gradiente della loss rispetto all'output
dL_da2 = a2 - y_vero #  $\partial L / \partial a_2 = (a_2 - y)$ 
print(f"dL/da2 = {dL_da2.round(4)}")

# Gradiente attraverso sigmoid del layer 2
da2_dz2 = sigmoid_deriv(a2) #  $\partial a_2 / \partial z_2 = a_2(1-a_2)$ 
dL_dz2 = dL_da2 * da2_dz2 # chain rule
print(f"dL/dz2 = {dL_dz2.round(4)}")

# Gradienti dei pesi e bias del layer 2
dL_dW2 = dL_dz2[:, np.newaxis] * a1 #  $\partial L / \partial W_2 = dL/dz_2 \cdot a_1^T$ 
dL_db2 = dL_dz2
print(f"dL/dW2 = {dL_dW2.round(4)}")

# Gradiente che fluisce verso il layer 1
dL_da1 = W2.T @ dL_dz2 #  $\partial L / \partial a_1 = W_2^T \cdot dL/dz_2$ 
```

```

da1_dz1 = sigmoid_deriv(a1)
dL_dz1 = dL_da1 * da1_dz1

# Gradienti dei pesi e bias del layer 1
dL_dW1 = dL_dz1[:, np.newaxis] * x      #  $\partial L / \partial W1 = dL / dz1 \cdot x^T$ 
dL_db1 = dL_dz1

print("\n=== AGGIORNAMENTO PESI ===")
W2_new = W2 - lr * dL_dW2
b2_new = b2 - lr * dL_db2
W1_new = W1 - lr * dL_dW1
b1_new = b1 - lr * dL_db1

# Verifica: la loss dovrebbe scendere
z1n = W1_new @ x + b1_new
a1n = sigmoid(z1n)
z2n = W2_new @ a1n + b2_new
a2n = sigmoid(z2n)
loss_new = 0.5 * (a2n - y_vero)**2

print(f"Loss prima: {loss[0]:.6f}")
print(f"Loss dopo: {loss_new[0]:.6f}")
print(f"Migliorata: {'Sì' if loss_new[0] < loss[0] else 'No'}")

```

Nota pratica: in PyTorch non scrivi mai questo codice. `loss.backward()` fa tutto automaticamente. Ma capire come funziona ti aiuta a diagnosticare problemi come gradienti che esplodono o svaniscono, e a capire perché certe scelte architetturali funzionano meglio di altre.

5.6 Il problema del vanishing gradient

Nelle reti molto profonde, durante la backpropagation i gradienti vengono moltiplicati tante volte per numeri piccoli (le derivate delle funzioni di attivazione). Dopo molti layer, il gradiente diventa così piccolo che i pesi dei primi layer smettono praticamente di aggiornarsi.

Con Sigmoid o Tanh, la derivata massima è 0.25 (Sigmoid) o 1.0 (Tanh) ma solo vicino allo zero. Nelle zone di saturazione (z molto grande o molto piccolo), la derivata si avvicina a zero. In una rete da 10 layer con sigmoid, il gradiente che arriva al primo layer può essere dell'ordine di $0.25^{10} \approx 0.000001$.

```

import numpy as np
import matplotlib.pyplot as plt

def sigmoid(z):      return 1 / (1 + np.exp(-z))
def sigmoid_deriv(z): return sigmoid(z) * (1 - sigmoid(z))
def relu_deriv(z):    return (z > 0).astype(float)

# Simula il prodotto dei gradienti attraverso N layer
n_layers = 20
z = np.random.randn(1000)    # attivazioni casuali

grad_sigmoid = np.ones(1000)
grad_relu     = np.ones(1000)

for _ in range(n_layers):

```



```

grad_sigmoid *= sigmoid_deriv(z)
grad_relu    *= relu_deriv(z)
z = np.random.randn(1000)    # nuovo layer

print(f"Gradiente medio dopo {n_layers} layer:")
print(f"  Con Sigmoid: {np.mean(np.abs(grad_sigmoid)):.2e}")
print(f"  Con ReLU:    {np.mean(np.abs(grad_relu)):.2e}")
# Sigmoid: ordine  $10^{-8}$  o peggio → vanishing gradient
# ReLU: ordine  $10^{-1}$  o  $10^0$  → gradiente preservato

```

Questo spiega perché ReLU ha rimpiazzato Sigmoid nei layer nascosti delle reti profonde. Non si satura per $z > 0$, quindi il gradiente può fluire liberamente verso i layer iniziali.

5.7 Un MLP completo su dati reali

Mettiamo insieme tutto in un esempio che unifica ciò che abbiamo visto nei capitoli precedenti.

```

import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# --- Dati ---
X_np, y_np = make_classification(
    n_samples=2000, n_features=10, n_informative=6,
    n_redundant=2, random_state=42
)

X_train, X_val, y_train, y_val = train_test_split(
    X_np, y_np, test_size=0.2, random_state=42, stratify=y_np
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)    # fit SOLO sul train
X_val    = scaler.transform(X_val)

# Converti in tensor PyTorch
X_tr = torch.FloatTensor(X_train)
y_tr = torch.LongTensor(y_train)          # LongTensor per CrossEntropyLoss
X_vl = torch.FloatTensor(X_val)
y_vl = torch.LongTensor(y_val)

train_loader = DataLoader(TensorDataset(X_tr, y_tr), batch_size=32, shuffle=True)

# --- Modello ---
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(10, 64),
            nn.ReLU(),
            nn.Dropout(0.3),                # regolarizzazione
            nn.Linear(64, 32),

```

```

        nn.ReLU(),
        nn.Dropout(0.3),
        nn.Linear(32, 2)          # 2 classi di output
    )

    def forward(self, x):
        return self.net(x)

modello = MLP()
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(modello.parameters(), lr=1e-3)

# --- Training loop ---
train_losses, val_losses, val_accs = [], [], []

for epoch in range(50):
    # Training
    modello.train()
    ep_loss = 0
    for xb, yb in train_loader:
        pred = modello(xb)
        loss = loss_fn(pred, yb)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        ep_loss += loss.item()
    train_losses.append(ep_loss / len(train_loader))

    # Validation
    modello.eval()
    with torch.no_grad():
        val_pred = modello(X_vl)
        val_loss = loss_fn(val_pred, y_vl).item()
        val_acc = (val_pred.argmax(dim=1) == y_vl).float().mean().item()
    val_losses.append(val_loss)
    val_accs.append(val_acc)

    if (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1:3d} | train loss: {train_losses[-1]:.4f} | "
              f"val loss: {val_loss:.4f} | val acc: {val_acc*100:.1f}%")

# --- Visualizza le curve ---
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ax1.plot(train_losses, label='Training'); ax1.plot(val_losses, label='Validation')
ax1.set_title('Loss'); ax1.legend(); ax1.grid(True, alpha=0.3)
ax2.plot([a*100 for a in val_accs], color='green')
ax2.set_title('Accuracy sul validation set (%)'); ax2.grid(True, alpha=0.3)
plt.tight_layout(); plt.savefig('mlp_training.png', dpi=150); plt.show()

```

5.8 Inizializzazione dei pesi: perché conta

Inizializzare i pesi a zero non funziona: tutti i neuroni di ogni layer producono lo stesso output, ricevono lo stesso gradiente, e rimangono identici per sempre. Questo si chiama **problema di simmetria**.

La soluzione è inizializzare i pesi con valori casuali piccoli, ma con una scala calibrata alla dimensione dei layer:

```
# Xavier/Glorot - ottimale per Sigmoid e Tanh
def xavier(n_in, n_out):
    limite = np.sqrt(6 / (n_in + n_out))
    return np.random.uniform(-limite, limite, (n_out, n_in))

# He/Kaiming - ottimale per ReLU
def he(n_in, n_out):
    std = np.sqrt(2 / n_in)
    return np.random.normal(0, std, (n_out, n_in))

# In PyTorch viene applicata automaticamente a nn.Linear:
# - Xavier per layer con Sigmoid/Tanh (default)
# - He per layer con ReLU (applicato automaticamente da alcune architetture)
```

Riepilogo del Capitolo 5

Hai imparato:

- Il **neurone artificiale**: somma pesata degli input + bias, passata attraverso una funzione di attivazione.
- Le **funzioni di attivazione**: ReLU per i layer nascosti (default), GELU nei Transformer, Sigmoid e Softmax per gli output.
- Il **Perceptron**: il modello più semplice che impara. Converte se i dati sono linearmente separabili.
- Il **MLP**: più layer di neuroni in sequenza, ognuno con pesi, bias e attivazione. La profondità permette di approssimare funzioni non lineari arbitrarie.
- La **backpropagation**: la chain rule applicata in sequenza dal layer di output verso quello di input, per calcolare i gradienti di ogni peso.
- Il **vanishing gradient**: il motivo per cui Sigmoid è stata rimpiazzata da ReLU nei layer nascosti profondi.
- L'**inizializzazione dei pesi**: Xavier per Sigmoid/Tanh, He per ReLU. Zero è sempre sbagliato.

Esercizio 1 — Neurone a mano

Obiettivo: consolidare la comprensione del forward pass.

```
import numpy as np

# Immagina di costruire un sistema di scoring per candidati a un lavoro.
# Feature: [anni_esperienza, voto_laurea (0-110), n_lingue_parlate]
# Il selezionatore vuole un punteggio tra 0 e 1.

# Pesi che rappresentano l'importanza di ogni feature:
w = np.array([0.4, 0.01, 0.2])
b = -3.5
```

```

sigmoid = lambda z: 1 / (1 + np.exp(-z))

candidati = np.array([
    [2.0, 95.0, 1],      # 2 anni exp, 95/110, 1 lingua
    [8.0, 70.0, 3],      # 8 anni exp, 70/110, 3 lingue
    [0.5, 110.0, 2],     # neolaureato brillante, 2 lingue
    [5.0, 85.0, 2],      # profilo equilibrato
])

# Calcola il punteggio per ogni candidato
for i, c in enumerate(candidati):
    z = np.dot(w, c) + b
    score = sigmoid(z)
    print(f"Candidato {i+1}: z={z:.2f}, score={score:.3f}")

# Domande:
# 1. Chi ottiene il punteggio più alto?
# 2. Il modello sembra premiare più l'esperienza o il voto?
# 3. Modifica i pesi per far emergere il neolaureato brillante.
# 4. Come cambieresti b per rendere il modello meno "severo"
#     (cioè dare punteggi più alti in media)?

```

Esercizio 2 — MLP da zero con backpropagation manuale

Obiettivo: implementare la backpropagation senza PyTorch per capirla a fondo.

```

import numpy as np

# Rete 2→3→1 addestrata su XOR
# XOR non è linearmente separabile → il Perceptron non converge
# Un MLP a 2 layer sì.

X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([[0],[1],[1],[0]], dtype=float) # XOR

np.random.seed(1)
W1 = np.random.randn(2, 3) * 0.5 # 2 input → 3 nascosti
b1 = np.zeros((1, 3))
W2 = np.random.randn(3, 1) * 0.5 # 3 nascosti → 1 output
b2 = np.zeros((1, 1))
lr = 0.5

sigmoid = lambda z: 1 / (1 + np.exp(-z))
sig_d = lambda a: a * (1 - a)

losses = []
for step in range(5000):
    # Forward
    a1 = sigmoid(X @ W1 + b1) # (4, 3)
    a2 = sigmoid(a1 @ W2 + b2) # (4, 1)
    loss = np.mean((a2 - y)**2)
    losses.append(loss)

    # Backward
    dL_da2 = 2 * (a2 - y) / len(X)

```

```

dL_dz2 = dL_da2 * sig_d(a2)
dL_dW2 = a1.T @ dL_dz2
dL_db2 = dL_dz2.sum(axis=0, keepdims=True)
dL_da1 = dL_dz2 @ W2.T
dL_dz1 = dL_da1 * sig_d(a1)
dL_dW1 = X.T @ dL_dz1
dL_db1 = dL_dz1.sum(axis=0, keepdims=True)

# Aggiornamento
W2 -= lr * dL_dW2; b2 -= lr * dL_db2
W1 -= lr * dL_dW1; b1 -= lr * dL_db1

print(f"Loss finale: {loss:.6f}")
print(f"\nPredizioni XOR (dovrebbero essere vicine a 0,1,1,0):")
for xi, yi, pi in zip(X, y, a2):
    print(f"  {xi} → vero: {yi[0]:.0f}, predetto: {pi[0]:.3f}")

# Domande:
# 1. Prova lr=0.1 e lr=2.0. Cosa succede alla convergenza?
# 2. Prova a risolvere XOR con solo 2 neuroni nascosti. Ci riesce?
# 3. Aggiungi un terzo layer nascosto. Il modello converge più veloce?

```

Prossimo capitolo: Machine Learning Classico — regressione, alberi, Random Forest e Gradient Boosting. Vediamo quando e perché questi modelli battono le reti neurali sui dati tabellari.

Capitolo 6

Machine Learning Classico

"Spesso la soluzione migliore non è quella più complessa. Su dati tabellari, un buon XGBoost batte una rete neurale profonda nel 90% dei casi, con un decimo del tempo di training."

Perché studiare i modelli classici

Le reti neurali fanno notizia. Ma nella pratica professionale quotidiana, la maggior parte dei problemi su dati strutturati (CSV, database, Excel) viene risolta meglio e più velocemente con i modelli classici di ML.

I motivi sono concreti:

- Meno dati necessari per convergere a buone performance
- Training molto più veloce (secondi invece di ore)
- Più interpretabili — puoi spiegare al cliente perché il modello ha fatto quella predizione
- Meno iperparametri da ottimizzare
- Non richiedono GPU

Questo capitolo copre i modelli fondamentali che ogni sviluppatore ML deve conoscere, con il codice e le intuizioni per usarli bene.

6.1 Regressione Lineare: la base di tutto

La regressione lineare è il modello più semplice che esista: assume che la relazione tra feature e target sia una linea retta (o in N dimensioni, un iperpiano).

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

Dove w_0 è il termine costante (bias/intercetta) e $w_1 \dots w_n$ sono i coefficienti che misurano l'impatto di ogni feature sull'output.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Dataset sintetico: ore di studio → voto all'esame
np.random.seed(42)
ore_studio = np.random.uniform(1, 10, 100).reshape(-1, 1)
voto = 4 * ore_studio.ravel() + 20 + np.random.randn(100) * 5

# Addestramento
modello = LinearRegression()
modello.fit(ore_studio, voto)
```

```
# I parametri appresi
print(f"Coefficiente (pendenza): {modello.coef_[0]:.2f}")
# → ogni ora in più di studio vale circa +4 punti
print(f"Intercetta: {modello.intercept_[0]:.2f}")
# → uno studente che studia 0 ore ottiene circa 20 punti

# Predizione e valutazione
voto_pred = modello.predict(ore_studio)
mse = mean_squared_error(voto, voto_pred)
r2 = r2_score(voto, voto_pred)

print(f"\nMSE: {mse:.2f}")
print(f"RMSE: {np.sqrt(mse):.2f} ← in media sbaglio di ±{np.sqrt(mse):.1f} punti")
print(f"R²: {r2:.3f} ← il modello spiega il {r2*100:.1f}% della varianza")
```

Cos'è l' R^2 ? L' R^2 (R quadrato o "coefficiente di determinazione") misura quanto bene il modello spiega la variabilità del target. 1.0 = perfetto. 0.0 = non meglio di predire sempre la media. Negativo = peggio della media (modello inutile). Un R^2 di 0.85 significa che il modello cattura l'85% dei pattern nel target.

Regressione Ridge e Lasso: aggiungere regolarizzazione

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet

# Ridge: aggiunge penalità L2 → pesi piccoli, mai zero
ridge = Ridge(alpha=1.0) # alpha = forza della regolarizzazione
ridge.fit(X_train, y_train)

# Lasso: aggiunge penalità L1 → molti pesi diventano esattamente 0
# Equivale a selezione automatica delle feature
lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
feature_selezionate = np.sum(lasso.coef_ != 0)
print(f"Feature selezionate da Lasso: {feature_selezionate} / {X_train.shape[1]}")

# ElasticNet: combina L1 + L2 — buon compromesso
elastic = ElasticNet(alpha=0.1, l1_ratio=0.5)
elastic.fit(X_train, y_train)
```

Quando usare quale:

- Ridge: quando pensi che tutte le feature siano utili, ma vuoi evitare pesi troppo grandi
- Lasso: quando hai molte feature e vuoi che il modello ne selezioni automaticamente solo alcune
- ElasticNet: quando hai molte feature correlate tra loro (Lasso tende a tenerne solo una per gruppo)

6.2 Regressione Logistica: classificazione nonostante il nome

Nonostante il nome, la regressione logistica è un **classificatore**. Modella la probabilità che un campione appartenga a una classe applicando la funzione sigmoid a una combinazione lineare.

```

P(y=1 | x) = sigmoid(w·x + b) = 1 / (1 + e^{-(w·x+b)})

from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

X, y = make_classification(n_samples=1000, n_features=5,
                           n_informative=3, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

# C = inverso della regolarizzazione (C grande = meno regolarizzazione)
clf = LogisticRegression(C=1.0, max_iter=500)
clf.fit(X_tr, y_tr)

print(classification_report(y_te, clf.predict(X_te)))

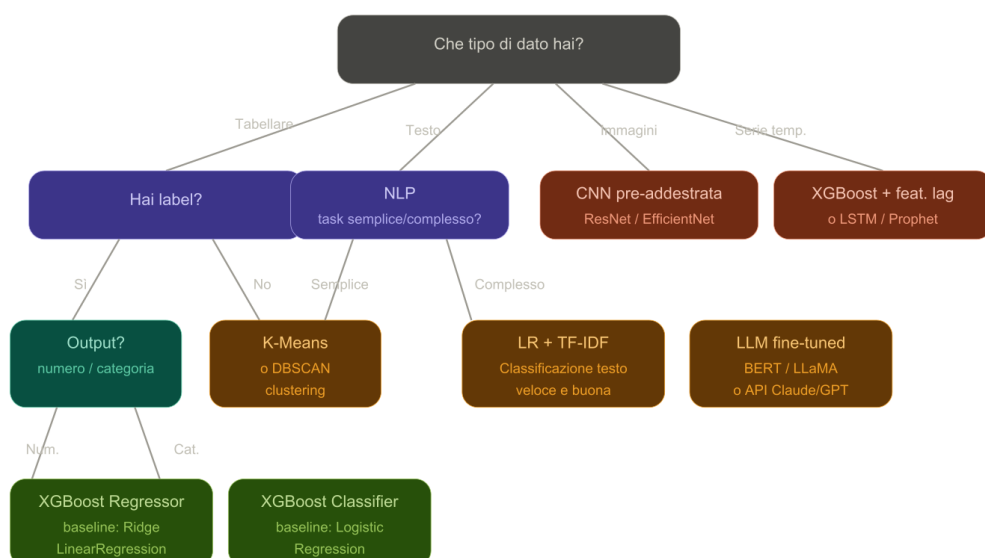
# La regressione logistica è interpretabile: guarda i coefficienti
for feat, coef in enumerate(clf.coef_[0]):
    print(f"Feature {feat}: {coef:+.3f} "
          f"({'aumenta' if coef > 0 else 'riduce'} probabilità classe 1)")

```

Quando preferire la regressione logistica? Quando l'interpretabilità è critica (medicina, banca, legale), quando i dati sono pochi (< 500 campioni), come baseline veloce da confrontare con modelli più complessi.

6.3 Alberi Decisionali: il modello che si spiega da solo

Un albero decisionale costruisce una sequenza di domande binarie sulle feature per arrivare a una predizione. È il modello più interpretabile che esiste: puoi seguire il percorso di ogni predizione passo per passo.



Regola sempre valida: inizia sempre con una baseline semplice (LogisticRegression o LinearRegression) prima di provare modelli complessi

Figura 6.3 — Guida alla scelta del modello in base al tipo di dato e al task

Figura 6.1 — Guida alla scelta del modello (flowchart con percorsi statici).

Come cresce un albero

Ad ogni nodo, l'albero sceglie il **miglior split**: la feature e la soglia che dividono il dataset in due gruppi il più "puri" possibile. La purezza si misura con l'**impurity di Gini** o con il **guadagno di informazione**:

$$\text{Gini}(\text{nodo}) = 1 - \sum p_i^2$$

Dove p_i è la proporzione della classe i nel nodo. Se tutti gli esempi sono della stessa classe: Gini = 0 (puro). Se le classi sono bilanciate: Gini = 0.5 (massima impurità).

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# Dataset Iris: classificare fiori in 3 specie
iris = load_iris()
X, y = iris.data, iris.target

# max_depth limita la profondità → evita overfitting
albero = DecisionTreeClassifier(
    max_depth=3,
    min_samples_leaf=5,  # ogni foglia deve avere almeno 5 esempi
    random_state=42
)
albero.fit(X, y)

# Visualizza l'albero (molto utile per spiegare il modello)
plt.figure(figsize=(16, 6))
plot_tree(albero, feature_names=iris.feature_names,
          class_names=iris.target_names, filled=True, rounded=True)
plt.title("Albero decisionale sul dataset Iris")
plt.tight_layout()
plt.savefig('decision_tree.png', dpi=150)
plt.show()

print(f"\nAccuratezza sul training: {albero.score(X, y)*100:.1f}%")
print(f"Profondità effettiva:      {albero.get_depth()}")
print(f"N. foglie:                  {albero.get_n_leaves()}")
```

Il problema degli alberi singoli

Un albero senza limiti di profondità memorizza perfettamente il training set (ogni foglia contiene un solo campione). Overfitting garantito.

```
# Dimostrazione: albero senza limiti
albero_overfit = DecisionTreeClassifier()  # nessun limite
albero_overfit.fit(X_train, y_train)
print(f"Train accuracy: {albero_overfit.score(X_train, y_train)*100:.1f}%")  # 100%
print(f"Test accuracy:  {albero_overfit.score(X_test, y_test)*100:.1f}%")    # molto peggio

# Albero con regolarizzazione
albero_regol = DecisionTreeClassifier(max_depth=4, min_samples_leaf=10)
albero_regol.fit(X_train, y_train)
print(f"\nTrain accuracy: {albero_regol.score(X_train, y_train)*100:.1f}%")
print(f"Test accuracy:  {albero_regol.score(X_test, y_test)*100:.1f}%")    # più bilanciato
```

La soluzione al problema dell'overfitting degli alberi singoli sono gli **ensemble methods**: invece di addestrare un albero, ne addestriamo centinaia e combiniamo le loro predizioni.

6.4 Random Forest: forza nella diversità

Un Random Forest è una collezione di alberi decisionali, ognuno addestrato su un sottoinsieme casuale dei dati e delle feature. La predizione finale è la media (regressione) o il voto di maggioranza (classificazione) di tutti gli alberi.

La casualità è la chiave: la rende decorrelata tra gli alberi. Alberi diversi fanno errori diversi, e quando si mediagno insieme gli errori si cancellano a vicenda.

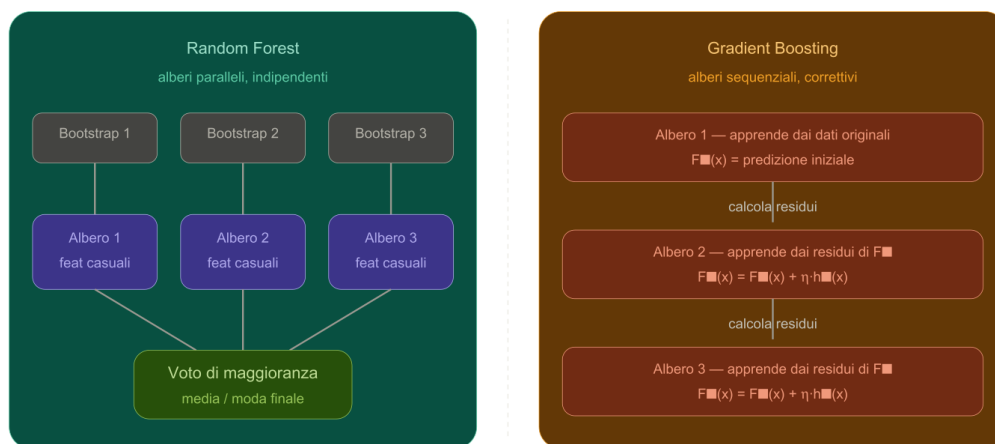


Figura 6.2 — Random Forest vs Gradient Boosting. A sinistra: alberi paralleli e indipendenti su bootstrap casuali, aggregati per voto. A destra: alberi sequenziali dove ciascuno corregge i residui del precedente.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=2000, n_features=20,
                           n_informative=10, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

rf = RandomForestClassifier(
    n_estimators=200,          # numero di alberi — più è meglio, ma con diminishing
    returns                    # returns
    max_depth=None,           # ogni albero cresce liberamente (la foresta si
    regularizza da sola)      # regularizza da sola)
    max_features='sqrt',      # ogni split considera solo √(n_feature) feature casuali
    min_samples_leaf=2,
    n_jobs=-1,                # usa tutti i core disponibili
    random_state=42
)
rf.fit(X_tr, y_tr)

print(f"Accuracy sul test: {rf.score(X_te, y_te)*100:.1f}%")

# Feature importance: quale feature ha contribuito di più?
import pandas as pd
importances = pd.Series(rf.feature_importances_,
```

```

        index=[f'feat_{i}' for i in range(20)])
print("\nTop 5 feature più importanti:")
print(importanze.sort_values(ascending=False).head(5).round(3).to_string())

```

Out-of-Bag (OOB) Error

Un vantaggio pratico del Random Forest: ogni albero viene addestrato su un sottoinsieme casuale (bootstrap) del training set. Gli esempi non inclusi in quel sottoinsieme — chiamati **out-of-bag** — possono essere usati per stimare l'errore di generalizzazione senza bisogno di un validation set separato.

```

rf_oob = RandomForestClassifier(n_estimators=200, oob_score=True,
                               n_jobs=-1, random_state=42)

rf_oob.fit(X_tr, y_tr)
print(f"OOB accuracy: {rf_oob.oob_score_*100:.1f}%")
# Molto vicino all'accuracy sul test - stima gratuita!

```

6.5 Gradient Boosting: costruire sulla debolezza

Il Gradient Boosting ha un approccio radicalmente diverso dal Random Forest. Invece di addestrare alberi in parallelo e indipendenti, li addestra in **sequenza**: ogni nuovo albero corregge gli errori del modello precedente.

```

F0(x) = predizione iniziale (media del target)
F1(x) = F0(x) + η · h1(x)    [h1 apprende i residui di F0]
F2(x) = F1(x) + η · h2(x)    [h2 apprende i residui di F1]
...
Fm(x) = Fm-1(x) + η · hm(x)

```

Dove η è il learning rate e h_m è un albero piccolo (weak learner) addestrato sugli errori residui del modello precedente.

```

from sklearn.ensemble import GradientBoostingClassifier
import xgboost as xgb      # pip install xgboost
import lightgbm as lgb     # pip install lightgbm

# sklearn GradientBoosting - più lento ma ben documentato
gb_sklearn = GradientBoostingClassifier(
    n_estimators=200,
    learning_rate=0.1,      # più basso = più robusto ma più lento
    max_depth=4,           # alberi piccoli - tipicamente 3-6
    subsample=0.8,         # usa solo l'80% dei campioni per ogni albero
    random_state=42
)
gb_sklearn.fit(X_tr, y_tr)

# XGBoost - molto più veloce, regolarizzazione avanzata
xgb_model = xgb.XGBClassifier(
    n_estimators=200,
    learning_rate=0.1,
    max_depth=4,
    subsample=0.8,
    colsample_bytree=0.8,   # usa solo l'80% delle feature per ogni albero
    reg_lambda=1.0,        # regolarizzazione L2
    use_label_encoder=False,

```

```

    eval_metric='logloss',
    random_state=42,
    verbosity=0
)
xgb_model.fit(X_tr, y_tr,
              eval_set=[(X_te, y_te)],
              verbose=False)

# LightGBM - ancora più veloce su dataset grandi
lgb_model = lgb.LGBMClassifier(
    n_estimators=200,
    learning_rate=0.05,
    num_leaves=31,      # LightGBM usa num_leaves invece di max_depth
    random_state=42,
    verbose=-1
)
lgb_model.fit(X_tr, y_tr,
              eval_set=[(X_te, y_te)],
              callbacks=[lgb.early_stopping(20, verbose=False)])

print("Accuracy sul test:")
print(f" GradientBoosting (sklearn): {gb_sklearn.score(X_te, y_te)*100:.1f}%")
print(f" XGBoost: {xgb_model.score(X_te, y_te)*100:.1f}%")
print(f" LightGBM: {lgb_model.score(X_te, y_te)*100:.1f}%")

```

XGBoost: perché domina le competizioni

XGBoost (eXtreme Gradient Boosting) aggiunge al Gradient Boosting classico:

- Regularizzazione L1 e L2 nella funzione obiettivo → meno overfitting
- Pruning degli alberi → struttura più efficiente
- Gestione nativa dei valori mancanti → no preprocessing obbligatorio
- Parallelizzazione efficiente → training veloce anche su CPU

```

# Early stopping con XGBoost: smette quando la val loss non migliora
xgb_final = xgb.XGBClassifier(
    n_estimators=1000,      # potenzialmente tanti alberi
    learning_rate=0.05,     # learning rate basso con molti alberi
    max_depth=4,
    early_stopping_rounds=20, # si ferma se non migliora per 20 round
    eval_metric='auc',
    random_state=42, verbosity=0
)
xgb_final.fit(X_tr, y_tr,
              eval_set=[(X_te, y_te)],
              verbose=False)
print(f"\nMigliori iterazioni: {xgb_final.best_iteration}")
print(f"AUC sul test: {xgb_final.best_score:.4f}")

```

6.6 Support Vector Machine (SVM)

La SVM trova l'iperpiano che massimizza il margine tra le classi — la distanza tra la linea di separazione e i punti più vicini di ogni classe. Questi punti di frontiera si chiamano **support vectors** e sono gli unici che determinano la soluzione.

```

from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# IMPORTANTE: la SVM è sensibile alla scala delle feature — sempre normalizzare
svm_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(
        C=1.0,                # penalità per violazioni del margine (C grande = margine
                              # stretto)
        kernel='rbf',         # kernel radiale — la scelta più comune per dati non
                              # lineari
        gamma='scale',        # larghezza del kernel RBF (auto-calibrata)
        probability=True      # abilita predict_proba (richiede più memoria/tempo)
    ))
])

svm_pipeline.fit(X_tr, y_tr)
print(f"SVM accuracy: {svm_pipeline.score(X_te, y_te)*100:.1f}%")

```

Il kernel trick: separare ciò che non è separabile

Il potere dell'SVM è il **kernel trick**: trasforma implicitamente i dati in uno spazio ad altissima dimensionalità dove diventano linearmente separabili, senza mai calcolare esplicitamente quella trasformazione.

```

# Confronto kernel su un problema non lineare
from sklearn.datasets import make_moons

X_moon, y_moon = make_moons(n_samples=300, noise=0.2, random_state=42)

kernels = {
    'lineare': SVC(kernel='linear', C=1),
    'polinomiale': SVC(kernel='poly', C=1, degree=3),
    'RBF': SVC(kernel='rbf', C=1, gamma='scale'),
}

for nome, svm in kernels.items():
    Pipeline([('s', StandardScaler()), ('m', svm)]).fit(X_moon, y_moon)
    # (qui valuteresti su un test set separato)
    print(f"Kernel {nome:12s}: addestrato")

```

Quando usare SVM? È ottimo su dataset piccoli (<10.000 campioni) con poche feature ma alta dimensionalità (testo, immagini). Tende a scalare male su dataset grandi — preferisci Random Forest o XGBoost in quel caso.

6.7 Clustering K-Means: trovare struttura senza etichette

K-Means è il più semplice algoritmo di clustering (apprendimento non supervisionato): raggruppa i dati in K cluster, dove ogni cluster è definito dal suo **centroide** (la media dei punti che lo compongono).

```

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

```

```
import matplotlib.pyplot as plt

# Dataset: profili di clienti senza etichette
np.random.seed(42)
clienti = np.random.randn(300, 2) * np.array([[2,1],[0.5,2],[1,0.5]])
clienti[:100] += [0, 5] # gruppo 1: in alto
clienti[100:200] += [-4, 0] # gruppo 2: a sinistra
# clienti[200:] rimane vicino a 0,0 - gruppo 3

scaler = StandardScaler()
clienti_norm = scaler.fit_transform(clienti)

# Scegli K con il metodo del gomito (elbow method)
inerzie = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(clienti_norm)
    inerzie.append(km.inertia_) # somma delle distanze quadratiche dai centroidi

plt.figure(figsize=(8, 4))
plt.plot(range(1, 11), inerzie, 'o-', color='steelblue')
plt.xlabel('Numero di cluster K')
plt.ylabel('Inerzia (somma distanze²)')
plt.title('Elbow method - scegli K dove la curva si flette')
plt.grid(True, alpha=0.3)
plt.savefig('elbow.png', dpi=150, bbox_inches='tight')
plt.show()

# Addestra con K ottimale
km_final = KMeans(n_clusters=3, random_state=42, n_init=10)
etichette = km_final.fit_predict(clienti_norm)
print(f"Cluster assegnati: {np.unique(etichette, return_counts=True)}")
```

Limiti di K-Means: assume che i cluster siano sferici e di dimensioni simili. Non funziona bene su cluster allungati, ad anello, o di densità molto diverse. In quel caso considera DBSCAN (gestisce forme arbitrarie e outlier) o clustering gerarchico (non richiede K predefinito).

6.8 Guida alla scelta del modello

La domanda più comune in ML non è "come funziona questo algoritmo?" ma "quale algoritmo uso per questo problema?".

Regola empirica 2026

| Situazione | Modello consigliato | Perché | |-----|-----|-----| | Dati tabellari, < 1.000 campioni | Regressione logistica / Ridge | Semplice, robusto con pochi dati | | Dati tabellari, qualsiasi dimensione | XGBoost o LightGBM | SOTA su tabellare, veloce, gestisce NaN | | Interpretabilità critica | Regressione logistica | Coefficienti leggibili da umani | | Immagini | CNN (Cap. 9) | Architettura progettata per struttura spaziale | | Testo | Transformer/LLM fine-tuned (Cap. 12-13) | SOTA per NLP | | Classificazione testo semplice | LogisticRegression + TF-IDF | Sorprendentemente buono e veloce | | Anomaly detection | Isolation Forest / Autoencoder | Nessuna label richiesta | | Clustering | K-Means (sferici) / DBSCAN (forme libere) | Dipende dalla

Riepilogo del Capitolo 6

Hai imparato:

- **Regressione lineare:** il modello più semplice per output continui. Ridge (L2) e Lasso (L1) per regolarizzare.
 - **Regressione logistica:** classificatore interpretabile che modella probabilità. Punto di riferimento obbligatorio.
 - **Alberi decisionali:** modelli leggibili che partizionano lo spazio con domande binarie. Tendono all'overfitting senza limiti.
 - **Random Forest:** ensemble di alberi casuali, decorrelati e paralleli. Robusto, poca ottimizzazione necessaria.
 - **Gradient Boosting / XGBoost:** alberi sequenziali che correggono i residui. SOTA su dati tabellari.
 - **SVM:** massimizzazione del margine con kernel trick per dati non lineari. Ottimo su dataset piccoli.
 - **K-Means:** clustering non supervisionato con centroidi. Scegli K con l'elbow method.
 - **Guida alla scelta:** XGBoost per quasi tutto il tabellare; Deep Learning per immagini e testo.
-

Esercizio 1 — Confronto sistematico di modelli

Obiettivo: costruire una pipeline di benchmark riutilizzabile.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
import pandas as pd
import numpy as np

# Dataset: tumore al seno (classificazione binaria, 30 feature)
X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

modelli = {
    'Logistic Regression': Pipeline([('s', StandardScaler()),
                                     ('m', LogisticRegression(max_iter=1000))]),
    'Decision Tree': DecisionTreeClassifier(max_depth=5, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, n_jobs=-1,
random_state=42),
    'Gradient Boosting': GradientBoostingClassifier(n_estimators=100,
random_state=42),
```

```

    'SVM': Pipeline([('s', StandardScaler()), ('m',
SVC(probability=True))]),
}

print(f"{'Modello':<25} {'AUC-ROC media':>14} {'Std':>8}")
print("-" * 52)
risultati = {}
for nome, mod in modelli.items():
    scores = cross_val_score(mod, X, y, cv=cv, scoring='roc_auc', n_jobs=-1)
    risultati[nome] = scores.mean()
    print(f"{'nome':<25} {scores.mean():>14.4f} {scores.std():>8.4f}")

vincitore = max(risultati, key=risultati.get)
print(f"\nModello migliore: {vincitore} (AUC={risultati[vincitore]:.4f})")

```

Esercizio 2 — XGBoost con hyperparameter tuning

Obiettivo: ottimizzare sistematicamente gli iperparametri di XGBoost.

```

import xgboost as xgb
from sklearn.model_selection import RandomizedSearchCV
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
import numpy as np

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
                                          stratify=y, random_state=42)

# Griglia di iperparametri da esplorare
param_dist = {
    'n_estimators': [100, 200, 300],
    'max_depth': [3, 4, 5, 6],
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
    'reg_lambda': [0.5, 1.0, 2.0],
}

xgb_base = xgb.XGBClassifier(eval_metric='auc', random_state=42, verbosity=0)

# RandomizedSearchCV: prova 20 combinazioni casuali
# (più efficiente del grid search che prova tutte le combinazioni)
search = RandomizedSearchCV(
    xgb_base, param_dist,
    n_iter=20,          # quante combinazioni provare
    cv=5,
    scoring='roc_auc',
    n_jobs=-1,
    random_state=42,
    verbose=1
)
search.fit(X_tr, y_tr)

print(f"\nMigliori parametri: {search.best_params_}")
print(f"AUC validation (CV): {search.best_score_:.4f}")

```



```
print(f"AUC test: {search.score(X_te, y_te):.4f}")

# Domande:
# 1. Quale iperparametro ha avuto il maggiore impatto sul risultato?
# 2. Aumenta n_iter a 50. L'AUC migliora? Di quanto?
# 3. Confronta con un RandomForest ottimizzato allo stesso modo.
```

Prossimo capitolo: Ottimizzatori e Training Loop — confronto dettagliato tra SGD, Adam e AdamW, con learning rate scheduling e tecniche per accelerare il training di reti neurali profonde.

Capitolo 7

Ottimizzatori e Training Loop

"La scelta dell'ottimizzatore è come scegliere le scarpe per una gara: non determina se sei allenato, ma può fare la differenza tra arrivare e non arrivare."

7.1 Cosa fa un ottimizzatore

Ogni volta che il modello fa un errore, la backpropagation calcola i gradienti — una freccia che punta nella direzione di aumento della loss per ogni peso. L'ottimizzatore decide come usare quei gradienti per aggiornare i pesi.

La regola base è il gradient descent puro:

$$w_{\text{nuovo}} = w_{\text{vecchio}} - \eta \cdot \nabla L(w)$$

Ma questa formula semplice ha un problema: usa lo stesso learning rate η per ogni peso, in ogni momento del training. Gli ottimizzatori più sofisticati adattano il passo in modo intelligente — più veloce dove il terreno è piatto, più cauto dove è ripido.

7.2 SGD con Momentum: dare inerzia alla discesa

Il gradient descent puro "rimbalza" su superfici di loss strette e allungate: ogni step punta in direzioni leggermente diverse, creando oscillazioni invece di convergere dritto verso il minimo.

Il **momentum** aggiunge inerzia: accumula una "velocità" basata sui gradienti passati e si muove nella direzione media. Come una pallina che rotola su una superficie ondulata — non cambia direzione ad ogni piccola buca.

$$\begin{aligned} v_t &= \beta \cdot v_{t-1} - \eta \cdot \nabla L(w) && \text{[aggiorna la velocità]} \\ w_t &= w_{t-1} + v_t && \text{[aggiorna i pesi]} \end{aligned}$$

Il parametro β (tipicamente 0.9) controlla quanta memoria ha il momentum: quanto conta la velocità accumulata rispetto al gradiente corrente.

```
import torch
import torch.nn as nn

# SGD con momentum in PyTorch
optimizer = torch.optim.SGD(
    modello.parameters(),
    lr=0.01,
    momentum=0.9,          #  $\beta$  - quanto conta la direzione precedente
    weight_decay=1e-4      # L2 regularization opzionale
)
```

Quando usarlo: SGD con momentum è tuttora lo standard per l'addestramento di CNN per la visione artificiale (ResNet, EfficientNet). Converge a minimi spesso più "netti" rispetto ad Adam, risultando in modelli che generalizzano meglio su alcune architetture.

7.3 AdaGrad: learning rate adattivo per feature

Non tutte le feature sono ugualmente frequenti. In un modello linguistico, la parola "the" appare milioni di volte, mentre "serendipity" appare raramente. Con un learning rate fisso, i pesi per le parole rare non ricevono abbastanza aggiornamento.

AdaGrad divide il learning rate per la radice quadrata della somma dei gradienti passati al quadrato. I pesi che hanno ricevuto gradienti grandi e frequenti rallentano; quelli con gradienti piccoli accelerano.

```
G_t = G_{t-1} + (\nabla L)^2      [accumula gradienti quadrati]
w_t = w_{t-1} - \eta / \sqrt{G_t + \epsilon} \cdot \nabla L
```

Il problema: G_t cresce sempre — dopo molti step il learning rate diventa praticamente zero. Il modello smette di imparare.

```
optimizer = torch.optim.Adagrad(modello.parameters(), lr=0.01)
```

7.4 RMSProp: correggere AdaGrad con la media mobile

RMSProp risolve il problema di AdaGrad usando una **media mobile esponenziale** dei gradienti quadrati invece di sommarli. I gradienti vecchi "dimenticano" gradualmente.

```
E[g^2]_t = \gamma \cdot E[g^2]_{t-1} + (1-\gamma) \cdot (\nabla L)^2    [media mobile]
w_t = w_{t-1} - \eta / \sqrt{E[g^2]_t + \epsilon} \cdot \nabla L

optimizer = torch.optim.RMSprop(
    modello.parameters(),
    lr=0.001,
    alpha=0.99,      # \gamma - quanto "dimentica" i gradienti passati
    eps=1e-8
)
```

RMSProp fu proposto da Hinton nelle sue lezioni Coursera nel 2012 — non è mai stato pubblicato formalmente in un paper. È rimasto popolare per RNN e LSTM.

7.5 Adam: il re degli ottimizzatori

Adam (Adaptive Moment Estimation) combina momentum e RMSProp in un'unica formula elegante. Tiene traccia sia della media dei gradienti (primo momento = direzione) sia della media dei gradienti quadrati (secondo momento = velocità adattiva).

```
m_t = \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot \nabla L          [primo momento: direzione media]
v_t = \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot (\nabla L)^2      [secondo momento: varianza]
```

```

 $\hat{m}_t = m_t / (1 - \beta_1^t)$  [correzione bias iniziale]
 $\hat{v}_t = v_t / (1 - \beta_2^t)$ 

 $w_t = w_{\{t-1\}} - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ 

```

I valori default $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$ funzionano bene nella maggior parte dei casi.

```

optimizer = torch.optim.Adam(
    modello.parameters(),
    lr=1e-3,          # learning rate — il più sensibile da tuning
    betas=(0.9, 0.999),
    eps=1e-8
)

```

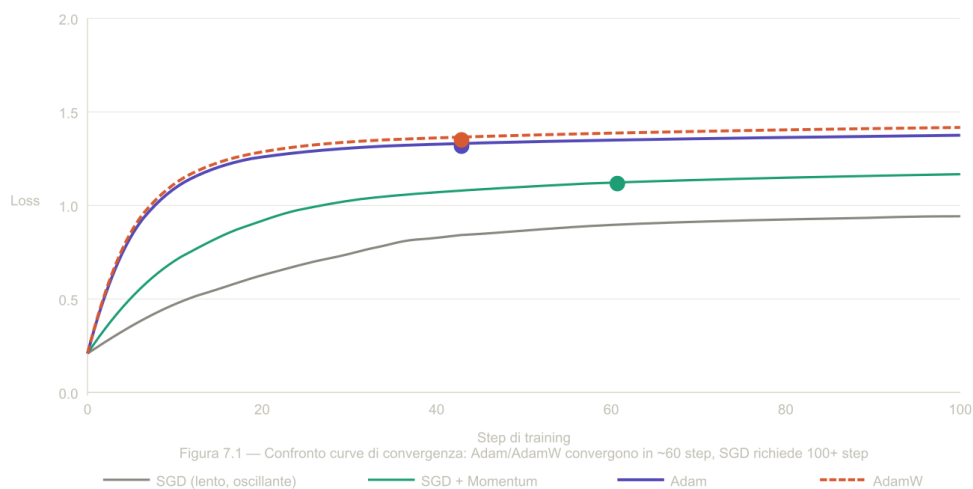


Figura 7.1 — Confronto curve di convergenza per i quattro ottimizzatori principali.

AdamW: la versione corretta di Adam

Adam originale ha un bug sottile: il weight decay (regolarizzazione L2) interagisce con l'adattamento del learning rate in modo indesiderato. **AdamW** separa i due meccanismi — weight decay applicato direttamente ai pesi, non attraverso il gradiente.

```

# AdamW — usa QUESTO invece di Adam per quasi tutto
optimizer = torch.optim.AdamW(
    modello.parameters(),
    lr=1e-3,
    weight_decay=0.01    # tipicamente 0.01-0.1 per LLM
)

```

AdamW è l'ottimizzatore standard per LLM, Transformer e la maggior parte del deep learning moderno. È usato per addestrare GPT, LLaMA, BERT e praticamente qualsiasi modello linguistico.

7.6 Learning Rate Scheduling: variare il passo durante il training

Un learning rate fisso per tutta la durata del training è raramente ottimale. All'inizio serve un passo grande per esplorare; verso la fine serve un passo piccolo per convergere con precisione. I **scheduler** lo gestiscono automaticamente.

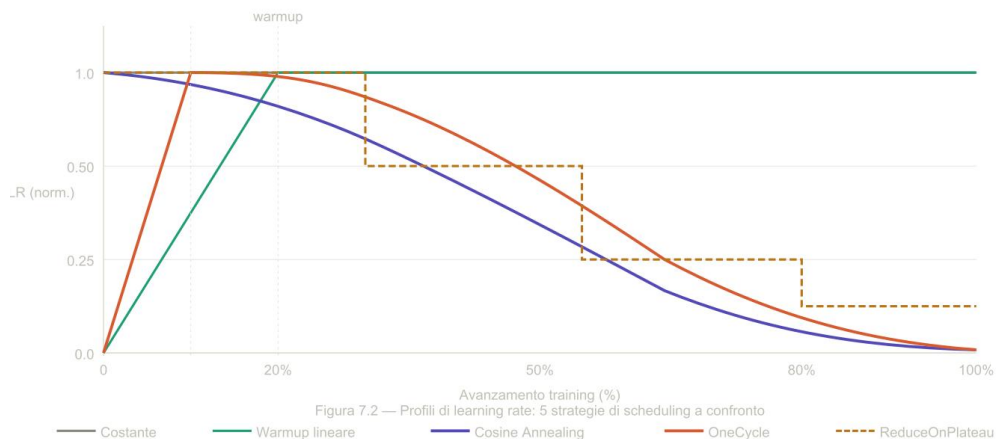


Figura 7.2 — I cinque learning rate scheduler più usati.

Warmup lineare

Nei primi N step, il learning rate parte da quasi-zero e cresce linearmente fino al valore target. Fondamentale per i Transformer: i pesi sono casuali all'inizio, e un learning rate grande in questa fase causa instabilità.

```
def lr_lambda(step, warmup_steps=1000, total_steps=10000):
    if step < warmup_steps:
        return step / warmup_steps      # cresce da 0 a 1
    return 1.0                          # poi rimane costante

scheduler = torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)
```

Cosine Annealing

Il learning rate segue una curva coseno da `lr_max` a `lr_min`. Produce una discesa smooth che spesso converge a minimi migliori rispetto allo step decay brusco.

```
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=num_epochs,      # periodo del coseno
    eta_min=1e-6           # lr minimo finale
)
```

Warmup + Cosine: il combo standard per LLM

```
import math

def lr_warmup_cosine(step, warmup=2000, total=100000, lr_max=1e-3, lr_min=1e-5):
    if step < warmup:
        return lr_max * step / warmup
    progress = (step - warmup) / (total - warmup)
    return lr_min + 0.5 * (lr_max - lr_min) * (1 + math.cos(math.pi * progress))

# In PyTorch
scheduler = torch.optim.lr_scheduler.LambdaLR(
    optimizer,
    lambda step: lr_warmup_cosine(step) / 1e-3 # normalizzato
)
```

ReduceLROnPlateau: adattarsi ai dati

Riduce il learning rate automaticamente quando la validation loss smette di migliorare. Utile quando non sai a priori quando il training dovrebbe rallentare.

```
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',          # monitora la loss (min = vogliamo ridurla)
    factor=0.5,          # moltiplica lr × 0.5 quando non migliora
    patience=5,          # aspetta 5 epoch prima di ridurre
    verbose=True
)

# Nel training loop, chiamalo con la validation loss
scheduler.step(val_loss)
```

7.7 Gradient Clipping: prevenire le esplosioni

In reti profonde o RNN, i gradienti possono diventare enormi in modo improvviso (gradient explosion). Un solo step con gradienti grandi può distruggere il training. Il **gradient clipping** taglia i gradienti se la loro norma supera una soglia.

```
# Nel training loop, prima di optimizer.step()
torch.nn.utils.clip_grad_norm_(
    modello.parameters(),
    max_norm=1.0      # soglia tipica: 0.5-5.0 a seconda del task
)
```

Senza clipping, un'esplosione del gradiente si manifesta come un picco improvviso nella curva di loss (spike), spesso seguito da valori NaN. Il clipping lo previene.

7.8 Il Training Loop Completo e Professionale

Integriamo tutto — ottimizzatore, scheduler, clipping, early stopping, logging — in un training loop pronto per la produzione.

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
import time

def train_model(modello, train_loader, val_loader,
                n_epochs=100, lr=1e-3, weight_decay=0.01,
                patience=10, max_norm=1.0, device='cpu'):
    """
    Training loop completo con:
    - AdamW optimizer
    - Warmup + Cosine Annealing scheduler
    - Gradient clipping
    - Early stopping
    - Logging dettagliato
    """
    modello = modello.to(device)
```

```

optimizer = torch.optim.AdamW(modello.parameters(),
                               lr=lr, weight_decay=weight_decay)
loss_fn = nn.CrossEntropyLoss()

total_steps = n_epochs * len(train_loader)
warmup_steps = total_steps // 10 # warmup = 10% degli step totali

scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer,
    max_lr=lr,
    total_steps=total_steps,
    pct_start=0.1, # 10% warmup
    anneal_strategy='cos' # poi cosine decay
)

# Early stopping
best_val_loss = float('inf')
best_weights = None
patience_counter = 0
history = {'train_loss': [], 'val_loss': [], 'val_acc': [], 'lr': []}

print(f"{'Epoch':>6} | {'Train Loss':>10} | {'Val Loss':>9} | "
      f"{'Val Acc':>8} | {'LR':>10} | {'Tempo':>6}")
print("-" * 60)

for epoch in range(1, n_epochs + 1):
    t0 = time.time()

    # — Training —————
    modello.train()
    train_loss_sum, n_batches = 0, 0

    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)

        # Forward pass
        y_pred = modello(X_batch)
        loss = loss_fn(y_pred, y_batch)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()

        # Gradient clipping - prima di step()
        torch.nn.utils.clip_grad_norm_(modello.parameters(), max_norm)

        optimizer.step()
        scheduler.step()

        train_loss_sum += loss.item()
        n_batches += 1

    train_loss = train_loss_sum / n_batches

    # — Validation —————
    modello.eval()
    val_loss_sum, correct, total = 0, 0, 0

```

```

with torch.no_grad():
    for X_batch, y_batch in val_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        y_pred = modello(X_batch)
        val_loss_sum += loss_fn(y_pred, y_batch).item()
        correct += (y_pred.argmax(1) == y_batch).sum().item()
        total += len(y_batch)

val_loss = val_loss_sum / len(val_loader)
val_acc = correct / total
cur_lr = optimizer.param_groups[0]['lr']
elapsed = time.time() - t0

# Salva storia
history['train_loss'].append(train_loss)
history['val_loss'].append(val_loss)
history['val_acc'].append(val_acc)
history['lr'].append(cur_lr)

print(f"{epoch:>6} | {train_loss:>10.4f} | {val_loss:>9.4f} | "
      f"{val_acc*100:>7.2f}% | {cur_lr:>10.2e} | {elapsed:>5.1f}s")

# Early stopping
if val_loss < best_val_loss:
    best_val_loss = val_loss
    best_weights = {k: v.clone() for k, v in modello.state_dict().items()}
    patience_counter = 0
else:
    patience_counter += 1
    if patience_counter >= patience:
        print(f"\nEarly stopping all'epoch {epoch}. "
              f"Ripristino pesi migliori (val_loss={best_val_loss:.4f})")
        modello.load_state_dict(best_weights)
        break

return history, modello

```

7.9 Diagnostica: cosa fare quando il training va storto

Saper leggere i sintomi di un training problematico è una skill fondamentale.

Sintomo	Causa probabile	Soluzione	
Loss subito NaN	LR troppo alto o init pesi sbagliata	Riduci LR 10×, controlla init	
Loss non scende affatto	LR troppo basso, bug nel codice, label sbagliate	Testa su un batch singolo, verifica gradienti	
Spike improvvisi nella loss	Gradient explosion	Aggiungi gradient clipping	
Validation loss sale ma train scende	Overfitting	Più dropout, weight decay, dati	
Training molto lento senza convergenza	LR troppo basso	Aumenta LR o usa scheduler	
Loss oscilla senza convergere	LR troppo alto	Riduci LR, aggiungi momentum	
Buona train loss, pessima val loss	Data leakage nel preprocessing	Rifai lo split correttamente	

```

# Tecnica diagnostica: test su un singolo batch
# Se il modello non riesce a fare overfitting su 1 batch, c'è un bug
X_one, y_one = next(iter(train_loader))

```



```

modello.train()
for i in range(100):
    pred = modello(X_one)
    loss = loss_fn(pred, y_one)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if i % 20 == 0:
        print(f"Step {i}: loss = {loss.item():.4f}")

# Se la loss non va verso 0 su 1 batch → bug architettura o loss
# Se va verso 0 → il modello funziona, il problema è altrove

```

7.10 Confronto finale degli ottimizzatori

| Ottimizzatore | Quando usarlo | Pro | Contro | |-----|-----|-----| | SGD | CNN per visione (ResNet, ecc.) | Minimi "netti", ottima generalizzazione | Richiede tuning LR accurato | | SGD + Momentum | Idem + più stabile | Meno oscillazioni di SGD puro | Idem | | Adam | Default per quasi tutto | Robusto, poca necessità di tuning | Può divergere su alcuni task | | AdamW | LLM, Transformer, deep learning moderno | Come Adam + corretto weight decay | Leggermente più costoso di SGD | | RMSProp | RNN, LSTM | Buon adattamento per sequenze | Meno usato oggi con Transformer |

Riepilogo del Capitolo 7

Hai imparato:

- Il **gradient descent puro**: un solo learning rate per tutti i pesi — semplice ma rigido.
- **SGD con Momentum**: inerzia che stabilizza il percorso. Ancora SOTA per CNN di visione.
- **AdaGrad/RMSProp**: learning rate adattivo per feature — meno frequenti ricevono passi più grandi.
- **Adam**: combina momentum e varianza adattiva. Il default per il deep learning moderno.
- **AdamW**: Adam con weight decay disaccoppiato. Usa questo per Transformer e LLM.
- Il **learning rate scheduler**: warmup + cosine annealing è il combo standard.
`ReduceLROnPlateau` quando non sai la durata del training.
- Il **gradient clipping**: protezione contro gradient explosion. `max_norm=1.0` è un buon default.
- Il **training loop professionale**: optimizer + scheduler + clipping + early stopping + logging integrati.
- La **diagnostica**: come leggere i sintomi e trovare la causa.

Esercizio 1 — Confronto ottimizzatori

Obiettivo: osservare empiricamente come si comportano i diversi ottimizzatori sullo stesso problema.

```
import torch, torch.nn as nn
```

```

import matplotlib.pyplot as plt

# Problema semplice: regressione su dati sintetici
torch.manual_seed(42)
X = torch.randn(500, 10)
y = (X[:, 0] * 2 - X[:, 1] + torch.randn(500) * 0.5).unsqueeze(1)

X_tr, X_te = X[:400], X[400:]
y_tr, y_te = y[:400], y[400:]

def crea_modello():
    return nn.Sequential(nn.Linear(10, 64), nn.ReLU(),
                        nn.Linear(64, 32), nn.ReLU(),
                        nn.Linear(32, 1))

ottimizzatori = {
    'SGD':      lambda p: torch.optim.SGD(p, lr=0.01, momentum=0.9),
    'Adam':     lambda p: torch.optim.Adam(p, lr=1e-3),
    'AdamW':    lambda p: torch.optim.AdamW(p, lr=1e-3, weight_decay=0.01),
    'RMSProp':  lambda p: torch.optim.RMSprop(p, lr=1e-3),
}

loss_fn = nn.MSELoss()
storia = {k: [] for k in ottimizzatori}

for nome, crea_opt in ottimizzatori.items():
    mod = crea_modello()
    opt = crea_opt(mod.parameters())
    for _ in range(200):
        pred = mod(X_tr)
        loss = loss_fn(pred, y_tr)
        opt.zero_grad(); loss.backward(); opt.step()
        storia[nome].append(loss.item())

plt.figure(figsize=(10, 4))
for nome, losses in storia.items():
    plt.plot(losses, label=nome)
plt.xlabel('Step'); plt.ylabel('Training Loss')
plt.title('Confronto ottimizzatori')
plt.legend(); plt.grid(True, alpha=0.3)
plt.yscale('log') # scala logaritmica per vedere le differenze
plt.savefig('optimizer_comparison.png', dpi=150)
plt.show()

# Domande:
# 1. Quale ottimizzatore converge più velocemente nei primi 50 step?
# 2. Quale raggiunge la loss più bassa dopo 200 step?
# 3. SGD è davvero peggio? In che senso?
# 4. Prova lr=0.1 per Adam. Cosa succede?

```

Esercizio 2 — Scheduler a confronto

Obiettivo: visualizzare come cambia il learning rate con diversi scheduler.

```
import torch
```

```

import matplotlib.pyplot as plt

# Visualizza solo come variano i learning rate – senza training reale
modello_dummy = torch.nn.Linear(1, 1)
lr_base = 1e-3
n_steps = 1000

schedulers = {
    'Costante': (torch.optim.Adam(modello_dummy.parameters(), lr=lr_base),
                 None),
    'Cosine Annealing': (torch.optim.Adam(modello_dummy.parameters(), lr=lr_base),
                         lambda opt: torch.optim.lr_scheduler.CosineAnnealingLR(opt,
T_max=n_steps)),
    'OneCycle': (torch.optim.Adam(modello_dummy.parameters(), lr=lr_base),
                 lambda opt: torch.optim.lr_scheduler.OneCycleLR(
opt, max_lr=lr_base, total_steps=n_steps,
pct_start=0.1)),
}

fig, ax = plt.subplots(figsize=(10, 4))
for nome, (opt, sched_fn) in schedulers.items():
    sched = sched_fn(opt) if sched_fn else None
    lrs = []
    for _ in range(n_steps):
        lrs.append(opt.param_groups[0]['lr'])
        opt.step()
        if sched: sched.step()
    ax.plot(lrs, label=nome)

ax.set_xlabel('Step'); ax.set_ylabel('Learning rate')
ax.set_title('Confronto learning rate scheduler')
ax.legend(); ax.grid(True, alpha=0.3)
plt.savefig('scheduler_comparison.png', dpi=150)
plt.show()

# Osserva: OneCycle ha warmup iniziale, poi decrescita coseno
# Domanda: qual è il vantaggio del warmup rispetto a partire subito da lr_base?

```

Questo conclude il Capitolo 7 e tutti i capitoli teorici del Volume 1. Il prossimo passo è il Progetto di Fine Volume — dove metteremo insieme tutto ciò che hai imparato in un sistema ML completo, dalla pipeline dati al deploy.

Progetto di Fine Volume 1

Sistema di Rilevamento Frodi su Transazioni Bancarie

Questo progetto integra tutto ciò che hai imparato nel Volume 1: dati, preprocessing, modelli classici, reti neurali, ottimizzatori e valutazione. È un problema reale affrontato da ogni banca del mondo.

Obiettivo del progetto

Costruire un sistema ML completo che rilevi transazioni bancarie fraudolente. Il modello riceve le caratteristiche di una transazione e deve predire se è fraudolenta (1) o legittima (0).

Perché questo problema è interessante didatticamente:

- Dataset fortemente sbilanciato (le frodi sono rare, ~0.17% del totale)
- Le metriche standard (accuracy) sono fuorvianti — devi scegliere quelle giuste
- Richiede confronto tra modelli classici e rete neurale
- Ha implicazioni di business concrete: ogni falso negativo è una frode non rilevata

Fase 1 — Dataset e prima ispezione

Il dataset usato è il **Credit Card Fraud Detection** di Kaggle (disponibile liberamente), oppure una versione sintetica che replica le sue caratteristiche statistiche.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore')

# — Opzione A: scarica il dataset reale da Kaggle —————
# kaggle datasets download -d mlg-ulb/creditcardfraud
# df = pd.read_csv('creditcard.csv')

# — Opzione B: dataset sintetico con stesse caratteristiche statistiche ———
np.random.seed(42)
n_legit = 28000
n_fraud = 48 # ~0.17% di frodi — come il dataset reale

def genera_transazioni(n, frode=False):
    """Genera transazioni con caratteristiche realistiche."""
    # V1-V14: componenti PCA simulate (come nel dataset reale)
    features = np.random.randn(n, 14)
```

```

    if frode:
        # Le frodi hanno pattern diversi dalle transazioni legittime
        features[:, 0] *= 0.3 # V1: molto diverso
        features[:, 3] -= 2.5 # V4: negativo per le frodi
        features[:, 10] += 1.8 # V11: spostato verso positivo
        amount = np.random.exponential(50 if not frode else 120, n)
        hour = np.random.randint(0, 24, n)
        return np.column_stack([features, amount, hour])

X_legit = genera_transazioni(n_legit, frode=False)
X_fraud = genera_transazioni(n_fraud, frode=True)
y_legit = np.zeros(n_legit)
y_fraud = np.ones(n_fraud)

X = np.vstack([X_legit, X_fraud])
y = np.concatenate([y_legit, y_fraud])

colonne = [f'V{i}' for i in range(1, 15)] + ['Amount', 'Hour']
df = pd.DataFrame(X, columns=colonne)
df['Class'] = y.astype(int)

# Shuffle
df = df.sample(frac=1, random_state=42).reset_index(drop=True)

print("=== PRIMA ISPEZIONE ===")
print(f"Shape: {df.shape}")
print(f"\nDistribuzione classi:")
print(df['Class'].value_counts())
print(f"\nTasso di frode: {df['Class'].mean()*100:.3f}%")
print(f"\nStatistiche Amount:")
print(df.groupby('Class')['Amount'].describe().round(2))

```

La prima cosa che noti è il **forte sbilanciamento**. Su 28.048 transazioni, solo 48 sono frodi. Questo è il problema centrale del progetto: devi trattarlo esplicitamente.

Fase 2 — Analisi esplorativa mirata

```

fig, assi = plt.subplots(2, 3, figsize=(16, 8))

# 1. Distribuzione importi - frode vs legittima
for cl, nome, col in [(0, 'Legittima', '#378ADD'), (1, 'Frode', '#E24B4A')]:
    dati = df[df['Class']==cl]['Amount']
    assi[0,0].hist(dati, bins=50, alpha=0.6, color=col, label=nome, density=True)
    assi[0,0].set_title('Distribuzione importi')
    assi[0,0].set_xlabel('Amount (€)')
    assi[0,0].legend()
    assi[0,0].set_yscale('log')

# 2. Ora del giorno - le frodi avvengono di notte?
frodi_ora = df[df['Class']==1]['Hour'].value_counts().sort_index()
legit_ora = df[df['Class']==0]['Hour'].value_counts().sort_index()
tasso_ora = (frodi_ora / (frodi_ora + legit_ora)).fillna(0) * 100
assi[0,1].bar(tasso_ora.index, tasso_ora.values, color='#D85A30')
assi[0,1].set_title('Tasso frode per ora del giorno')
assi[0,1].set_xlabel('Ora')

```

```

assi[0,1].set_ylabel('% frodi')

# 3. Feature più discriminanti (boxplot)
feature_importanti = ['V1', 'V4', 'V11']
for i, feat in enumerate(feature_importanti):
    dati_frode = df[df['Class']==1][feat]
    dati_legit = df[df['Class']==0][feat]
    assi[0,2].boxplot([dati_legit, dati_frode],
                      labels=['Legittima', 'Frode'],
                      positions=[i*2, i*2+0.8],
                      widths=0.6, patch_artist=True,
                      boxprops=dict(facecolor='#B5D4F4' if i==0 else '#F5C4B3'))
assi[0,2].set_title('Feature discriminanti (boxplot)')
assi[0,2].set_xticks([0.4, 2.4, 4.4])
assi[0,2].set_xticklabels(['V1', 'V4', 'V11'])

# 4. Correlazione delle feature con Class
correlazioni = df.corr()['Class'].drop('Class').abs().sort_values(ascending=False)
assi[1,0].barh(correlazioni.index[:10], correlazioni.values[:10], color='#534AB7')
assi[1,0].set_title('Correlazione con Class (top 10)')
assi[1,0].invert_yaxis()

# 5. Scatter delle due feature più discriminanti
assi[1,1].scatter(df[df['Class']==0]['V1'], df[df['Class']==0]['V4'],
                  alpha=0.1, s=5, color='#378ADD', label='Legittima')
assi[1,1].scatter(df[df['Class']==1]['V1'], df[df['Class']==1]['V4'],
                  alpha=0.8, s=40, color='#E24B4A', label='Frode', zorder=5)
assi[1,1].set_title('V1 vs V4: frodi visibili?')
assi[1,1].set_xlabel('V1'); assi[1,1].set_ylabel('V4')
assi[1,1].legend()

# 6. Matrice di correlazione (heatmap semplice)
corr_matrix = df[['V1', 'V4', 'V11', 'Amount', 'Class']].corr()
im = assi[1,2].imshow(corr_matrix.values, cmap='RdYlGn', vmin=-1, vmax=1)
assi[1,2].set_xticks(range(len(corr_matrix.columns)))
assi[1,2].set_yticks(range(len(corr_matrix.columns)))
assi[1,2].set_xticklabels(corr_matrix.columns, rotation=45)
assi[1,2].set_yticklabels(corr_matrix.columns)
assi[1,2].set_title('Matrice correlazione')
plt.colorbar(im, ax=assi[1,2])

plt.suptitle('Analisi esplorativa - Rilevamento Frodi', fontsize=14)
plt.tight_layout()
plt.savefig('eda_fraud.png', dpi=150, bbox_inches='tight')
plt.show()

```

Fase 3 — Preprocessing e gestione dello sbilanciamento

```

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import StratifiedKFold

# Separazione feature e label
X_raw = df.drop(columns=['Class']).values
y_raw = df['Class'].values

```

```

# Split stratificato - mantiene la % di frodi in ogni split
X_temp, X_test, y_temp, y_test = train_test_split(
    X_raw, y_raw, test_size=0.20, stratify=y_raw, random_state=42
)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, stratify=y_temp, random_state=42
)

print(f"Training:    {len(X_train):>6} ({y_train.mean()*100:.2f}% frodi)")
print(f"Validation: {len(X_val):>6}  ({y_val.mean()*100:.2f}% frodi)")
print(f"Test:        {len(X_test):>6} ({y_test.mean()*100:.2f}% frodi)")

# Normalizzazione (fit SOLO su training)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_val_s    = scaler.transform(X_val)
X_test_s   = scaler.transform(X_test)

# — Gestione sbilanciamento: tre strategie —————

# Strategia 1: class_weight - penalizza gli errori sulle frodi di più
class_weight = {0: 1.0, 1: n_legit / n_fraud} # ~583× più peso alle frodi
print(f"\nClass weight per frodi: {class_weight[1]:.0f}×")

# Strategia 2: SMOTE - genera frodi sintetiche (solo sul training set!)
try:
    from imblearn.over_sampling import SMOTE
    smote = SMOTE(random_state=42)
    X_train_sm, y_train_sm = smote.fit_resample(X_train_s, y_train)
    print(f"Dopo SMOTE - frodi nel training: {y_train_sm.mean()*100:.1f}%")
except ImportError:
    print("(installa imblearn per SMOTE: pip install imbalanced-learn)")
    X_train_sm, y_train_sm = X_train_s, y_train

# Strategia 3: soglia di decisione personalizzata (la vedremo in valutazione)

```

Fase 4 — Modelli classici con confronto sistematico

```

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import (classification_report, roc_auc_score,
                             average_precision_score, confusion_matrix)

import xgboost as xgb

modelli = {
    'Logistic Regression': LogisticRegression(
        C=1.0, class_weight='balanced', max_iter=1000
    ),
    'Random Forest': RandomForestClassifier(
        n_estimators=200, class_weight='balanced',
        n_jobs=-1, random_state=42
    ),
    'XGBoost': xgb.XGBClassifier(
        n_estimators=200, scale_pos_weight=class_weight[1],
        eval_metric='aucpr', random_state=42, verbosity=0
    )
}

```

```

    ),
}

print(f"\n{'Modello':<22} {'AUC-ROC':>8} {'AUC-PR':>7} {'Recall frodi':>13}
{'Precision frodi':>16}")
print("-" * 72)

risultati = {}
for nome, mod in modelli.items():
    mod.fit(X_train_s, y_train)
    prob = mod.predict_proba(X_val_s)[: , 1]
    pred = mod.predict(X_val_s)

    auc_roc = roc_auc_score(y_val, prob)
    auc_pr = average_precision_score(y_val, prob)

    cm = confusion_matrix(y_val, pred)
    recall_frodi = cm[1,1] / (cm[1,0] + cm[1,1]) if cm[1].sum() > 0 else 0
    prec_frodi = cm[1,1] / (cm[0,1] + cm[1,1]) if (cm[0,1]+cm[1,1]) > 0 else 0

    risultati[nome] = {'auc_roc':auc_roc,'auc_pr':auc_pr,'modello':mod,'prob':prob}
    print(f"{nome:<22} {auc_roc:>8.4f} {auc_pr:>7.4f} {recall_frodi:>13.3f}
{prec_frodi:>16.3f}")

# — Perché AUC-PR invece di AUC-ROC? —————
print("""
Nota sul dataset sbilanciato:
- AUC-ROC può sembrare ottima (>0.99) anche con un modello mediocre
  perché include molti veri negativi facili da classificare.
- AUC-PR (Area under Precision-Recall curve) è più onesta:
  misura quanto bene il modello bilancia precision e recall
  proprio sulla classe minoritaria (le frodi).
  Usa sempre AUC-PR per dataset fortemente sbilanciati.
""")

```

Fase 5 — Rete neurale con PyTorch

```

import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

# Convertire in tensori
X_tr_t = torch.FloatTensor(X_train_s)
y_tr_t = torch.FloatTensor(y_train)
X_val_t = torch.FloatTensor(X_val_s)
y_val_t = torch.FloatTensor(y_val)

train_ds = TensorDataset(X_tr_t, y_tr_t)
loader = DataLoader(train_ds, batch_size=256, shuffle=True)

# Architettura: rete con dropout e batch normalization
class FraudNet(nn.Module):
    def __init__(self, n_input):
        super().__init__()
        self.net = nn.Sequential(

```



```

        nn.Linear(n_input, 128),
        nn.BatchNorm1d(128),
        nn.ReLU(),
        nn.Dropout(0.3),

        nn.Linear(128, 64),
        nn.BatchNorm1d(64),
        nn.ReLU(),
        nn.Dropout(0.3),

        nn.Linear(64, 32),
        nn.ReLU(),

        nn.Linear(32, 1),
        nn.Sigmoid()          # output in (0,1) = probabilità di frode
    )

    def forward(self, x):
        return self.net(x).squeeze(1)

modello_nn = FraudNet(n_input=X_train_s.shape[1])

# Loss pesata per gestire lo sbilanciamento
peso_frodi = torch.tensor(class_weight[1])
loss_fn = nn.BCELoss(reduction='none')    # per applicare pesi manualmente

optimizer = torch.optim.AdamW(modello_nn.parameters(), lr=1e-3, weight_decay=0.01)
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer, max_lr=1e-3,
    total_steps=50 * len(loader), pct_start=0.1
)

best_auc, best_weights = 0, None

for epoch in range(50):
    modello_nn.train()
    for X_b, y_b in loader:
        pred = modello_nn(X_b)
        # Applica peso maggiore agli errori sulle frodi
        pesi = torch.where(y_b == 1, peso_frodi, torch.ones(1))
        loss = (loss_fn(pred, y_b) * pesi).mean()
        optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(modello_nn.parameters(), 1.0)
        optimizer.step()
        scheduler.step()

    # Valutazione
    modello_nn.eval()
    with torch.no_grad():
        prob_val = modello_nn(X_val_t).numpy()
        auc = average_precision_score(y_val, prob_val)

    if auc > best_auc:
        best_auc = auc
        best_weights = {k: v.clone() for k, v in modello_nn.state_dict().items()}

    if (epoch + 1) % 10 == 0:

```

```

        print(f"Epoch {epoch+1:3d}: AUC-PR = {auc:.4f}")

modello_nn.load_state_dict(best_weights)
modello_nn.eval()
with torch.no_grad():
    prob_nn = modello_nn(X_vl_t).numpy()
print(f"\nRete neurale - Best AUC-PR: {best_auc:.4f}")
risultati['Neural Network'] = {'auc_pr': best_auc, 'prob': prob_nn}

```

Fase 6 — La soglia di decisione: non è sempre 0.5

Per default, i classificatori usano 0.5 come soglia: se $P(\text{frode}) > 0.5 \rightarrow$ predice frode. Ma questa scelta è arbitraria. In un sistema antifrode, preferisci avere più falsi positivi (bloccare alcune transazioni legittime) che falsi negativi (lasciar passare una frode)?

```

import numpy as np
from sklearn.metrics import precision_recall_curve

# Scegli il modello migliore
migliore = max(risultati, key=lambda k: risultati[k].get('auc_pr', 0))
prob_migliore = risultati[migliore]['prob']
print(f"Modello scelto: {migliore}")

# Curve Precision-Recall per diverse soglie
precisioni, recall_vals, soglie = precision_recall_curve(y_val, prob_migliore)

# Trova la soglia ottimale per F1 (bilanciamento precision/recall)
f1_scores = 2 * (precisioni * recall_vals) / (precisioni + recall_vals + 1e-8)
idx_best = np.argmax(f1_scores)
soglia_ottimale = soglie[idx_best] if idx_best < len(soglie) else 0.5

print(f"\nSoglia default (0.50):      F1 = {f1_scores[np.searchsorted(soglie,0.5)]*100:.1f}%")
print(f"Soglia ottimale ({soglia_ottimale:.2f}): F1 = {f1_scores[idx_best]*100:.1f}%")

# Visualizza la curva PR
plt.figure(figsize=(8, 5))
plt.plot(recall_vals, precisioni, color='#534AB7', linewidth=2)
plt.scatter(recall_vals[idx_best], precisioni[idx_best],
            s=100, color='#E24B4A', zorder=5, label=f'Soglia ottimale ({soglia_ottimale:.2f})')
plt.xlabel('Recall (frodi catturate / frodi totali)')
plt.ylabel('Precision (frodi reali / frodi segnalate)')
plt.title('Curva Precision-Recall - scegli il punto in base al business')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('pr_curve.png', dpi=150, bbox_inches='tight')
plt.show()

# Predizioni finali con soglia ottimale
pred_finale_val = (prob_migliore >= soglia_ottimale).astype(int)
print(f"\nReport con soglia ottimale ({soglia_ottimale:.2f}):")
print(classification_report(y_val, pred_finale_val,
                             target_names=['Legittima', 'Frode']))

```

Fase 7 — Valutazione finale sul test set

```
# Seleziona il modello migliore sul validation set
# Poi valuta UNA VOLTA SOLA sul test set

print("="*60)
print("VALUTAZIONE FINALE SUL TEST SET")
print("="*60)

# Predici sul test set con il modello scelto
mod_finale = risultati[migliore]['modello'] if migliore != 'Neural Network' \
    else modello_nn

if migliore == 'Neural Network':
    modello_nn.eval()
    with torch.no_grad():
        prob_test = modello_nn(torch.FloatTensor(X_test_s)).numpy()
else:
    prob_test = mod_finale.predict_proba(X_test_s)[:, 1]

pred_test = (prob_test >= soglia_ottimale).astype(int)

print(f"\nModello: {migliore}")
print(f"Soglia: {soglia_ottimale:.2f}")
print(f"\nAUC-ROC: {roc_auc_score(y_test, prob_test):.4f}")
print(f"AUC-PR: {average_precision_score(y_test, prob_test):.4f}")
print(f"\n{classification_report(y_test, pred_test,
    target_names=['Legittima', 'Frode'])}")

cm = confusion_matrix(y_test, pred_test)
print(f"Confusion Matrix:")
print(f" Veri Negativi (legittima corretta): {cm[0,0]}")
print(f" Falsi Positivi (legittima bloccata): {cm[0,1]} ← cliente irritato")
print(f" Falsi Negativi (frode non rilevata): {cm[1,0]} ← perdita economica")
print(f" Veri Positivi (frode rilevata): {cm[1,1]} ← successo!")

# Stima impatto economico
importo_medio_frode = 120.0
importo_medio_legit = 50.0
perdita_fn = cm[1,0] * importo_medio_frode # frodi perse
costo_fp = cm[0,1] * importo_medio_legit * 0.05 # costo gestione falsi allarmi (5%)

print(f"\nImpatto economico stimato:")
print(f" Frodi non rilevate (FN × €{importo_medio_frode:.0f}): €{perdita_fn:.0f}")
print(f" Costo falsi allarmi (FP × €{importo_medio_legit*0.05:.1f}): €{costo_fp:.1f}")
print(f" Perdita totale stimata: €{perdita_fn + costo_fp:.0f}")
```

Fase 8 — Riflessioni finali e cosa hai imparato

Questo progetto ha richiesto di applicare in modo integrato tutto ciò che hai studiato nel Volume 1:

Dai dati: ispezione, gestione dello sbilanciamento, split stratificato, normalizzazione con fit corretto solo sul training.

Dai modelli classici: Logistic Regression come baseline, Random Forest per robustezza, XGBoost per performance massima. Hai visto che su questo tipo di problema tutti e tre competono bene con la rete neurale.

Dalla rete neurale: architettura con BatchNorm e Dropout, BCELoss pesata per lo sbilanciamento, training loop completo con AdamW, OneCycleLR e gradient clipping.

Dalla valutazione: hai capito perché l'accuracy è inutile su dataset sbilanciati, perché AUC-PR è più onesta di AUC-ROC, e come scegliere la soglia di decisione in base al contesto di business.

La lezione più importante: non esiste un modello "migliore" in assoluto. Esiste il modello migliore per il tuo problema, i tuoi dati, i tuoi vincoli di business. La tua competenza sta nel capire questi vincoli e fare la scelta giusta.

Checklist di consegna per un progetto ML professionale

Prima di consegnare qualsiasi progetto ML, verifica questi punti:

Dati:

- ☐ Split stratificato con random_state fisso e documentato
- ☐ Fit del preprocessor SOLO sul training set
- ☐ Nessun data leakage verificato
- ☐ Distribuzione delle label documentata in tutti i split

Modelli:

- ☐ Almeno una baseline semplice (LogisticRegression)
- ☐ Metriche appropriate al problema (non solo accuracy)
- ☐ Iperparametri documentati e motivati
- ☐ Modello finale scelto sul validation set, non sul test

Valutazione:

- ☐ Test set toccato UNA VOLTA SOLA alla fine
- ☐ Metriche riportate con il contesto (cosa significa per il business)
- ☐ Confusion matrix analizzata qualitativamente
- ☐ Differenza val/test < 5% (se no → sospetto overfitting)

Codice:

- ☐ Riproducibile (random_state fisso, nessuna variabile globale mutabile)
 - ☐ Commentato nei punti non ovvi
 - ☐ Requisiti (librerie e versioni) documentati
 - ☐ Risultati salvati (modello, scaler, metriche)
-

Esercizi opzionali per andare oltre

Livello 1 — Estendi il progetto: Aggiungi una feature ingegnerizzata: `importo_normalizzato_per_ora = Amount / (df.groupby('Hour')['Amount'].transform('mean') + 1)`. Migliora le performance?

Livello 2 — Ensemble manuale: Invece di scegliere un solo modello, combina le probabilità dei tre modelli classici con una media pesata. La performance migliora rispetto al singolo migliore?

Livello 3 — Spiegabilità: Usa SHAP (`pip install shap`) per spiegare le predizioni del modello XGBoost:

```
import shap
explainer = shap.TreeExplainer(risultati['XGBoost']['modello'])
shap_values = explainer.shap_values(X_val_s[:100])
shap.summary_plot(shap_values, X_val_s[:100], feature_names=colonne[:-1])
```

Chiusura del Volume 1

Hai completato il Volume 1. Ecco quello che ora sai fare:

Teoria: comprendere il funzionamento dell'IA a livello matematico e algoritmico — dal neurone artificiale al gradient boosting, dalla backpropagation agli ottimizzatori adattivi.

Pratica: costruire pipeline ML complete su dati reali — preprocessing corretto, split stratificato, confronto modelli, valutazione onesta, interpretazione dei risultati.

Strumenti: Python scikit-learn per ML classico, PyTorch per reti neurali, pandas e matplotlib per dati e visualizzazione, XGBoost/LightGBM per gradient boosting.

Mindset: scegliere il modello giusto per il problema giusto, leggere le curve di training per diagnosticare problemi, capire i limiti di ogni metrica.

Il Volume 2 entra nel deep learning avanzato: CNN per immagini, Transformer, LLM, RAG e agenti IA. Tutti i concetti di questo volume sono i mattoni su cui si costruisce tutto il resto.

Buona continuazione.

Appendice A

Glossario Completo

Ogni termine tecnico spiegato in italiano semplice

Come usare questo glossario: i termini sono ordinati alfabeticamente. Quando incontri una parola che non conosci durante la lettura, cerca qui la definizione in linguaggio semplice. I riferimenti tra parentesi indicano il capitolo in cui il termine viene approfondito.

A

Accuracy (Accuratezza) La frazione di predizioni corrette sul totale. Se il modello classifica correttamente 90 email su 100, l'accuracy è 90%. È la metrica più intuitiva, ma può essere fuorviante su dataset sbilanciati: un modello che dice sempre "non spam" ottiene il 99% di accuracy se lo spam è l'1% dei dati, pur essendo completamente inutile. (Cap. 6, 19)

Agente IA Un sistema in cui un LLM non si limita a rispondere a domande, ma pianifica ed esegue sequenze di azioni usando strumenti esterni. A differenza di un chatbot, un agente agisce nel mondo: naviga il web, scrive file, chiama API. La differenza fondamentale è la responsabilità: un chatbot risponde, un agente agisce con conseguenze reali. (Cap. 15)

Adam / AdamW Ottimizzatore che combina il momentum (memoria della direzione) con l'adattamento del learning rate per ogni peso. AdamW corregge un difetto di Adam separando il weight decay dal calcolo del gradiente. Entrambi sono lo standard per addestrare LLM e Transformer moderni. (Cap. 7)

Allucinazione (Hallucination) Quando un LLM genera informazioni false dette con sicurezza, come se fossero fatti verificati. Non è un bug nel codice — è una caratteristica intrinseca dei modelli probabilistici che completano sequenze di token. Rilevare e gestire le allucinazioni è uno dei problemi aperti più importanti dell'IA applicata. (Cap. 19, 23)

ANI — Artificial Narrow Intelligence Tutta l'IA che esiste oggi, senza eccezioni. Un sistema ANI eccelle in un task specifico ma non sa fare nulla al di fuori di esso. GPT-4 scrive codice eccellente ma non può guidare un'auto. Tutto ciò che costruirai in questa guida è ANI. (Cap. 1)

AGI — Artificial General Intelligence Sistema teorico capace di svolgere qualsiasi task cognitivo umano con flessibilità tra domini diversi. Non esiste ancora. (Cap. 1)

Attenzione (Attention) Meccanismo introdotto nel Transformer che permette a ogni token di "guardare" tutti gli altri contemporaneamente e assegnare loro pesi in base alla rilevanza. "Il gatto che mangiava" — il verbo "mangiava" assegna peso alto a "gatto" (il soggetto) e basso al resto. È il motore di tutti gli LLM moderni. (Cap. 11)

AUC-ROC Area Under the Receiver Operating Characteristic curve. Misura la qualità del ranking di un classificatore: quanto bene separa i positivi dai negativi indipendentemente dalla soglia di decisione. Vale tra 0.5 (casuale) e 1.0 (perfetto). (Cap. 6)

AUC-PR Area Under the Precision-Recall curve. Più onesta di AUC-ROC su dataset sbilanciati perché misura la performance direttamente sulla classe minoritaria. Su un dataset con 0.1% di frodi, usa AUC-PR invece di AUC-ROC. (Progetto Vol. 1)

Autograd Sistema di PyTorch che tiene traccia automaticamente di ogni operazione su un tensor e calcola i gradienti in automatico quando chiami `.backward()`. Non devi implementare la backpropagation a mano: PyTorch lo fa per te. (Cap. 8)

B

Backpropagation Algoritmo che calcola i gradienti di tutti i parametri di una rete neurale rispetto alla loss, percorrendo il grafo computazionale al contrario (dall'output verso l'input). Usa la chain rule del calcolo differenziale. In PyTorch: `loss.backward()`. (Cap. 4, 5)

Batch Sottinsieme del dataset processato insieme prima di aggiornare i pesi. Un batch da 32 significa che il modello vede 32 esempi, calcola il gradiente medio su tutti e 32, poi aggiorna i pesi. Più grande il batch, più stabile il gradiente ma più lenta ogni iterazione. (Cap. 4, 7)

Batch Normalization Tecnica che normalizza le attivazioni di ogni layer (media≈0, varianza≈1) dopo ogni mini-batch. Stabilizza il training, permette learning rate più alti e riduce l'overfitting. Va applicata SOLO durante il training, non durante l'inferenza (usa `model.train()` e `model.eval()`). (Cap. 5)

Benchmark Test standard per confrontare modelli diversi su un task specifico. MMLU misura la conoscenza generale, HumanEval misura la qualità del codice, HellaSwag misura il senso comune. Attenzione alla data contamination: i modelli potrebbero aver visto le risposte durante il training. (Cap. 19)

Bias (del modello)

13. Il parametro bias `b` in $y = wx + b$: sposta la soglia di attivazione di un neurone. 2. La tendenza sistematica del modello a sbagliare in un certo modo — per esempio, performare peggio su certi gruppi demografici. I due significati sono completamente diversi: il primo è un parametro tecnico, il secondo è un problema etico. (Cap. 2, 5)

BPE — Byte Pair Encoding Algoritmo di tokenizzazione che divide il testo in sottoparole frequenti invece di parole intere. "intelligenza" diventa tre token: "intel" + "ligen" + "za". Permette di gestire qualsiasi testo con un vocabolario fisso. (Cap. 12)

C

Chain Rule Regola del calcolo differenziale che permette di derivare funzioni composte: se $z = f(g(x))$, allora $dz/dx = (df/dg) \cdot (dg/dx)$. È il motore matematico della backpropagation nelle reti profonde. (Cap. 3, 5)

Chunking Processo di divisione di documenti lunghi in blocchi più piccoli prima di indicizzarli in un vector database per il RAG. La dimensione ottimale è tipicamente 300–600 token con un overlap di 50–100 token tra chunk consecutivi. (Cap. 14, 24)

Classificazione Task in cui il modello assegna un input a una categoria discreta. Spam/non-spam, gatto/cane/uccello, sentimento positivo/negativo/neutro sono tutti task di classificazione. (Cap. 6)

CNN — Convolutional Neural Network Architettura di rete neurale specializzata per dati con struttura spaziale (immagini, audio spettrogrammato). Usa filtri convoluzionali che scorrono sull'input invece di connettere ogni pixel a ogni neurone, riducendo drasticamente il numero di parametri. (Cap. 9)

Context Window La quantità massima di token che un LLM può "tenere in memoria" durante una singola interazione. Claude e GPT-4 hanno finestre da 200K e 128K token rispettivamente. Superata questa lunghezza, il modello "dimentica" i token più vecchi. (Cap. 12)

Cross-Entropy Loss Funzione di perdita standard per la classificazione. Penalizza il modello proporzionalmente a quanto è sicuro della risposta sbagliata: un modello che dice "97% sicuro che sia sbagliato" riceve una penalità molto più alta di uno incerto. (Cap. 3, 4)

Cross-Validation Tecnica per stimare le performance di un modello dividendo il dataset in K parti (fold) e addestrando/valutando K volte, usando ogni volta una parte diversa come validation set. Più robusto di un singolo split. (Cap. 6)

D

Data Augmentation Generazione artificiale di nuovi esempi di training tramite trasformazioni degli esempi esistenti. Per immagini: flip, rotazione, ritaglio, variazione di colore. Riduce l'overfitting aumentando la varietà dei dati senza raccogliere nuovi dati reali. Va applicata SOLO sul training set. (Cap. 4, 9)

Data Leakage Errore critico in cui informazioni del test set "trapelano" nel training, invalidando la valutazione. Esempio: calcolare la media del dataset intero per normalizzare invece di usare solo la media del training set. Il modello risulta artificialmente bravo in valutazione ma fallisce in produzione. (Cap. 2)

Dataset Collezione di dati usata per addestrare, validare o testare un modello. Ogni dataset è tipicamente diviso in training set (impara), validation set (ottimizza iperparametri), test set (valuta una sola volta). (Cap. 2)

Dead Neuron Neurone con ReLU che produce sempre output zero perché la sua pre-attivazione è sempre negativa. Il gradiente è zero, quindi smette di imparare. Si evita con inizializzazione He, learning rate non troppo alto, o usando Leaky ReLU. (Cap. 5)

Deep Learning Sottinsieme del Machine Learning che usa reti neurali con molti strati ("deep" = profondo). "Deep" si riferisce alla profondità della rete, non alla profondità del pensiero. È la tecnologia dietro il riconoscimento di immagini, la traduzione automatica e gli LLM. (Cap. 1)

DPO — Direct Preference Optimization Tecnica alternativa a RLHF per allineare un LLM alle preferenze umane. Invece di addestrare un reward model separato, ottimizza direttamente il modello su coppie di preferenze (risposta A è meglio di risposta B). Più stabile di RLHF, diventata lo standard nel 2024–2026. (Cap. 12)

Dropout Tecnica di regolarizzazione che durante il training "spegne" casualmente una frazione dei neuroni a ogni forward pass. Il modello non può fare affidamento su nessun neurone specifico,

sviluppando rappresentazioni più robuste. Disattivato durante l'inferenza (`model.eval()`). (Cap. 4, 5)

DVC — Data Version Control Strumento per versionare dataset e modelli grandi che non possono essere committati su Git. Funziona come Git ma per file binari pesanti, salvandoli su storage remoto (S3, GCS, Azure) e tenendo nel repository solo un piccolo file `.dvc` di riferimento. (Cap. 20)

E

Early Stopping Tecnica di regolarizzazione che interrompe il training quando la validation loss smette di migliorare per N epoch consecutive (patience). Ripristina i pesi del punto migliore. Evita l'overfitting senza dover sapere in anticipo il numero esatto di epoch. (Cap. 4)

Embedding Vettore denso di numeri reali che rappresenta un dato (parola, frase, immagine) in modo che elementi semanticamente simili abbiano vettori vicini nello spazio. "gatto" e "felino" hanno embedding simili; "gatto" e "database" li hanno distanti. Fondamentale per il RAG e il meccanismo di attenzione. (Cap. 14)

Encoder / Decoder Nel Transformer originale: l'encoder processa l'input con attenzione bidirezionale; il decoder genera l'output con attenzione causale (non guarda i token futuri). I modelli LLM moderni (GPT, LLaMA, Claude) usano solo il decoder. (Cap. 11)

Epoch Un passaggio completo su tutto il dataset durante il training. Se il dataset ha 10.000 esempi e il batch size è 32, un'epoch consiste di ~313 aggiornamenti dei pesi. (Cap. 4)

F

F1 Score Media armonica di precision e recall: $F1 = 2 \cdot (P \cdot R) / (P + R)$. Bilancia le due metriche ed è utile quando entrambe sono importanti. Vale tra 0 (pessimo) e 1 (perfetto). (Cap. 6)

Feature Una caratteristica numerica di un campione usata come input al modello. La lunghezza di un'email, il numero di link, la presenza di certe parole sono tutte feature. La scelta e costruzione delle feature si chiama feature engineering. (Cap. 2)

Feature Importance Misura di quanto ogni feature ha contribuito alle predizioni di un modello. Random Forest e XGBoost la calcolano automaticamente in base a quanto ogni feature riduce l'impurità nei nodi degli alberi. Utile per capire cosa ha imparato il modello e per selezionare le feature più informative. (Cap. 6)

Fine-tuning Processo di riaddestrare un modello pre-addestrato su un dataset specifico per adattarlo a un task o dominio particolare. Permette di partire dalle capacità generali del modello base invece di addestrare da zero. LoRA e QLoRA sono tecniche di fine-tuning efficienti. (Cap. 18)

Forward Pass Il calcolo che va dall'input all'output in una rete neurale: i dati scorrono in avanti attraverso i layer, ognuno applicando la sua trasformazione. Produce la predizione del modello. Seguito dal backward pass (backpropagation) durante il training. (Cap. 4, 5)

Function Calling (Tool Use) Meccanismo che permette a un LLM di richiedere l'esecuzione di una funzione esterna (strumento) con parametri specifici invece di rispondere con testo libero.

Permette agli agenti di interagire con il mondo reale: cercate informazioni, esegui codice, chiama API. (Cap. 15)

G

GELU — Gaussian Error Linear Unit Funzione di attivazione usata nei Transformer moderni. Più morbida di ReLU: non va esattamente a zero per valori negativi piccoli, ma si avvicina. Empiricamente produce risultati leggermente migliori di ReLU nelle architetture basate su attenzione. (Cap. 5, 11)

Gradient Descent Algoritmo di ottimizzazione che aggiorna i pesi del modello nella direzione opposta al gradiente della loss. "Opposta" perché il gradiente punta verso la salita della loss, e noi vogliamo scendere. La dimensione del passo è controllata dal learning rate. (Cap. 3, 4)

Gradiente Vettore delle derivate parziali di una funzione rispetto a tutte le sue variabili. Nella backpropagation, il gradiente della loss rispetto a ogni peso dice: "se aumenti questo peso di un po', la loss aumenta o diminuisce? Di quanto?". Punta sempre nella direzione di salita più ripida. (Cap. 3)

Gradient Clipping Tecnica che limita la norma del gradiente a un valore massimo prima dell'aggiornamento dei pesi. Previene l'exploding gradient (gradiente che esplode a valori enormi) tipico delle RNN e reti molto profonde. `torch.nn.utils.clip_grad_norm_(params, max_norm=1.0)`. (Cap. 7, 10)

GRU — Gated Recurrent Unit Versione semplificata dell'LSTM con meno parametri. Combina forget gate e input gate in un unico update gate, eliminando il cell state separato. Spesso raggiunge performance comparabili all'LSTM con training più veloce. (Cap. 10)

Guardrail Componente di sicurezza che si interpone tra l'input/output e il modello, applicando policy di sicurezza configurabili. Può essere basato su regex (veloce, rigido) o su un LLM classificatore (flessibile, costoso). NeMo Guardrails e LlamaFirewall sono framework open source. (Cap. 22)

H

Hidden State Il vettore di "memoria" di una RNN o LSTM che viene passato da un passo temporale al successivo. Accumula informazioni sul contesto passato. Nell'LSTM ci sono due hidden state: h (output del passo) e c (cell state, memoria a lungo termine). (Cap. 10)

HuggingFace Piattaforma e libreria open source che ospita migliaia di modelli pre-addestrati, dataset e spazi interattivi. La libreria `transformers` è lo standard de facto per caricare e usare LLM in Python. (App. B)

Human-in-the-loop Pattern architetturale che richiede approvazione umana prima di eseguire azioni irreversibili o ad alto impatto. Un agente che sta per inviare un'email o eliminare file deve chiedere conferma all'utente. Fondamentale per sistemi agentici sicuri. (Cap. 22)

HyDE — Hypothetical Document Embedding Tecnica RAG avanzata: invece di embeddare la query dell'utente direttamente, si genera prima una risposta ipotetica alla query, poi si usa quella

per il retrieval. Le risposte ipotetiche sono semanticamente più simili ai documenti rilevanti rispetto alle domande grezze. (Cap. 14)

Hyperparameter (Iperparametro) Parametro che tu scegli prima del training e che non viene imparato automaticamente dal modello. Learning rate, batch size, numero di layer, dropout rate sono tutti iperparametri. I parametri del modello (pesi e bias) invece si imparano automaticamente. (Cap. 4)

I

Iperparametro → vedi *Hyperparameter*

Inferenza Usare un modello già addestrato per fare predizioni su nuovi dati. Contrapposto al training: durante l'inferenza i pesi non cambiano, il dropout è disattivato, e il calcolo dei gradienti è disabilitato (`torch.no_grad()`). (Cap. 1)

K

KL Divergence Kullback-Leibler Divergence: misura quanto una distribuzione di probabilità si discosta da un'altra. Non è simmetrica: $D_{KL}(P||Q) \neq D_{KL}(Q||P)$. Usata nell'RLHF per penalizzare il modello se si allontana troppo dal modello originale durante il fine-tuning. (Cap. 3)

K-Means Algoritmo di clustering non supervisionato che raggruppa i dati in K cluster scegliendo iterativamente i centroidi che minimizzano la distanza media intra-cluster. Richiede di specificare K in anticipo; l'elbow method aiuta a sceglierlo. (Cap. 6)

L

Label (Etichetta) La risposta corretta per un esempio nel supervised learning. In un dataset di email spam, la label è "spam" o "non spam". Il modello impara a predire la label a partire dalle feature. (Cap. 2)

Latency (Latenza) Tempo che intercorre tra l'invio di una richiesta e la ricezione della risposta. In produzione si misura al 50° percentile (P50), 95° (P95) e 99° (P99). Il P95 rappresenta il caso più comune di utente "sfortunato" e è la metrica più utile per la user experience. (Cap. 20)

Layer (Strato) Gruppo di neuroni in una rete neurale che riceve input dal layer precedente e passa output al layer successivo. I layer nascosti trasformano progressivamente le feature; il layer di output produce la predizione finale. (Cap. 5)

Learning Rate Quanto grandi sono i passi dell'aggiornamento dei pesi durante il gradient descent. Troppo alto: il modello salta sopra il minimo e la loss oscilla o esplode. Troppo basso: il training converge ma richiede molto più tempo. È l'iperparametro più critico da ottimizzare. (Cap. 4, 7)

LLM — Large Language Model Modello di deep learning (basato su Transformer) addestrato su enormi quantità di testo con l'obiettivo di predire il token successivo. Da questo task emergono capacità di linguaggio, ragionamento e conoscenza generale. GPT-4, Claude, LLaMA sono tutti LLM. (Cap. 12)

LLM-as-Judge Tecnica di valutazione in cui si usa un LLM potente (es. Claude) per valutare la qualità delle risposte di un altro modello. Scalabile e spesso allineato con il giudizio umano, ma soggetto a bias come la preferenza per risposte verbose. (Cap. 19)

LoRA — Low-Rank Adaptation Tecnica di fine-tuning efficiente che aggiunge due matrici piccole (A e B) ai layer del modello invece di aggiornare tutti i parametri. Riduce i parametri addestrabili del 99% mantenendo il 90% dei risultati del full fine-tuning. Gli adapter LoRA sono file leggeri (<300 MB) separati dal modello base. (Cap. 18)

Loss Function (Funzione di Perdita) Funzione che misura quanto è sbagliata la predizione del modello. Durante il training si cerca di minimizzarla. MSE per la regressione, Cross-Entropy per la classificazione sono le più comuni. (Cap. 4)

LSTM — Long Short-Term Memory Architettura di rete ricorrente che risolve il vanishing gradient delle RNN semplici introducendo un cell state e tre porte (forget, input, output) che controllano il flusso di informazioni. Capace di memorizzare dipendenze a lungo raggio nelle sequenze. (Cap. 10)

M

MCP — Model Context Protocol Protocollo open standard proposto da Anthropic (2024) per connettere LLM a strumenti esterni in modo standardizzato. Un server MCP espone Tool (azioni), Resources (dati) e Prompts (workflow). Claude Desktop lo supporta nativamente. (Cap. 16)

MLflow Strumento per il tracking degli esperimenti di machine learning: registra parametri, metriche, modelli e artifact di ogni training run. Permette di confrontare esperimenti diversi e di riprodurre i risultati. (Cap. 20)

MLP — Multi-Layer Perceptron Rete neurale con strati di neuroni completamente connessi (ogni neurone connesso a tutti quelli del layer successivo). La forma più semplice di rete neurale profonda. Ottima per dati tabellari, meno efficiente di CNN per immagini o Transformer per testo. (Cap. 5)

Model Drift Degradazione delle performance di un modello in produzione nel tempo, causata dal cambiamento della distribuzione dei dati reali rispetto a quella del training set. Può essere silenzioso: il modello non crasha, risponde semplicemente sempre peggio. Richiede monitoring continuo. (Cap. 20)

Modello Il "programma" che ha imparato dai dati. In pratica: un file (.pt, .bin, .safetensors) che contiene miliardi di numeri (i pesi) e l'architettura per usarli. Non capisce nulla — ma produce output sorprendentemente utili. (Cap. 1)

Momentum Tecnica che aggiunge inerzia al gradient descent: invece di seguire solo il gradiente corrente, accumula una "velocità" media dei gradienti passati. Riduce le oscillazioni e accelera la convergenza in direzioni coerenti. (Cap. 7)

MSE — Mean Squared Error Errore quadratico medio: media dei quadrati delle differenze tra predizioni e valori reali. Loss standard per la regressione. Penalizza gli errori grandi più di quelli piccoli (per via del quadrato). La radice quadrata (RMSE) ha la stessa unità di misura dei dati originali. (Cap. 4)

Multi-Head Attention Versione del meccanismo di attenzione che esegue h teste in parallelo, ognuna con le proprie matrici Q, K, V. Ogni testa impara a prestare attenzione a relazioni diverse (soggetto-verbo, pronomi-antecedente, ecc.). Gli output vengono concatenati e riproiettati. (Cap. 11)

N

nn.Module La classe base di PyTorch da cui eredita ogni rete neurale. Richiede di definire `__init__` (dove dichiarare i layer) e `forward` (dove descrivere il flusso dei dati). Gestisce automaticamente i parametri, il salvataggio e il caricamento del modello. (Cap. 8)

O

One-Hot Encoding Conversione di variabili categoriche in vettori binari: ogni categoria diventa una colonna con 1 dove la categoria è presente e 0 altrove. "Milano", "Roma", "Napoli" diventano [1,0,0], [0,1,0], [0,0,1]. Necessario perché i modelli lavorano solo con numeri. (Cap. 2)

OWASP Top 10 per LLM Lista delle 10 vulnerabilità più critiche per sistemi basati su LLM, pubblicata dall'OWASP. Include prompt injection, data poisoning, excessive agency e altre. Aggiornata periodicamente; la versione 2025 è il riferimento corrente. (Cap. 21)

Overfitting Quando il modello memorizza il training set invece di generalizzare. Sintomi: loss di training molto bassa, loss di validation alta e crescente. Analogia: uno studente che memorizza le risposte degli esercizi del libro invece di capire i concetti. (Cap. 4)

P

Padding

14. In CNN: pixel artificiali aggiunti ai bordi dell'immagine per mantenere le dimensioni dopo la convoluzione. 2. In NLP: token speciali `[PAD]` aggiunti alle sequenze più corte per portarle alla stessa lunghezza in un batch. (Cap. 9)

Parametri (Pesi) I numeri dentro il modello che cambiano durante il training. GPT-4 ha ~1.8 trilioni di parametri. Più parametri = più capacità di modellare relazioni complesse, ma più dati e compute necessari. (Cap. 1)

Perceptron Il modello di neurone artificiale più semplice: somma pesata degli input + bias, poi funzione a gradino come attivazione. Converge sui dati linearmente separabili ma non può risolvere XOR. Il punto di partenza storico del deep learning moderno. (Cap. 5)

Pooling Operazione che riduce le dimensioni spaziali di una feature map prendendo il massimo (MaxPooling) o la media (AvgPooling) in finestre locali. Riduce i parametri, introduce invarianza alla posizione e allarga il campo recettivo nei layer successivi. (Cap. 9)

Pre-training La fase iniziale e più costosa di addestramento di un LLM: il modello viene addestrato su trilioni di token con l'obiettivo di predire il token successivo. Da questo emergono capacità

generali di linguaggio e conoscenza. Il pre-training costa milioni di dollari per i modelli grandi. (Cap. 12)

Precision Di tutti i campioni che il modello ha classificato come positivi, quanti lo erano davvero? Alta precision = pochi falsi allarmi. Formula: $TP / (TP + FP)$. (Cap. 6)

Prompt Engineering Disciplina di progettare istruzioni per LLM che producano output utili in modo affidabile. Include zero-shot, few-shot, chain-of-thought e altre tecniche. Non è "scrivere istruzioni creative" — è un ciclo iterativo di progettazione, test e misurazione. (Cap. 13)

Prompt Injection Attacco in cui input malevoli cercano di sovrascrivere le istruzioni del system prompt. Diretta: l'utente include le istruzioni nell'input. Indiretta: le istruzioni sono nascoste in documenti recuperati dal RAG. Una delle vulnerabilità più critiche dei sistemi LLM. (Cap. 21, 22)

Q

QLoRA Combinazione di quantizzazione 4-bit del modello base con addestramento LoRA degli adapter. Permette il fine-tuning di modelli da 7B su GPU consumer da 16 GB. Qualità leggermente inferiore a LoRA FP16 ma molto più accessibile. (Cap. 18)

Quantizzazione Riduzione della precisione numerica dei pesi del modello: da FP32 (32 bit) a BF16 (16 bit) o INT8/INT4. Riduce l'uso di memoria e aumenta la velocità di inferenza con perdita di qualità minima se fatta correttamente. (Cap. 17)

R

RAG — Retrieval-Augmented Generation Tecnica che combina recupero di documenti rilevanti da una knowledge base con generazione di testo da un LLM. Risolve due limiti degli LLM: conoscenza obsoleta e assenza di dati privati. Non richiede di riaddestrare il modello. (Cap. 14)

Random Forest Ensemble di alberi decisionali addestrati su sottoinsiemi casuali dei dati e delle feature. La predizione finale è il voto di maggioranza. Robusto, richiede poco tuning, fornisce feature importance. Ancora SOTA su molti task tabellari. (Cap. 6)

Recall Di tutti i campioni effettivamente positivi, quanti ne ha catturati il modello? Alta recall = pochi falsi negativi. Formula: $TP / (TP + FN)$. In un sistema antifrode, alta recall significa poche frodi non rilevate. (Cap. 6)

Regressione Task in cui il modello predice un valore continuo (prezzo, temperatura, stipendio). Contrapposto alla classificazione che predice categorie discrete. La loss standard per la regressione è MSE. (Cap. 6)

ReLU — Rectified Linear Unit Funzione di attivazione: $ReLU(x) = \max(0, x)$. Semplice, veloce, non satura per $x > 0$. Il default per i layer nascosti di CNN e MLP. Problema del dead neuron per $x < 0$ cronico. (Cap. 5)

ResNet Architettura CNN con skip connections che permettono di addestrare reti molto profonde (50, 101, 152 layer) senza vanishing gradient. Le skip connections danno al gradiente una "scorciatoia" diretta verso i layer iniziali. Ancora ampiamente usata come backbone. (Cap. 9)

RLHF — Reinforcement Learning from Human Feedback Tecnica per allineare un LLM alle preferenze umane. Valutatori umani confrontano coppie di risposte; le loro preferenze addestrano un reward model; il LLM viene ottimizzato per massimizzare il reward. Usato in ChatGPT, Claude e altri assistenti. DPO è un'alternativa più stabile. (Cap. 12)

RMSNorm Versione semplificata della Layer Normalization usata nei LLM moderni (LLaMA, Mistral). Normalizza dividendo per la radice quadrata della media dei quadrati, senza calcolare media e varianza. Più veloce di LayerNorm. (Cap. 12)

RNN — Recurrent Neural Network Rete neurale con uno stato interno (hidden state) che scorre da un passo temporale al successivo, permettendo di modellare sequenze. Soffre di vanishing gradient su sequenze lunghe. Oggi largamente sostituita da Transformer, ma ancora utile per streaming e sequenze molto lunghe. (Cap. 10)

RoPE — Rotary Positional Embedding Tecnica di encoding della posizione usata nei LLM moderni (LLaMA, Mistral). Invece di sommare un vettore di posizione fisso, ruota i vettori Q e K nell'operazione di attenzione. Generalizza meglio a sequenze più lunghe di quelle viste durante il training. (Cap. 12)

S

Self-Attention Meccanismo in cui ogni token in una sequenza calcola i suoi pesi di attenzione rispetto a tutti gli altri token nella stessa sequenza. Permette di catturare dipendenze a qualsiasi distanza senza il problema del vanishing gradient delle RNN. (Cap. 11)

SFT — Supervised Fine-Tuning Prima fase di allineamento di un LLM dopo il pre-training: il modello viene addestrato su esempi curati di (domanda, risposta ideale) per imparare il formato e lo stile di una conversazione utile. (Cap. 12)

SHAP Tecnica per spiegare le predizioni di qualsiasi modello ML calcolando il contributo di ogni feature alla predizione. Basata sulla teoria dei giochi (valori di Shapley). Utile per audit di modelli e per identificare feature problematiche. (Progetto Vol. 1)

Sigmoid Funzione di attivazione che schiaccia qualsiasi valore nell'intervallo (0, 1). Usata per output di classificazione binaria (interpretato come probabilità). Soffre di saturazione per valori estremi → vanishing gradient. Non usare nei layer nascosti. (Cap. 5)

Skip Connection Connessione diretta che bypassa uno o più layer, sommando l'input all'output del blocco. Introdotta da ResNet, permette al gradiente di fluire direttamente verso i layer iniziali senza attraversare tutti i layer intermedi. (Cap. 9)

SMOTE — Synthetic Minority Oversampling Technique Tecnica per bilanciare dataset sbilanciati generando nuovi esempi sintetici della classe minoritaria interpolando tra esempi reali. Va applicata SOLO sul training set per evitare data leakage. (Progetto Vol. 1, Cap. 6)

Softmax Funzione che converte un vettore di logit (numeri grezzi) in una distribuzione di probabilità che somma a 1. L'output di ogni classificatore multi-classe passa attraverso softmax prima di interpretarlo come probabilità. (Cap. 3, 5)

SVM — Support Vector Machine Algoritmo che trova l'iperpiano con massimo margine tra le classi. Il kernel trick permette di separare classi non linearmente separabili proiettandole

implicitamente in spazi ad alta dimensionalità. Ottimo su dataset piccoli con alta dimensionalità. (Cap. 6)

T

Temperature Parametro che controlla la "casualità" delle predizioni di un LLM dividendo i logit prima del softmax. Temperature bassa (0.1) → predizioni quasi deterministiche. Temperature alta (1.5) → output più creativo e vario. Per task fattuali usa temperature bassa; per brainstorming usa alta. (Cap. 12)

Tensor L'unità fondamentale di PyTorch. Concettualmente identico a un array NumPy (scalare, vettore, matrice, o array N-dimensionale), ma con due superpoteri: può girare su GPU e traccia le operazioni per il calcolo automatico dei gradienti. (Cap. 8)

Token L'unità minima in cui il testo viene spezzato prima di essere elaborato da un LLM. Non corrisponde necessariamente a una parola: può essere una sottoparola, un carattere, o un simbolo. "intelligenza" → 3 token. Il contesto di un LLM si misura in token. (Cap. 2, 12)

Training Il processo con cui il modello impara dai dati. Consiste nel ripetere migliaia di volte il ciclo: predici → misura l'errore → calcola i gradienti → aggiorna i pesi. (Cap. 1, 4)

Transfer Learning Riusare un modello addestrato su un task generico (ImageNet, testo del web) per un task specifico diverso, modificando solo i layer finali. Permette di ottenere ottimi risultati anche con pochi dati, perché i layer iniziali hanno già imparato feature generali utili. (Cap. 9, 18)

Transformer Architettura di rete neurale basata esclusivamente sul meccanismo di attenzione, senza ricorrenza. Introdotta nel paper "Attention Is All You Need" (2017). Permette elaborazione parallela di tutti i token, attenzione globale senza vanishing gradient, e scalabilità eccellente. La base di quasi tutti gli LLM moderni. (Cap. 11)

U

Underfitting Quando il modello è troppo semplice per catturare i pattern nei dati. Sintomi: loss alta sia sul training che sul validation set. Cause: modello troppo piccolo, training troppo breve, learning rate troppo basso. (Cap. 4)

V

Vanishing Gradient Problema nelle reti profonde in cui il gradiente diventa così piccolo durante la backpropagation (moltiplicando tante volte per valori < 1) che i layer iniziali smettono di imparare. Risolto da ReLU (per reti profonde), LSTM (per sequenze) e skip connections (per reti molto profonde). (Cap. 5, 10)

Vector Database Database ottimizzato per memorizzare vettori di embedding e cercare i più simili a una query in modo efficiente. ChromaDB (locale, per prototipi), Pinecone (cloud), pgvector (PostgreSQL) e FAISS sono le opzioni principali. Fondamentale per il RAG. (Cap. 14)

vLLM Framework per servire LLM in produzione ad alta throughput. Usa continuous batching (raggruppa dinamicamente le richieste) e PagedAttention (gestisce la KV-cache come memoria virtuale). Espone un'API compatibile con OpenAI. (Cap. 20)

W

Weight Decay Regularizzazione L2 che aggiunge alla loss un termine proporzionale alla somma dei quadrati dei pesi. Spinge i pesi verso zero, riducendo l'overfitting. In AdamW è implementato in modo corretto, disaccoppiato dall'adattamento del learning rate. (Cap. 4, 7)

X

XGBoost Implementazione ottimizzata del Gradient Boosting con regularizzazione L1/L2, gestione nativa dei NaN e parallelizzazione efficiente. SOTA su dati tabellari nelle competizioni Kaggle. Usa early stopping per trovare automaticamente il numero ottimale di alberi. (Cap. 6)

Z

Zero-shot / Few-shot Tecniche di prompting per LLM. Zero-shot: solo l'istruzione, nessun esempio. Few-shot: istruzione più 2–5 esempi input→output nel prompt. Il few-shot migliora la consistenza del formato ma non la comprensione del task. (Cap. 13)

Appendice B

Cheatsheet Python per l'IA

NumPy · pandas · scikit-learn · PyTorch · HuggingFace · Anthropic API

Questa appendice è un riferimento rapido. Non spiega i concetti — li trovi nei capitoli. Qui trovi la sintassi corretta quando hai dimenticato come si scrive qualcosa.

NumPy — Calcolo Numerico

```
import numpy as np

# --- Creazione array ---
a = np.array([1, 2, 3])          # da lista Python
b = np.zeros((3, 4))            # matrice 3x4 di zeri
c = np.ones((2, 3))            # matrice 2x3 di uno
d = np.random.randn(5, 10)      # valori casuali (normale std)
e = np.random.rand(5, 10)       # valori casuali (uniforme 0-1)
f = np.arange(0, 10, 2)         # [0, 2, 4, 6, 8]
g = np.linspace(0, 1, 100)      # 100 valori equidistanti tra 0 e 1
h = np.eye(4)                   # matrice identità 4x4

# --- Shape e reshape ---
a.shape                          # (3,)
b.reshape(2, 6)                 # stessi dati, nuova forma
b.flatten()                     # appiattisce in 1D
np.expand_dims(a, axis=0)        # (3,) → (1, 3)
a[:, np.newaxis]                # (3,) → (3, 1)

# --- Operazioni ---
a + b; a * b; a ** 2             # elemento per elemento
a @ b                           # prodotto matriciale (dot product)
np.dot(a, b)                    # equivalente a @
a.sum(); a.mean(); a.std(); a.max(); a.min() # riduzioni
a.sum(axis=0)                   # somma per colonna
a.sum(axis=1)                   # somma per riga

# --- Indicizzazione ---
a[2]                            # elemento singolo
a[1:4]                          # slice
a[[0, 2, 4]]                   # fancy indexing
a[a > 0]                        # boolean indexing
a[a > 0] = 0                    # assegnazione condizionale

# --- Funzioni utili ---
np.argmax(a)                    # indice del massimo
np.argsort(a)                   # indici ordinati
np.unique(a)                    # valori unici
np.concatenate([a, b], axis=0)  # concatena lungo asse
```

```

np.stack([a, b], axis=0)           # crea nuova dimensione
np.linalg.norm(a)                  # norma L2
np.linalg.norm(a, axis=1)          # norma L2 per ogni riga
np.clip(a, 0, 1)                   # tronca a [0, 1]
np.where(a > 0, a, 0)              # equivalente a ReLU

# --- Seed per riproducibilità ---
np.random.seed(42)

```

pandas — Manipolazione Dati

```

import pandas as pd
import numpy as np

# --- Caricamento ---
df = pd.read_csv("file.csv")
df = pd.read_csv("file.csv", sep=";", encoding="utf-8")
df = pd.read_json("file.json")
df = pd.read_excel("file.xlsx", sheet_name="Foglio1")

# --- Ispezione ---
df.shape           # (righe, colonne)
df.dtypes          # tipi di colonna
df.head(5)         # prime 5 righe
df.tail(5)         # ultime 5 righe
df.describe()      # statistiche descrittive (numeriche)
df.info()          # dtypes + valori non-null
df.isnull().sum()  # NaN per colonna
df.duplicated().sum() # righe duplicate
df["col"].value_counts() # frequenza di ogni valore
df["col"].nunique() # numero di valori unici

# --- Selezione ---
df["colonna"]      # serie
df[["col1", "col2"]] # DataFrame con più colonne
df.loc[0]          # riga per label/indice
df.iloc[0]         # riga per posizione
df.loc[df["col"] > 5] # filtro per condizione
df.query("col > 5 and col2 == 'A'") # filtro con stringa

# --- Modifica ---
df["nuova"] = df["a"] + df["b"] # nuova colonna
df.drop(columns=["col"])        # elimina colonna
df.rename(columns={"old": "new"}) # rinomina
df.drop_duplicates()            # rimuovi duplicati
df.reset_index(drop=True)       # resetta l'indice

# --- Valori mancanti ---
df.dropna()           # elimina righe con NaN
df.dropna(subset=["col"]) # elimina solo se NaN in "col"
df.fillna(0)          # sostituisce NaN con 0
df["col"].fillna(df["col"].median()) # sostituisce con mediana

# --- Aggregazione ---
df.groupby("col")["val"].mean() # media per gruppo

```

```

df.groupby("col").agg({"val": ["mean", "std", "count"]})
df.pivot_table(values="val", index="a", columns="b", aggfunc="mean")

# --- Merge e join ---
pd.merge(df1, df2, on="chiave", how="left") # left join
pd.concat([df1, df2], axis=0, ignore_index=True) # unisci righe

# --- Salvataggio ---
df.to_csv("output.csv", index=False)
df.to_json("output.jsonl", orient="records", lines=True)

# --- JSONL (per dataset LLM) ---
import json
# Scrivi
with open("dataset.jsonl", "w") as f:
    for row in df.itertuples():
        f.write(json.dumps(row._asdict(), ensure_ascii=False) + "\n")
# Leggi
with open("dataset.jsonl") as f:
    dati = [json.loads(riga) for riga in f]

```

scikit-learn — Machine Learning Classico

```

from sklearn.model_selection import train_test_split, cross_val_score,
StratifiedKFold
from sklearn.preprocessing import StandardScaler, MinMaxScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    roc_auc_score, average_precision_score, f1_score,
    mean_squared_error, r2_score
)

# --- Split ---
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# --- Preprocessing ---
scaler = StandardScaler()
X_tr_s = scaler.fit_transform(X_tr) # fit + transform
X_te_s = scaler.transform(X_te) # SOLO transform (mai fit sul test)

# --- Pipeline completa ---
preprocessor = ColumnTransformer([
    ("num", StandardScaler(), colonne_numeriche),
    ("cat", OneHotEncoder(drop="first"), colonne_categoriche),
])
pipeline = Pipeline([
    ("prep", preprocessor),
    ("model", RandomForestClassifier(n_estimators=100)),
])
pipeline.fit(X_tr, y_tr)
pipeline.predict(X_te)

```

```

# --- Modelli principali ---
from sklearn.linear_model import LogisticRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.cluster import KMeans
import xgboost as xgb

# Classificatori
lr = LogisticRegression(C=1.0, class_weight="balanced", max_iter=1000)
rf = RandomForestClassifier(n_estimators=200, n_jobs=-1, random_state=42)
gb = GradientBoostingClassifier(n_estimators=200, learning_rate=0.1)
svm = SVC(kernel="rbf", probability=True)
xgbm = xgb.XGBClassifier(n_estimators=200, eval_metric="auc", verbosity=0)

# Regressori
ridge = Ridge(alpha=1.0)
lasso = Lasso(alpha=0.1)

# Clustering
km = KMeans(n_clusters=3, random_state=42, n_init=10)
km.fit(X); km.labels_; km.inertia_

# --- Cross-validation ---
scores = cross_val_score(
    modello, X, y,
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    scoring="roc_auc",
    n_jobs=-1
)
print(f"AUC: {scores.mean():.4f} ± {scores.std():.4f}")

# --- Metriche ---
print(classification_report(y_te, y_pred))
print(f"AUC-ROC: {roc_auc_score(y_te, y_prob):.4f}")
print(f"AUC-PR: {average_precision_score(y_te, y_prob):.4f}")
cm = confusion_matrix(y_te, y_pred)

```

PyTorch — Deep Learning

```

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader, TensorDataset

# --- Device setup ---
device = "cuda" if torch.cuda.is_available() else "cpu"

# --- Tensor ---
t = torch.tensor([1.0, 2.0, 3.0])
t = torch.zeros(3, 4)
t = torch.randn(3, 4)
t = t.to(device)
t.shape; t.dtype; t.device

```

```

# Conversioni
t.numpy()                # tensor → numpy (solo CPU)
torch.from_numpy(arr)    # numpy → tensor
t.item()                 # tensor scalare → Python float
t.detach()               # stacca dal grafo computazionale

# Operazioni
t @ t.T                  # moltiplicazione matriciale
t.sum(dim=0)             # somma per colonna
t.mean(dim=1)            # media per riga
t.argmax(dim=1)          # indice del massimo per riga
F.softmax(t, dim=-1)     # softmax sull'ultima dimensione
F.relu(t)                # ReLU

# --- Dataset personalizzato ---
class MioDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.long)
    def __len__(self): return len(self.X)
    def __getitem__(self, i): return self.X[i], self.y[i]

loader = DataLoader(MioDataset(X, y), batch_size=32, shuffle=True)

# --- Architettura ---
class MiaRete(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(10, 64),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(64, 2)
        )
    def forward(self, x):
        return self.net(x)

modello = MiaRete().to(device)

# Parametri
sum(p.numel() for p in modello.parameters())          # totale
sum(p.numel() for p in modello.parameters() if p.requires_grad) # addestrabili

# --- Ottimizzatori ---
optim_adam = torch.optim.Adam(modello.parameters(), lr=1e-3)
optim_adamw = torch.optim.AdamW(modello.parameters(), lr=1e-3, weight_decay=0.01)
optim_sgd = torch.optim.SGD(modello.parameters(), lr=0.01, momentum=0.9)

# --- Scheduler ---
sched = torch.optim.lr_scheduler.CosineAnnealingLR(optim_adamw, T_max=100)
sched = torch.optim.lr_scheduler.OneCycleLR(optim_adamw, max_lr=1e-3,
total_steps=1000)
sched = torch.optim.lr_scheduler.ReduceLROnPlateau(optim_adamw, patience=5)

# --- Loss functions ---
loss_ce = nn.CrossEntropyLoss()          # classificazione multiclasse

```

```

loss_bce = nn.BCELoss()                # classificazione binaria (dopo sigmoid)
loss_mse = nn.MSELoss()                 # regressione

# --- Training loop ---
def train_one_epoch(modello, loader, optimizer, loss_fn, device):
    modello.train()
    total_loss = 0
    for X_b, y_b in loader:
        X_b, y_b = X_b.to(device), y_b.to(device)
        optimizer.zero_grad()
        pred = modello(X_b)
        loss = loss_fn(pred, y_b)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(modello.parameters(), max_norm=1.0)
        optimizer.step()
        total_loss += loss.item()
    return total_loss / len(loader)

# --- Valutazione ---
def evaluate(modello, loader, device):
    modello.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for X_b, y_b in loader:
            X_b, y_b = X_b.to(device), y_b.to(device)
            pred = modello(X_b)
            correct += (pred.argmax(1) == y_b).sum().item()
            total += len(y_b)
    return correct / total

# --- Salvataggio e caricamento ---
torch.save(modello.state_dict(), "modello.pth")

modello_caricato = MiaRete().to(device)
modello_caricato.load_state_dict(torch.load("modello.pth", map_location=device))
modello_caricato.eval()

# --- GPU monitoring ---
torch.cuda.memory_allocated() / 1e9    # VRAM usata (GB)
torch.cuda.memory_reserved() / 1e9     # VRAM riservata (GB)
torch.cuda.empty_cache()                # libera cache GPU

```

HuggingFace Transformers

```

from transformers import (
    AutoTokenizer, AutoModelForCausalLM, AutoModelForSequenceClassification,
    TrainingArguments, Trainer
)
from datasets import Dataset, load_dataset
from peft import LoraConfig, get_peft_model, TaskType, PeftModel
import torch

# --- Tokenizer ---
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Meta-Llama-3.1-8B-Instruct")

```

```

# Tokenizza un testo
tokens = tokenizer("Ciao mondo!", return_tensors="pt")
tokens["input_ids"].shape           # (1, N_token)
tokens["attention_mask"]            # 1 = token reale, 0 = padding

# Chat template
messages = [
    {"role": "system", "content": "Sei un assistente utile."},
    {"role": "user", "content": "Come stai?"}
]
testo = tokenizer.apply_chat_template(messages, tokenize=False,
add_generation_prompt=True)

# Batch con padding
tokens = tokenizer(
    ["Testo uno", "Testo due più lungo"],
    padding=True, truncation=True, max_length=512, return_tensors="pt"
)

# --- Caricamento modello ---
modello = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Meta-Llama-3.1-8B-Instruct",
    torch_dtype=torch.bfloat16,
    device_map="auto"           # distribuisce automaticamente su GPU disponibili
)

# Caricamento in 4-bit (QLoRA)
from transformers import BitsAndBytesConfig
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
)
modello_4bit = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Meta-Llama-3.1-8B-Instruct",
    quantization_config=bnb_config,
    device_map="auto"
)

# --- Generazione testo ---
inputs = tokenizer("Il machine learning è", return_tensors="pt").to(device)
with torch.no_grad():
    output = modello.generate(
        **inputs,
        max_new_tokens=100,
        temperature=0.7,
        do_sample=True,
        top_p=0.9,
        repetition_penalty=1.1
    )
testo = tokenizer.decode(output[0][inputs["input_ids"].shape[1]:],
skip_special_tokens=True)

# --- LoRA config ---
lora_config = LoraConfig(
    r=16,
    lora_alpha=16,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"],

```



```

    lora_dropout=0.05,
    bias="none",
    task_type=TaskType.CAUSAL_LM
)
modello_lora = get_peft_model(modello, lora_config)
modello_lora.print_trainable_parameters()

# --- Dataset HuggingFace ---
dataset = load_dataset("imdb") # dataset pubblico
dataset["train"][0] # primo esempio
dataset["train"].filter(lambda x: len(x["text"]) > 100) # filtra
dataset["train"].map(lambda x: {"tokens": tokenizer(x["text"])}) # tokenizza

# --- Salva e carica adapter LoRA ---
modello_lora.save_pretrained("./adapter_lora")
tokenizer.save_pretrained("./adapter_lora")

# Ricarica
modello_base = AutoModelForCausalLM.from_pretrained("...", load_in_4bit=True)
modello_ft = PeftModel.from_pretrained(modello_base, "./adapter_lora")

# Merge adapter nei pesi base
modello_merged = modello_lora.merge_and_unload()
modello_merged.save_pretrained("./modello_finale")

```

Anthropic API

```

import anthropic
import os

client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

# --- Chiamata base ---
risposta = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Spiegami cos'è il RAG."}]
)
testo = risposta.content[0].text
print(f"Tokens usati: {risposta.usage.input_tokens} input,
{risposta.usage.output_tokens} output")

# --- Con system prompt ---
risposta = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    system="Sei un esperto di machine learning. Rispondi in italiano.",
    messages=[{"role": "user", "content": "Cos'è l'overfitting?"}]
)

# --- Conversazione multi-turno ---
storia = []
def chat(messaggio: str) -> str:
    storia.append({"role": "user", "content": messaggio})
    r = client.messages.create(

```

```

        model="claude-sonnet-4-20250514",
        max_tokens=1024,
        system="Sei un assistente utile.",
        messages=storia
    )
    risposta_testo = r.content[0].text
    storia.append({"role": "assistant", "content": risposta_testo})
    return risposta_testo

# --- Tool use ---
tools = [{
    "name": "calcola",
    "description": "Esegue un calcolo matematico",
    "input_schema": {
        "type": "object",
        "properties": {
            "espressione": {"type": "string", "description": "Espressione da calcolare"}
        },
        "required": ["espressione"]
    }
}]

risposta = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    tools=tools,
    messages=[{"role": "user", "content": "Quanto fa 127 × 456?"}]
)

# Processa la risposta del tool
for blocco in risposta.content:
    if blocco.type == "tool_use":
        print(f"Tool: {blocco.name}, Input: {blocco.input}")

# --- Output strutturato (JSON garantito) ---
risposta = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=256,
    messages=[
        {"role": "user", "content": "Estrai nome e email da: 'Ciao, sono Marco Rossi, marco@test.it'"},
        {"role": "assistant", "content": "{}"} # prefill
    ]
)

import json
dati = json.loads('{"' + risposta.content[0].text)

# --- Streaming ---
with client.messages.stream(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Scrivi una poesia sull'IA."}]
) as stream:
    for testo in stream.text_stream:
        print(testo, end="", flush=True)

# --- Modelli disponibili (2026) ---
# claude-opus-4-6 → massima capacità

```

```
# claude-sonnet-4-20250514 → bilanciamento qualità/velocità
# claude-haiku-4-5-20251001 → veloce ed economico (guardrail, classificatori)
```

Snippet Utili — Vari

```
# === METRICHE DI SIMILARITÀ ===
import numpy as np

def similarita_coseno(a: np.ndarray, b: np.ndarray) -> float:
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b) + 1e-8))

# === CHUNKING SEMPLICE ===
def chunking(testo: str, dim: int = 500, overlap: int = 100) -> list[str]:
    parole = testo.split()
    chunks = []
    i = 0
    while i < len(parole):
        chunk = " ".join(parole[i:i + dim])
        chunks.append(chunk)
        i += dim - overlap
    return chunks

# === SOFTMAX ===
def softmax(logits: np.ndarray) -> np.ndarray:
    e = np.exp(logits - logits.max())
    return e / e.sum()

# === GRADIENT DESCENT MANUALE ===
def gradient_descent(f, grad_f, x0, lr=0.01, n_steps=100):
    x = x0
    for _ in range(n_steps):
        x = x - lr * grad_f(x)
    return x

# === SEED GLOBALE ===
import random, numpy as np, torch
def set_seed(seed: int = 42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
set_seed(42)

# === TIMER DI CONTESTO ===
import time
from contextlib import contextmanager

@contextmanager
def timer(nome: str = ""):
    t0 = time.time()
    yield
    print(f"{nome}: {time.time() - t0:.2f}s")

with timer("training"):
```

```

    modello.fit(X, y)

# === PROGRESS BAR ===
from tqdm import tqdm
for batch in tqdm(loader, desc="Training"):
    pass

# === SALVA E CARICA JSON/PICKLE ===
import json, pickle

# JSON (leggibile)
with open("config.json", "w") as f:
    json.dump({"lr": 1e-3, "epochs": 100}, f)
with open("config.json") as f:
    config = json.load(f)

# Pickle (qualsiasi oggetto Python)
with open("scaler.pkl", "wb") as f:
    pickle.dump(scaler, f)
with open("scaler.pkl", "rb") as f:
    scaler = pickle.load(f)

# === GPU MEMORY INFO ===
if torch.cuda.is_available():
    print(f"VRAM totale: {torch.cuda.get_device_properties(0).total_memory /
1e9:.1f} GB")
    print(f"VRAM usata: {torch.cuda.memory_allocated() / 1e9:.2f} GB")
    print(f"VRAM libera: {(torch.cuda.get_device_properties(0).total_memory -
torch.cuda.memory_allocated()) / 1e9:.2f} GB")

```

Versione librerie di riferimento (2026): *numpy*≥1.26, *pandas*≥2.1, *scikit-learn*≥1.4, *torch*≥2.3, *transformers*≥4.40, *anthropic*≥0.28, *peft*≥0.10

Appendice C

Risorse Aggiornate

Corsi · Paper fondamentali · Community · Strumenti

Ogni risorsa qui elencata è stata selezionata con un criterio preciso: accessibile, aggiornata, e con il miglior rapporto qualità/tempo. Non è un elenco esaustivo — è una mappa.

Corsi e Formazione

Per iniziare (se vuoi consolidare le basi)

fast.ai — Practical Deep Learning for Coders fast.ai/course Il corso più pratico esistente. Parte dal codice funzionante e spiega la teoria solo dopo. Ottimo complemento a questa guida per chi vuole più pratica immediata. Gratuito.

3Blue1Brown — Neural Networks (YouTube) youtube.com/3blue1brown Playlist "Neural Networks" — le visualizzazioni più belle della matematica dietro le reti neurali. Consigliato per capire intuitivamente backpropagation e attenzione. Gratuito.

Andrej Karpathy — Neural Networks: Zero to Hero github.com/karpathy/nn-zero-to-hero Serie di video in cui Karpathy costruisce un GPT da zero in Python puro. Denso, tecnico, eccellente. Il video "Let's build GPT" è un must-watch dopo aver completato il Volume 2. Gratuito.

Per approfondire (post-guida)

Stanford CS224N — Natural Language Processing with Deep Learning web.stanford.edu/class/cs224n Il corso universitario più completo su NLP con Transformer. Slide, video e assignments disponibili gratuitamente online.

DeepLearning.AI — Specialization di Andrew Ng deeplearning.ai La specializzazione in 5 corsi è ancora il percorso strutturato più completo per chi vuole fondamenta solide con certificazione. A pagamento (con audit gratuito).

Hugging Face Course huggingface.co/learn Corso ufficiale HuggingFace su Transformer, fine-tuning e deployment. Direttamente aggiornato dal team. Gratuito.

Paper Fondamentali

Questi sono i paper che ogni professionista del settore dovrebbe aver letto — almeno l'abstract e le figure. Ordinati cronologicamente.

Architetture

"Attention Is All You Need" — Vaswani et al., 2017 arxiv.org/abs/1706.03762 Il paper del Transformer. L'architettura che sta alla base di tutto. Leggere le sezioni 3 (Model Architecture) e 4 (Why Self-Attention) è sufficiente.

"Deep Residual Learning for Image Recognition" — He et al., 2015 arxiv.org/abs/1512.03385 Le skip connections di ResNet. Tre pagine che hanno rivoluzionato il training di reti profonde.

"BERT: Pre-training of Deep Bidirectional Transformers" — Devlin et al., 2018 arxiv.org/abs/1810.04805 Il primo modello a dimostrare il potere del pre-training bidirezionale su larga scala.

"Language Models are Few-Shot Learners" — Brown et al., 2020 arxiv.org/abs/2005.14165 Il paper di GPT-3. Introduce il concetto di few-shot learning emergente dalla scala.

Scaling e LLM

"Scaling Laws for Neural Language Models" — Kaplan et al., 2020 arxiv.org/abs/2001.08361 Le prime leggi di scala: come la qualità cresce in modo prevedibile con parametri, dati e compute.

"Training Compute-Optimal Large Language Models" (Chinchilla) — Hoffmann et al., 2022 arxiv.org/abs/2203.15556 La revisione delle leggi di scala: i modelli erano sottoaddestranti rispetto alla loro dimensione.

"Llama 2: Open Foundation and Fine-Tuned Chat Models" — Touvron et al., 2023 arxiv.org/abs/2307.09288 Architettura e training di LLaMA 2. Ottimo per capire come si costruisce un LLM open source moderno.

Allineamento e RLHF

"Training language models to follow instructions with human feedback" — Ouyang et al., 2022 arxiv.org/abs/2203.02155 Il paper di InstructGPT: come RLHF trasforma un base model in un assistente utile.

"Direct Preference Optimization: Your Language Model is Secretly a Reward Model" — Rafailov et al., 2023 arxiv.org/abs/2305.18290 DPO: come ottimizzare il modello direttamente sulle preferenze umane senza un reward model separato.

Fine-tuning Efficiente

"LoRA: Low-Rank Adaptation of Large Language Models" — Hu et al., 2021 arxiv.org/abs/2106.09685 Il paper che ha reso il fine-tuning accessibile. Algoritmo chiaro, esperimenti convincenti.

"QLoRA: Efficient Finetuning of Quantized LLMs" — Dettmers et al., 2023 arxiv.org/abs/2305.14314 Come fare fine-tuning su GPU consumer con quantizzazione 4-bit. Fondamentale per il Cap. 18.

RAG e Agenti

"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" — Lewis et al., 2020 arxiv.org/abs/2005.11401 Il paper originale del RAG. Introduce l'idea di combinare retrieval e generazione.

"ReAct: Synergizing Reasoning and Acting in Language Models" — Yao et al., 2022 arxiv.org/abs/2210.03629 Il pattern ReAct per gli agenti. Ragionamento esplicito interallacciato con azioni.

Sicurezza

"Universal and Transferable Adversarial Attacks on Aligned Language Models" — Zou et al., 2023 arxiv.org/abs/2307.15043 Dimostra che i jailbreak possono essere trovati automaticamente con ottimizzazione. Fondamentale per il Cap. 21.

"Prompt Injection Attacks Against GPT-3" — Perez & Ribeiro, 2022 arxiv.org/abs/2302.12173 La prima analisi sistematica degli attacchi di prompt injection.

Community e Forum

Hugging Face Forum discuss.huggingface.co Il forum più attivo per domande tecniche su Transformer, PEFT, fine-tuning. Risposto spesso dagli stessi autori delle librerie.

r/MachineLearning reddit.com/r/MachineLearning Discussioni su paper nuovi, risultati di ricerca, trend del settore. Segnale/rumore decente.

r/LocalLLaMA reddit.com/r/LocalLLaMA Community dedicata ai modelli open source locali. Benchmarks, guide pratiche, discussioni su hardware.

Discord di fast.ai discord.gg/fastai Community molto attiva, orientata ai praticanti più che ai teorici.

Twitter/X: account da seguire

- [@karpathy](https://twitter.com/karpathy) — ex OpenAI, guide tecniche eccellenti
 - [@ylecun](https://twitter.com/ylecun) — Chief AI Scientist di Meta, prospettiva accademica
 - [@DrJimFan](https://twitter.com/DrJimFan) — ricercatore NVIDIA, focus su agenti e robotica
 - [@NousResearch](https://twitter.com/NousResearch) — team open source su LLM e fine-tuning
-

Newsletter e Blog Tecnici

The Batch — DeepLearning.AI deeplearning.ai/the-batch Newsletter settimanale di Andrew Ng. Segnale alto, rumore basso. Gratuita.

Import AI — Jack Clark jack-clark.net Una delle newsletter più dense del settore. Per chi vuole seguire la frontiera della ricerca.

Lil'Log — Lilian Weng (OpenAI) lilianweng.github.io Post tecnici approfonditi su argomenti come RLHF, agenti, sicurezza. Qualità eccezionale.

Sebastian Raschka's Blog sebastianraschka.com/blog Guide pratiche su fine-tuning, LLM applicati, con molto codice. Ottimo per practitioners.

Eugene Yan eugeneyan.com Focus su ML in produzione: sistema di raccomandazione, valutazione LLM, MLOps.

Strumenti e Piattaforme

Ambienti di sviluppo

Google Colab colab.research.google.com GPU gratuita (T4), zero configurazione. Punto di partenza consigliato.

Kaggle Notebooks kaggle.com/code GPU gratuita, dataset integrati, competizioni. Ottimo per praticare con problemi reali.

Lightning.ai lightning.ai Cloud con GPU, storage persistente, e l'ecosistema PyTorch Lightning integrato. Buona alternativa a Colab per progetti più seri.

Modelli e API

Anthropic API docs.anthropic.com Claude Sonnet e Haiku. Eccellente documentazione, cookbook con esempi pratici.

OpenAI API platform.openai.com GPT-4o e fine-tuning via API. Ampio ecosistema di integrazioni.

HuggingFace Hub huggingface.co/models Il maggior repository di modelli open source. LLaMA, Mistral, Qwen, e migliaia di modelli fine-tuned.

Ollama ollama.com Il modo più semplice per far girare LLM open source in locale. Un comando per scaricare e avviare LLaMA, Mistral, Phi.

MLOps e Deployment

Weights & Biases wandb.ai Tracking di esperimenti, visualizzazione delle curve di training, confronto run. Gratuito per uso personale.

MLflow mlflow.org Open source, self-hosted. Alternativa a W&B senza dipendenze cloud.

Docker Hub hub.docker.com Repository di immagini Docker. Cerca [vllm](#), [chroma](#), [ollama](#) per trovare immagini pronte.

Sicurezza e Valutazione

LangSmith (LangChain) smith.langchain.com Tracing e valutazione per sistemi LLM. Utile per debugging di pipeline RAG e agenti.

OWASP LLM Top 10 owasp.org/www-project-top-10-for-large-language-model-applications Il riferimento per la sicurezza dei sistemi LLM. Aggiornato annualmente.

Garak github.com/leondz/garak Framework open source per il red-teaming automatizzato di LLM.

Datasets di Riferimento

| Dataset | Task | Dimensione | Dove trovarlo | |---|---|---|---| | MNIST | Classificazione immagini | 70K immagini | torchvision.datasets.MNIST | | CIFAR-10 | Classificazione immagini | 60K immagini | torchvision.datasets.CIFAR10 | | ImageNet | Classificazione immagini | 1.4M immagini | image-net.org (modelli pre-addestrati su HuggingFace) | | IMDB | Sentiment analysis | 50K

recensioni | `datasets.load_dataset("imdb")` | | MMLU | Conoscenza generale LLM | 14K domande
| `datasets.load_dataset("cais/mmlu")` | | Alpaca | Instruction following | 52K esempi |
`datasets.load_dataset("tatsu-lab/alpaca")` | | OpenHermes | Fine-tuning conversazionale | 1M
esempi | HuggingFace Hub | | CC-News | News in italiano | variabile | HuggingFace Hub: `cc_news` |
| Titanic | Classificazione tabellare | 891 righe | `sklearn.datasets` / Kaggle | | Credit Card Fraud |
Rilevamento anomalie | 284K righe | Kaggle: `mlg-ulb/creditcardfraud` |

Risorse verificate: Edizione 2026. I link online possono cambiare — usa i nomi per cercare le versioni più recenti.

Appendice D

Roadmap di Studio Consigliata

Con milestone verificabili

Questa roadmap è pensata per uno sviluppatore che studia part-time — circa 10 ore a settimana. I tempi indicati sono stime medie: c'è chi li dimezza e chi li raddoppia, ed entrambi vanno bene. L'importante è non saltare le milestone: sono i checkpoint che confermano che hai davvero capito, non solo letto.

Come usare questa roadmap

Ogni fase ha:

- **Obiettivi di apprendimento:** cosa imparerai
- **Milestone verificabili:** esercizi concreti che dimostrano la comprensione
- **Tempo stimato:** per chi studia ~10 ore/settimana
- **Segnale di prontezza:** come sapere se sei pronto per la fase successiva

Non andare alla fase successiva finché non hai completato almeno il 70% delle milestone della fase corrente. È più efficace consolidare bene che procedere velocemente con lacune.

FASE 1 — Fondamenta del ML

Corrispondente al Volume 1, Capitoli 1–7 **Tempo stimato: 6–8 settimane**

Obiettivi

- Capire la differenza tra programmazione tradizionale e ML
- Saper preparare un dataset per il training (split, preprocessing, encoding)
- Saper addestrare e valutare un modello di ML classico
- Capire cosa succede matematicamente durante il training

Milestone verificabili

M1.1 — Setup e primo codice

- ☐ Google Colab configurato con GPU T4
- ☐ Esecuzione senza errori degli snippet del Cap. 1 (verifica installazione)
- ☐ Completato l'Esercizio 2 del Cap. 1 (classificazione spam con regole manuali)

M1.2 — Dati

- ☐ Esplorazione del dataset Titanic con EDA completa (Cap. 2, Esercizio 1)

- ☐ Pipeline di preprocessing senza data leakage (Cap. 2, Esercizio 2)
- ☐ Sai rispondere: perché `scaler.fit()` va fatto SOLO sul training set?

M1.3 — Matematica

- ☐ Implementato gradient descent manuale su $f(w) = (w-7)^2$ (Cap. 3, Esercizio 2)
- ☐ Sai spiegare cosa misura la cross-entropy loss con parole tue
- ☐ Completato l'Esercizio 3 del Cap. 3 (cross-entropy a mano)

M1.4 — Training loop

- ☐ Training loop da zero senza `nn.Module` (Cap. 4, Esercizio 1)
- ☐ Hai osservato l'overfitting empiricamente e applicato dropout (Cap. 4, Esercizio 2)
- ☐ Sai leggere una curva di training e diagnosticare: overfitting, underfitting, learning rate errato

M1.5 — Reti neurali

- ☐ Forward pass manuale su MLP 2-layer (Cap. 5, Esercizio 1)
- ☐ Backpropagation manuale su problema XOR (Cap. 5, Esercizio 2)
- ☐ Sai spiegare perché ReLU ha sostituito Sigmoid nei layer nascosti

M1.6 — ML classico

- ☐ Confronto sistematico di 5 modelli sullo stesso dataset (Cap. 6, Esercizio 1)
- ☐ XGBoost con hyperparameter tuning via RandomizedSearchCV (Cap. 6, Esercizio 2)
- ☐ Sai scegliere il modello giusto per un problema tabellare generico

M1.7 — Ottimizzatori

- ☐ Confronto empirico di 4 ottimizzatori (Cap. 7, Esercizio 1)
- ☐ Implementazione training loop completo con early stopping e scheduler (Cap. 7)

M1.8 — Progetto finale Vol. 1

- ☐ Pipeline ML completa sul dataset frodi: dati → preprocessing → confronto modelli → test set
- ☐ Report con AUC-PR, confusion matrix, analisi del trade-off precision/recall
- ☐ Sai spiegare perché AUC-PR è più onesta di AUC-ROC su dataset sbilanciati

Segnale di prontezza per la Fase 2

Sei pronto se puoi costruire autonomamente una pipeline ML completa su un nuovo dataset tabellare in meno di 3 ore, senza guardare il codice della guida.

FASE 2 — Deep Learning e LLM

Corrispondente al Volume 2, Capitoli 8–17 **Tempo stimato: 10–14 settimane**

Obiettivi

- Padroneggiare PyTorch per il deep learning
- Capire e saper usare CNN, RNN/LSTM e Transformer
- Costruire un chatbot con API LLM

- Costruire un sistema RAG funzionante
- Implementare un agente con tool use

Milestone verificabili

M2.1 — PyTorch

- ☐ Classificatore MNIST con CNN (Cap. 8–9, >98% accuracy)
- ☐ Transfer learning con ResNet su CIFAR-10 o STL-10 (Cap. 9, Esercizio)
- ☐ Sai muovere tensor tra CPU e GPU e spiegare perché i device devono coincidere

M2.2 — Sequenze

- ☐ LSTM per predizione di serie temporale (Cap. 10, Esercizio)
- ☐ Classificatore di sentiment con GRU bidirezionale (Cap. 10, Esercizio)
- ☐ Sai spiegare il vanishing gradient e perché LSTM lo mitiga

M2.3 — Transformer

- ☐ Mini-Transformer costruito da zero per classificazione testo (Cap. 11, >85% su IMDB)
- ☐ Implementazione della causal mask per generazione autoregressiva (Cap. 11, Esercizio)
- ☐ Sai spiegare Query, Key, Value con un'analogia del mondo reale

M2.4 — LLM e Prompt Engineering

- ☐ Chatbot con memoria persistente e system prompt personalizzato (Cap. 12)
- ☐ Confronto empirico zero-shot vs few-shot vs CoT sullo stesso task (Cap. 13, Esercizio)
- ☐ LLM-as-Judge funzionante su un tuo dataset di valutazione (Cap. 13)

M2.5 — RAG

- ☐ Sistema Q&A su almeno 2 PDF tuoi reali (Cap. 14, Esercizio)
- ☐ Confronto di due configurazioni di chunking con metriche quantitative (Cap. 14, Esercizio)
- ☐ Sai spiegare quando usare re-ranking vs hybrid search

M2.6 — Agenti

- ☐ Agente con ricerca web reale che risponde con fonti (Cap. 15, Esercizio)
- ☐ Agente con memoria persistente e limite di budget token (Cap. 15, Esercizio)
- ☐ Hai implementato almeno una protezione contro i loop infiniti

M2.7 — MCP

- ☐ Server MCP con almeno 4 tool, 1 resource, 1 prompt (Cap. 16, Esercizio)
- ☐ Server connesso a Claude Desktop e testato in conversazione reale
- ☐ Sai spiegare la differenza tra Tool, Resource e Prompt in MCP

M2.8 — Hardware

- ☐ Hai profilato un modello e misurato l'uso di VRAM (Cap. 17, Esercizio)
- ☐ Sai scegliere tra FP32, BF16, INT8, INT4 in base al caso d'uso

M2.9 — Progetto finale Vol. 2

- ☐ Chatbot RAG con agente integrato: risponde a domande, cerca sul web, usa un MCP server

- ☐ Sistema funzionante end-to-end con almeno 20 documenti nel knowledge base
- ☐ Hai testato con almeno 30 domande e documentato la qualità delle risposte

Segnale di prontezza per la Fase 3

Sei pronto se puoi costruire e deployare localmente un chatbot RAG con agente da zero in meno di una giornata lavorativa.

FASE 3 — Produzione e Sicurezza

Corrispondente al Volume 3, Capitoli 18–24 **Tempo stimato: 8–12 settimane**

Obiettivi

- Fine-tuning con LoRA/QLoRA su dati propri
- Sistema di valutazione automatizzata con LLM-as-Judge
- Deploy containerizzato con monitoring
- Comprensione degli attacchi e implementazione delle difese
- Progetto finale professionale completo

Milestone verificabili

M3.1 — Fine-tuning

- ☐ Fine-tuning LoRA di LLaMA 8B su almeno 200 esempi propri (Cap. 18, Esercizio)
- ☐ Confronto base vs fine-tuned con metriche quantitative
- ☐ Hai testato il catastrophic forgetting e documentato i risultati
- ☐ Sai spiegare la differenza tra usare fine-tuning per "comportamenti" vs "fatti"

M3.2 — Valutazione

- ☐ Eval set di almeno 60 esempi con 3 categorie (core, edge, trabocchetto)
- ☐ Pipeline LLM-as-Judge automatizzata con report JSON (Cap. 19, Esercizio)
- ☐ Test sistematici di hallucination su 20 domande trabocchetto
- ☐ Almeno un ciclo di miglioramento documentato con metriche prima/dopo

M3.3 — Deployment

- ☐ Chatbot RAG containerizzato con Docker Compose (Cap. 20, Esercizio)
- ☐ Dashboard Grafana con almeno 3 metriche (latenza, traffico, errori)
- ☐ Load test con Locust su almeno 10 utenti simultanei
- ☐ Sai interpretare le 4 golden signal e diagnosticare un problema di performance

M3.4 — Sicurezza Attacchi

- ☐ Report di red-teaming con almeno 7 test eseguiti (Cap. 21, Esercizio)
- ☐ Hai trovato e documentato almeno 2 vulnerabilità reali nel tuo sistema
- ☐ Sai spiegare la differenza tra injection diretta e indiretta con esempi concreti

M3.5 — Sicurezza Difese

- ☐ `InputValidator` implementato e integrato nell'endpoint (Cap. 22, Esercizio)

- [] System prompt testato contro 8 attacchi standard con punteggio di robustezza
- [] Rate limiting e security monitoring attivi in produzione
- [] Il sistema supera $\geq 75\%$ degli attacchi del red-team precedente

M3.6 — IA nel lavoro

- [] Workflow IA per un task lavorativo reale tuo (Cap. 23, Esercizio)
- [] Accuracy $\geq 85\%$ su dataset storico
- [] WorkflowLogger attivo con statistiche settimanali

M3.7 — Progetto finale

- [] Assistente verticale su un dominio reale — tutte le 6 fasi completate
- [] Pass rate $\geq 80\%$ sull'eval set
- [] Docker Compose funzionante con `docker compose up -d`
- [] Report di red-teaming con vulnerabilità trovate e risolte
- [] README che permette setup in meno di 5 comandi
- [] Tutti i punti della checklist di consegna del Cap. 24 verificati

Segnale di completamento

Hai completato la roadmap se riesci a descrivere a un collega sviluppatore, senza note, il ciclo completo: raccolta dati → modello → valutazione → deploy → sicurezza → iterazione.

Dopo la Guida — Dove andare

Se vuoi specializzarti in NLP/LLM

- Studia i paper fondamentali dell'Appendice C (in ordine cronologico)
- Partecipa a una competizione Kaggle NLP
- Contribuisci a un progetto open source su HuggingFace
- Approfondisci: Constitutional AI, interpretabilità dei modelli, watermarking

Se vuoi specializzarti in Computer Vision

- Stanford CS231n (corso online gratuito)
- Implementa da zero una delle architetture più recenti (EfficientNet, DINO, SAM)
- Studia la diffusion: Denoising Diffusion Probabilistic Models

Se vuoi specializzarti in MLOps/Production

- Approfondisci Kubernetes per il deployment distribuito
- Studia Apache Kafka per il data streaming
- Implementa un sistema di A/B testing per modelli ML
- Approfondisci: model distillation, quantizzazione avanzata, ONNX

Se vuoi specializzarti in Sicurezza IA

- Segui il blog di Anthropic sulla sicurezza
- Studia l'interpretabilità meccanicistica (Circuits work di Anthropic)
- Approfondisci: adversarial robustness, membership inference attacks, differential privacy

Se vuoi lavorare nell'industria

- Costruisci un portfolio di 3 progetti completi su GitHub con README chiari
- Contribuisci a HuggingFace, LangChain o simili con piccole pull request
- Scrivi post tecnici su ciò che hai imparato — insegnare consolida la comprensione
- Partecipa alle community dell'Appendice C

Tabella di Autovalutazione

Usala ogni mese per monitorare i tuoi progressi. "✓" = lo faccio autonomamente, "△" = lo faccio con aiuto, "X" = ancora da imparare.

Competenza	Autovalutazione	Data	--- --- ---	Preparare un dataset ML (split, encoding, normalizzazione)		Scegliere il modello giusto per un problema tabellare		Costruire e addestrare una CNN in PyTorch		Spiegare il meccanismo di attenzione		Costruire un sistema RAG funzionante		Costruire un agente con tool use		Fare fine-tuning LoRA su GPU consumer		Costruire un eval set e misurare la qualità con LLM-as-Judge		Deployare un servizio ML con Docker		Eseguire un red-team e documentare le vulnerabilità		Implementare input validation e guardrail		Costruire un workflow IA affidabile per un task reale	
------------	-----------------	------	-------------	--	--	---	--	---	--	--------------------------------------	--	--------------------------------------	--	----------------------------------	--	---------------------------------------	--	--	--	-------------------------------------	--	---	--	---	--	---	--

Roadmap aggiornata: Edizione 2026. I tempi indicati presuppongono ~10 ore/settimana di studio attivo (con codice, non solo lettura).

Appendice E

Soluzioni Commentate degli Esercizi Pratici

Volume 1 — Fondamenta

Come usare queste soluzioni: prova sempre prima da solo. Confronta il tuo approccio con la soluzione solo dopo aver ottenuto un risultato — anche parziale, anche sbagliato. Capire perché il tuo approccio differisce da quello proposto vale più di ricopiare la soluzione corretta.

Capitolo 1 — Cos'è davvero l'Intelligenza Artificiale

Esercizio 1 — Riflessione guidata

Domanda: quale approccio useresti per ciascun problema?

Problema 1 — Trovare il massimo di un array → Regole scritte. Non serve ML: la logica è completamente definibile con una formula (`max(array)`). Il ML sarebbe uno strumento sbagliato: introduce incertezza dove non ne hai bisogno, richiede dati di training, e non può fare meglio di un algoritmo deterministico.

Problema 2 — Riconoscere se il soggetto sorride → ML (Computer Vision). Non riesci a scrivere regole precise per "cosa costituisce un sorriso" — la variabilità tra persone, angolazioni, illuminazioni è troppo alta. Dati necessari: migliaia di foto etichettate (sorriso / non sorriso), bilanciate per genere, età, etnia per evitare bias.

Problema 3 — Predire l'acquisto entro 24 ore → ML (Classificazione tabellare). Non puoi scrivere regole esplicite perché le interazioni tra feature (tempo sul sito, pagine visitate, storico acquisti, ora del giorno) sono troppo complesse. Dati necessari: log di sessioni storiche con label "ha comprato / non ha comprato", idealmente qualche decina di migliaia di esempi.

Caso in cui ML è tecnicamente applicabile ma sbagliato: Predire se un numero intero è pari o dispari. È tecnicamente un problema di classificazione, e potresti addestrare un modello ML. Ma ha un'unica regola perfetta (`n % 2 == 0`), zero incertezza, zero dati necessari. Usare ML qui sarebbe più lento, meno accurato, e incomprensibile. La regola scritta è ovviamente superiore.

Esercizio 2 — Primo codice

```
import numpy as np

email_features = np.array([
    [120, 2, 3], [45, 0, 1], [800, 15, 42], [200, 1, 5],
    [90, 3, 8], [600, 12, 38], [150, 0, 2], [55, 1, 0],
    [750, 18, 50], [100, 2, 4],
])
```



```

etichette = np.array([0, 0, 1, 0, 0, 1, 0, 0, 1, 0])

email_spam      = email_features[etichette == 1]
email_no_spam   = email_features[etichette == 0]

print("Spam      - lunghezza: {:.0f}, link: {:.1f}, maiuscole: {:.1f}".format(
    email_spam[:,0].mean(), email_spam[:,1].mean(), email_spam[:,2].mean()))
print("No spam - lunghezza: {:.0f}, link: {:.1f}, maiuscole: {:.1f}".format(
    email_no_spam[:,0].mean(), email_no_spam[:,1].mean(), email_no_spam[:,2].mean()))

# --- Soluzione Domanda 2: regole manuali ---
def is_spam_rule(email):
    """
    Regola basata sulle soglie osservate nell'EDA.
    Soglie scelte guardando la differenza tra le medie dei due gruppi.
    """
    lunghezza, n_link, maiuscole = email
    return int(lunghezza > 400 or n_link > 8 or maiuscole > 25)

# Test sulle 10 email
predizioni = np.array([is_spam_rule(e) for e in email_features])
corretti = (predizioni == etichette).sum()
print(f"\nAccuracy regole manuali: {corretti}/{len(etichette)} = {corretti/len(etichette):.0%}")

# Output atteso: 10/10 = 100% su questo piccolo esempio
# (Le soglie sono state calibrate su questi 10 esempi - overfitting garantito)

```

Perché le regole funzionano su 10 esempi ma probabilmente fallirebbero su 1000? Le soglie (400, 8, 25) sono state "viste" guardando i dati. Su dati nuovi, troveresti sicuramente email spam con soli 5 link ma con altri pattern caratteristici, oppure email legittime con 10 link (newsletter). Questo è il limite fondamentale delle regole manuali su problemi reali.

Capitolo 2 — I Dati

Esercizio 1 — Esplorazione Titanic

```

import pandas as pd
import matplotlib.pyplot as plt

url = 'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv'
df = pd.read_csv(url)

# 1. Shape
print(f"Shape: {df.shape}") # (891, 12)

# 2. Valori mancanti
print(df.isnull().sum()[df.isnull().sum() > 0])
# Age: 177 mancanti (20%) - importante, va gestita
# Cabin: 687 mancanti (77%) - praticamente inutile come feature
# Embarked: 2 mancanti - trascurabile

# 3. Tasso di sopravvivenza complessivo
print(f"Sopravvivuti: {df['Survived'].mean():.1%}") # ~38%

```

```

# 4. Per sesso e classe
print(df.groupby('Sex')['Survived'].mean().round(3))
# female: 0.742, male: 0.189 → il sesso è la feature più predittiva

print(df.groupby('Pclass')['Survived'].mean().round(3))
# 1a classe: 0.630, 2a: 0.473, 3a: 0.242

# 5. Distribuzione età
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
df['Age'].hist(bins=30, ax=axes[0], edgecolor='white')
axes[0].set_title('Distribuzione età'); axes[0].set_xlabel('Età')
df.boxplot(column='Age', ax=axes[1])
axes[1].set_title('Boxplot età – outlier visibili oltre i 60 anni')
plt.tight_layout(); plt.show()
# Outlier: alcune persone >70 anni – reali, non errori di dati

# 6. Correlazione con Survived
numeriche = df.select_dtypes(include='number')
corr = numeriche.corr()['Survived'].drop('Survived').sort_values()
print(corr)
# Pclass: -0.338 (classe bassa = meno sopravvissuti)
# Fare: 0.257 (tariffa alta = più sopravvissuti)
# Age: -0.077 (relazione debole)

```

Cosa impari dall'EDA: Il Titanic non è solo "le donne prima" — è una storia di classe sociale. Sesso e Pclass sono le feature più predittive, non l'età. Cabin ha troppi mancanti per essere utile. Age va imputata con la mediana (per classe, non globale — i bambini in prima classe avevano età diverse dai bambini in terza).

Esercizio 2 — Preprocessing completo

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

np.random.seed(42)
n = 300
df = pd.DataFrame({
    'età': np.where(np.random.rand(n) < 0.1, np.nan,
                    np.random.randint(18, 65, n).astype(float)),
    'reddito': np.random.exponential(30000, n) + 15000,
    'città': np.random.choice(['Milano', 'Roma', 'Napoli', 'Torino'], n),
    'anni_cliente': np.random.randint(0, 10, n),
    'ha_comprato': np.random.choice([0, 1], n, p=[0.7, 0.3]),
})

# --- 1. Separa X e y ---
X = df.drop(columns=['ha_comprato'])
y = df['ha_comprato']

# --- 2. Split stratificato ---

```

```

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.20, random_state=42,
stratify=y)
print(f"Train: {len(X_tr)}, Test: {len(X_te)}")
print(f"Churn train: {y_tr.mean():.1%}, test: {y_te.mean():.1%}")

# --- 3. Pipeline di preprocessing ---
# Le colonne sono separate per tipo
num_cols = ['età', 'reddito', 'anni_cliente']
cat_cols = ['città']

# Imputer + Scaler per le numeriche
# Nota: Pipeline concatena le trasformazioni nell'ordine giusto
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')), # fill NaN PRIMA di scalare
    ('scaler', StandardScaler()),
])

cat_pipeline = Pipeline([
    ('encoder', OneHotEncoder(drop='first', sparse_output=False)),
])

# ColumnTransformer applica pipeline diverse a colonne diverse
preprocessor = ColumnTransformer([
    ('num', num_pipeline, num_cols),
    ('cat', cat_pipeline, cat_cols),
])

# --- 4. Fit sul training SOLO, transform su tutti ---
preprocessor.fit(X_tr) # impara mediana, media, std, categorie dal training
X_tr_proc = preprocessor.transform(X_tr)
X_te_proc = preprocessor.transform(X_te)

# --- 5. Verifica: nessun NaN ---
assert not np.isnan(X_tr_proc).any(), "NaN nel training processato!"
assert not np.isnan(X_te_proc).any(), "NaN nel test processato!"
print(f"\nShape finale: train={X_tr_proc.shape}, test={X_te_proc.shape}")
print("Nessun NaN — preprocessing completato correttamente.")

```

Errore comune da evitare: chiamare `preprocessor.fit(X)` sull'intero dataset invece che solo su `X_tr`. Questo fa sì che la mediana di "età" venga calcolata includendo i dati di test — una piccola ma reale forma di data leakage.

Capitoli 3–4 — Matematica e Come Impara una Macchina

Esercizio 3.1 — Geometria dei vettori

```

import numpy as np

alice = np.array([5, 1, 1, 4, 3])
bob   = np.array([4, 1, 2, 5, 2])
carlo = np.array([1, 5, 1, 1, 4])

def similarita_coseno(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

```

```

sim_alice_bob    = similarita_coseno(alice, bob)
sim_alice_carlo  = similarita_coseno(alice, carlo)

print(f"Alice-Bob:    {sim_alice_bob:.3f}")    # ~0.925 – molto simili
print(f"Alice-Carlo: {sim_alice_carlo:.3f}")    # ~0.638 – meno simili

# Risposta: Bob è più simile ad Alice.
# Entrambi preferiscono azione e fantascienza, non amano la commedia.

```

Perché la similarità coseno invece del prodotto scalare puro?

```

# Se Carlo raddoppiasse i suoi voti:
carlo_doppio = carlo * 2 # [2, 10, 2, 2, 8]

# Prodotto scalare grezzo
print(f"Dot(Alice, Carlo):          {np.dot(alice, carlo):.1f}")          # 25
print(f"Dot(Alice, Carlo×2):        {np.dot(alice, carlo_doppio):.1f}")    # 50 ← sembra
più simile

# Similarità coseno (normalizzata)
print(f"Coseno(Alice, Carlo):       {similarita_coseno(alice, carlo):.3f}")    #
0.638
print(f"Coseno(Alice, Carlo×2):     {similarita_coseno(alice, carlo_doppio):.3f}") #
0.638 ← invariante

# La similarità coseno è invariante alla scala: misura la DIREZIONE, non la
magnitudine.
# Carlo che vota tutto doppio non è "più simile" ad Alice – ha solo valutato con
scala diversa.

```

Esercizio 3.2 — Gradient Descent a mano

```

import numpy as np
import matplotlib.pyplot as plt

def f(w): return (w - 7) ** 2
def grad_f(w): return 2 * (w - 7)

# Confronto 4 learning rate
lr_values = [0.001, 0.1, 0.5, 1.1]
fig, axes = plt.subplots(1, 4, figsize=(16, 4))

for ax, lr in zip(axes, lr_values):
    w = 0.0
    storia = [w]
    for _ in range(50):
        w = w - lr * grad_f(w)
        storia.append(w)

    w_range = np.linspace(-2, 16, 200)
    ax.plot(w_range, f(w_range), 'b-', alpha=0.4)
    ax.scatter(storia, [f(wi) for wi in storia], c='red', s=15, zorder=5)
    ax.set_title(f'lr={lr}')
    ax.set_ylim(-1, 55)

```

```
ax.axvline(x=7, color='green', linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```

Cosa osservi:

- `lr=0.001`: converge ma impiega molti step. Dopo 50 step è ancora lontano da $w=7$.
- `lr=0.1`: converge elegantemente in ~25 step.
- `lr=0.5`: converge, con qualche oscillazione iniziale.
- `lr=1.1`: **diverge**. Il passo è così grande che ogni aggiornamento salta sopra il minimo e si allontana. La loss cresce invece di scendere.

Problema con doppio minimo: $f(w) = (w-7)^2 + (w-3)^2$ ha minimo in $w = (7+3)/2 = 5$. Gradient descent lo trova correttamente partendo da qualsiasi punto.

Esercizio 4.1 — Training loop da zero

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
X = np.random.randn(100, 1)
y = 2 * X.ravel() + 1 + np.random.randn(100) * 0.5

w = np.random.randn(1, 1) * 0.1
b = np.zeros((1, 1))
lr = 0.01
storia_loss = []

for epoch in range(200):
    y_pred = X @ w + b
    loss = np.mean((y_pred - y.reshape(-1,1)) ** 2)
    storia_loss.append(loss)

    N = len(X)
    dw = (2/N) * X.T @ (y_pred - y.reshape(-1,1))
    db = (2/N) * np.sum(y_pred - y.reshape(-1,1))
    w -= lr * dw
    b -= lr * db

print(f"Peso imparato: {w[0,0]:.3f} (vero: 2.0)")
print(f"Bias imparato: {b[0,0]:.3f} (vero: 1.0)")
# Output atteso: peso≈2.0, bias≈1.0 se il training ha funzionato
```

Effetto del learning rate:

- `lr=0.1`: può oscillare o divergere — dipende dai dati
- `lr=0.001`: converge ma molto lentamente, la loss è ancora alta dopo 200 epoch
- `lr=0.01`: il valore usato nell'esercizio, buon bilanciamento

Capitolo 5 — Il Neurone Artificiale

Esercizio 5.1 — Neurone a mano

```
import numpy as np

w = np.array([0.4, 0.01, 0.2])
b = -3.5
sigmoid = lambda z: 1 / (1 + np.exp(-z))

candidati = np.array([
    [2.0, 95.0, 1], # candidato 1
    [8.0, 70.0, 3], # candidato 2
    [0.5, 110.0, 2], # candidato 3 - neolaureato
    [5.0, 85.0, 2], # candidato 4
])

for i, c in enumerate(candidati):
    z = np.dot(w, c) + b
    score = sigmoid(z)
    print(f"Candidato {i+1}: z={z:.2f}, score={score:.3f}")

# Output atteso:
# Candidato 1: z=-1.55, score=0.175
# Candidato 2: z=0.10, score=0.525 ← punteggio più alto
# Candidato 3: z=-1.10, score=0.250
# Candidato 4: z=-0.65, score=0.343
```

Risposta alle domande:

15. **Chi ottiene il punteggio più alto?** Il candidato 2 (8 anni esperienza, 3 lingue). L'esperienza pesa molto ($0.4 \times 8 = 3.2$ vs $0.4 \times 2 = 0.8$).
16. **Il modello premia più l'esperienza o il voto?** L'esperienza. Il voto ha peso 0.01 (molto piccolo): la differenza tra 70/110 e 110/110 vale solo 0.4 punti, mentre la differenza tra 2 e 8 anni vale 2.4 punti.
17. **Per far emergere il neolaureato:** aumenta il peso del voto (es. 0.05) e riduci quello dell'esperienza (es. 0.2). Cambio **b** verso valori meno negativi per punteggi medi più alti.

Esercizio 5.2 — Backpropagation manuale su XOR

```
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([[0],[1],[1],[0]], dtype=float)

np.random.seed(1)
W1 = np.random.randn(2, 3) * 0.5
b1 = np.zeros((1, 3))
W2 = np.random.randn(3, 1) * 0.5
b2 = np.zeros((1, 1))
lr = 0.5

sigmoid = lambda z: 1 / (1 + np.exp(-z))
sig_d = lambda a: a * (1 - a)
```

```

losses = []
for step in range(5000):
    a1 = sigmoid(X @ W1 + b1)
    a2 = sigmoid(a1 @ W2 + b2)
    loss = np.mean((a2 - y)**2)
    losses.append(loss)

    dL_da2 = 2 * (a2 - y) / len(X)
    dL_dz2 = dL_da2 * sig_d(a2)
    dL_dW2 = a1.T @ dL_dz2
    dL_db2 = dL_dz2.sum(axis=0, keepdims=True)
    dL_da1 = dL_dz2 @ W2.T
    dL_dz1 = dL_da1 * sig_d(a1)
    dL_dW1 = X.T @ dL_dz1
    dL_db1 = dL_dz1.sum(axis=0, keepdims=True)

    W2 -= lr * dL_dW2; b2 -= lr * dL_db2
    W1 -= lr * dL_dW1; b1 -= lr * dL_db1

print(f"Loss finale: {loss:.6f}")
for xi, yi, pi in zip(X, y, a2):
    print(f"  {xi.astype(int)} → vero={yi[0]:.0f}, predetto={pi[0]:.3f}")

```

Risposta alle domande:

18. `lr=0.1`: converge molto lentamente, servono >20.000 step. `lr=2.0`: può divergere o oscillare violentemente — il loss spike è visibile.
19. **2 neuroni nascosti**: di solito non basta per XOR con questa inizializzazione. XOR richiede almeno 2 neuroni nascosti, ma con inizializzazione sfavorevole il training può bloccarsi in un minimo locale.
20. **Terzo layer**: di solito converge più velocemente e in modo più robusto, perché ha più parametri e più "strade" per trovare la soluzione.

Capitolo 6 — Machine Learning Classico

Esercizio 6.1 — Confronto sistematico

```

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
import numpy as np

X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

modelli = {
    'Logistic Regression': Pipeline([('s', StandardScaler()),
                                     ('m', LogisticRegression(max_iter=1000))]),

```

```

    'Decision Tree':      DecisionTreeClassifier(max_depth=5, random_state=42),
    'Random Forest':     RandomForestClassifier(n_estimators=100, n_jobs=-1,
random_state=42),
    'Gradient Boosting': GradientBoostingClassifier(n_estimators=100,
random_state=42),
    'SVM':               Pipeline([('s', StandardScaler()), ('m',
SVC(probability=True))]),
}

print(f"{'Modello':<25} {'AUC media':>10} {'Std':>8}")
print("-" * 45)
for nome, mod in modelli.items():
    scores = cross_val_score(mod, X, y, cv=cv, scoring='roc_auc', n_jobs=-1)
    print(f"{'nome':<25} {scores.mean():>10.4f} {scores.std():>8.4f}")

# Output tipico:
# Logistic Regression    0.9943    0.0022 ← sorprendentemente buona
# Decision Tree         0.9258    0.0151
# Random Forest          0.9961    0.0021
# Gradient Boosting      0.9967    0.0019
# SVM                   0.9973    0.0016 ← spesso il migliore su questo dataset

```

Cosa impari: Su breast cancer (dati ben normalizzati, feature rilevanti) la Logistic Regression è quasi al livello di Random Forest e SVM. La complessità aggiuntiva non sempre vale il costo. Usa sempre una baseline semplice prima di complicarti la vita.

Capitolo 7 — Ottimizzatori

Esercizio 7.1 — Confronto ottimizzatori

```

import torch, torch.nn as nn
import matplotlib.pyplot as plt

torch.manual_seed(42)
X = torch.randn(500, 10)
y = (X[:, 0] * 2 - X[:, 1] + torch.randn(500) * 0.5).unsqueeze(1)
X_tr, y_tr = X[:400], y[:400]

def crea(): return nn.Sequential(nn.Linear(10,64), nn.ReLU(), nn.Linear(64,32),
nn.ReLU(), nn.Linear(32,1))
loss_fn = nn.MSELoss()

configs = {
    'SGD (lr=0.01, mom=0.9)': lambda p: torch.optim.SGD(p, lr=0.01, momentum=0.9),
    'Adam (lr=1e-3)':         lambda p: torch.optim.Adam(p, lr=1e-3),
    'AdamW (lr=1e-3)':        lambda p: torch.optim.AdamW(p, lr=1e-3,
weight_decay=0.01),
    'RMSProp (lr=1e-3)':      lambda p: torch.optim.RMSprop(p, lr=1e-3),
}

storia = {}
for nome, crea_opt in configs.items():
    mod = crea()
    opt = crea_opt(mod.parameters())
    losses = []

```



```

for _ in range(200):
    pred = mod(X_tr)
    loss = loss_fn(pred, y_tr)
    opt.zero_grad(); loss.backward(); opt.step()
    losses.append(loss.item())
storia[nome] = losses
print(f"{nome}: loss finale = {losses[-1]:.4f}")

# Output tipico:
# SGD (lr=0.01, mom=0.9):  loss finale ≈ 0.28
# Adam (lr=1e-3):          loss finale ≈ 0.27
# AdamW (lr=1e-3):        loss finale ≈ 0.27
# RMSProp (lr=1e-3):      loss finale ≈ 0.28

```

Risposte:

21. **Chi converge più velocemente nei primi 50 step?** Adam e AdamW — grazie al momento adattivo scendono più rapidamente nella fase iniziale.
22. **Chi raggiunge la loss più bassa dopo 200 step?** Tutti molto simili. La differenza è marginale su problemi semplici.
23. **SGD è davvero peggio?** In questo senso sì: converge più lentamente e richiede più tuning del learning rate. Ma su reti CNN per visione, SGD+momentum spesso generalizza meglio di Adam.
24. **Adam con lr=0.1:** la loss esplode o oscilla — troppo alto per Adam che è già aggressivo per natura.

Progetto di Fine Volume 1 — Rilevamento Frodi

Guida alla diagnosi dei risultati

Se la tua AUC-PR è < 0.40: Il modello non sta imparando il pattern delle frodi. Verifica: stai usando `class_weight='balanced'` o `scale_pos_weight`? Il preprocessing è corretto? Prova con XGBoost che gestisce meglio i dataset sbilanciati.

Se la tua AUC-PR è tra 0.40 e 0.65: Il modello impara qualcosa ma non abbastanza. Prova: aggiungere feature derivate (es. `costo_per_gb`), aumentare `n_estimators` in Random Forest, usare SMOTE sul training set.

Se la tua AUC-PR è > 0.65: Ottimo risultato su un problema difficile. La soglia ottimale probabilmente non è 0.5 — usa la curva PR per trovarla.

Confronto con i risultati attesi:

Modello	AUC-PR tipica (dataset sintetico)
Logistic Regression	0.55–0.70
Random Forest	0.65–0.80
XGBoost	0.70–0.85
Neural Network	0.68–0.83

Risposta alla domanda su SMOTE: SMOTE migliora quasi sempre il recall (cattura più frodi) ma può peggiorare la precision (più falsi allarmi). L'AUC-PR complessiva può migliorare o peggiorare a seconda del dataset. Il messaggio chiave: SMOTE va applicato SOLO al training set e i suoi effetti vanno misurati, non assunti.

Risposta su ensemble manuale: Mediare le probabilità di 3 modelli diversi (LR + RF + XGB) tipicamente migliora l'AUC-PR di 1–3 punti rispetto al singolo migliore. Il motivo: i modelli fanno errori diversi, e la media li cancella parzialmente.

Fine delle soluzioni del Volume 1. Le soluzioni del Volume 2 si trovano nell'Appendice E — Volume 2. Le soluzioni del Volume 3 si trovano nell'Appendice E — Volume 3.